

Verification Continuum™

Verdi® AMS Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	8
Synopsys Statement on Inclusivity and Diversity	9
1. Overview	10
2. Setting Up the AMS Debug	11
Prerequisites	11
Generating the Simulation Result	12
Loading the Design and Simulation Result into Verdi	12
Compiling a Mixed-Signal Design	12
Compiling and Importing a SPICE Design	13
Compiling With the SPICE Unified Front-End Flow	14
Support Spectre Format Probes in Unify Dumping	14
Loading FSDB Files	14
Setting Up the AMS Cosimulation Debug	15
AMS Configuration Widget for Pure Digital Designs	17
The Choose Tab	18
The A/D Tab	19
The E/R Tab	21
The Resistances Tab	22
Extra Options	23
An AMS Configuration Widget for Mixed Signal Designs	24
The Swap View Tab	27
3. Interactive Simulation Debugging of Mixed-Signal Designs	29
Performing Interactive Simulation Debug of Mixed-Signal Designs	29
Getting Started	30
Setting the Simulator Type in Preference Page	32
Points to Note	33
Using the Interactive Watch Tab	33
Annotating Values	35
Dumping Analog Signals	35

Contents

Dumping Analog Scope	37
Dumping Batch Signals	38
Using Force/Release Node Voltage	38
Saving or Restoring Simulation Session or State	38
Using the Interactive Console	41
CustomSim Signal Name Format	41
Supporting Verilog-AMS in the Interactive Debug Mode	42
Limitations	42
VCS/CustomSim	42
CustomSim/PLI	42
Verdi AP	42
Dumping Analog Signals in FSDB File in VCS-CustomSim Co-simulation Flow . . .	43
Dumping of the Analog/Digital Signals into an FSDB File	43
Usage Example	44
Dumping Current Probes Using the UCLI Command	45
Performing Merge Dumping	45
Usage Example for Merge Dumping	46
Limitation	47
Dumping Automatic Inserted Connect Modules (AICM) Signals in the Verilog-AMS Flow	47
Enabling Through Net Dumping Information in FSDB File	48
Use Model	48
Limitations	49
Debugging in Source Code Pane	49
Tracing Signals	50
Active Annotation	51
Interface Elements Enhancements	52
Interface Element Browser in nTrace	52
Use Model	53
Filter Types	53
Interface Element Nets in nSchema	54
Use Model	54
Tooltip Format	54
RMB Menu Interface Element Information	56
Viewing Interface Elements	57
Editing the Interface Elements in Verilog-AMS Under Interactive Debug Mode in the nTrace Window	59
The A/D Tab	62
The E/R Tab	63
The E/N Tab	64

Contents

Supporting Verilog-AMS in the nTrace Window	65
Interface Element in nWave	66
Interface Element Property Table	69
IE Property Table Functions	71
Tcl Commands	78
Debugging in Waveform View	78
Supporting Verilog-AMS in the nWave Window	81
Dumping of nWave Signals	82
Supporting Unknown (X) and Inactive (Z) States for WReal Objects	82
Viewing Runtime Interface Elements in nWave	83
Limitations	84
Viewing Force/Release Events in nWave	84
Displaying Force/Release Events in the Value Pane	85
Viewing Force/Release Events in the Waveform Pane	85
Searching by Forced/Deposited/Released Events	86
Reporting Selected Signals	86
Debugging in the Schematic View	87
Show Focus Connection	90
Show Current Debug Focus Connection	91
Block Information	94
Current Information	94
Voltage Information	95
I/O Port Information	96
Preference Setting for Show Current Debug Focus Connection	96
Viewing Interface Elements in nSchema	96
Viewing Runtime Interface Elements in nSchema	98
Editing the Interface Elements in Verilog-AMS Under Interactive Debug Mode in the nSchema Window	100
Viewing Through-Nets in nSchema	101
Color-Coding Signals and Blocks in nSchema	102
Mapping Symbols and Models in nSchema	104
Support for Verilog-AMS in the nSchema Window	108
Debugging in WaveView	109
Prerequisites	109
Invoking WaveView	110
Adding Signals to WaveView	112
Invoking WaveView and Loading FSDB File Automatically	113
WaveView Tab Layout Limitation	114
Synchronizing Cursor and Viewing Range from nWave to WaveView	114

Contents

Updating WaveView Waveform in Verdi Interactive Mode	115
Limitations	115
Tracing Signals in the Temporal Flow View	115
Filtering Signals Based on Signal Types in nTrace	117
Mapping the Port Configuration	117
Limitations	119
Modifying the Analog Configuration During Runtime (Using ace reread)	119
Prerequisite	120
Use Model	120
Example	121
Limitations	122
Tracing Results of Connect Modules in the AMS Designs	122
IE_All_Only Option for MSV Common Dump	122
Use Model	123
Example	123
Limitations	123
Event-Based Real-Time View Swapping (RTVS)	124
Licensing Requirements	124
Use Model of Event-Based RTVS Configurations	124
Arguments	124
Example	125
User Interface Changes in Verdi	126
nTrace	126
nSchema	127
nWave	131
Limitations	131
4. Interactive Simulation Debugging of Verilog-A Designs	132
Compiling Mixed-Signal Design	132
Compiling Verilog-A Blocks	132
Performing Interactive Simulation Debug of Verilog-A Mixed-Signal Design	133
Getting Started	134
Setting the Simulator Type in the Preference Page	135
Using the Interactive Watch Tab	135
Limitations	137
Annotating Values	137
Limitations	138

Contents

Dumping Verilog-A Signals	139
Dumping Nodes	139
Dumping Branches	140
Methods for Dumping Signals	140
Saving or Restoring Simulation Session or State	141
Debugging Verilog-A Design in the Source View	142
Setting Preferences	144
Active Annotation	144
Dragging and Dropping Verilog-A Signals	146
Viewing the Signal List	146
Limitations	147
Debugging Verilog-A Designs in the Waveform View	147
Drag and Drop to nWave	147
Limitations	149
Drag and Drop from nWave	149
Dumping	150
Batch Dumping	150
Debugging Verilog-A Designs in the Schematic View	150
Color-Coding Signals in nSchema	152
Tracing Verilog-A Designs in the Temporal Flow View	153
Tracing Verilog-A Signals	154
Creating Verilog-A Signals in TFV	155
Synchronization with nWave, nTrace, and nSchema	155
Dragging and Dropping Signals	156
Force debug for Verilog-A signals	156
Limitation	156
 5. Known Problems and Limitations	 157

Preface

The *Verdi AMS Debug User Guide* provides visibility into design hierarchy and source code for both analog and digital designs.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

- You can call for support at:

<https://www.synopsys.com/support/global-support-centers.html>

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then please provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Overview

The Verdi® Automated Debug Platform provides a unique approach to analog mixed-signal (AMS) verification that enables engineers to debug complex AMS designs. AMS debug features provide visibility into design hierarchy and source code for both analog and digital designs. When digital and analog FSDB files are loaded, you can view and debug analog signals through the annotated source code, schematic and waveform. This document focuses on the VHDL/Verilog-SPICE Mixed-Signal design scope.

2

Setting Up the AMS Debug

This chapter contains the following topics:

- [Prerequisites](#)
- [Generating the Simulation Result](#)
- [Loading the Design and Simulation Result into Verdi](#)
- [Compiling a Mixed-Signal Design](#)
- [Loading FSDB Files](#)
- [Setting Up the AMS Cosimulation Debug](#)

Prerequisites

The following are the prerequisites to use the mixed-signal debug features in the Verdi platform:

- Use the Verdi platform with version J-2014.12-2 and later.
- Generate the analog FSDB file by using the `CustomSim` simulator with version 2014.09-SP3 in the VCS cosim flow.
- Generate the digital FSDB file by using VCS simulator with version J-2014.12-1 and later in the VCS cosim flow.
- Compile your design using the `vericom/vhdlcom` and `spicom` utilities.

The following sections describe how you can generate the FSDB and KDB files.

- [Generating the Simulation Result](#)
- [Loading the Design and Simulation Result into Verdi](#)
- [Compiling a Mixed-Signal Design](#)
- [Loading FSDB Files](#)
- [Setting Up the AMS Cosimulation Debug](#)

Generating the Simulation Result

Perform the following steps to run the simulation and generate the waveform of your mixed-signal design into Verdi:

- Use the `set_waveform_option -format FSDB` option inside the `CustomSim` command file to have `CustomSim` generate an FSDB output with analog signals¹.
- Add the `-ad` option in VCS command line to compile your AMS design². For example:

```
//Compile and analysis your design
%> vcs digital_design_file(s) -ad
    [=mixed-signal_control_file] [vcs options]

//Execution the simulation
%> simv [vcs run-time options]
```

Executing the simulation command generates two FSDB files. One includes digital hierarchy and signals, and another includes analog hierarchy and signals.

Loading the Design and Simulation Result into Verdi

The **Import Design from Library** feature is supported to load your mixed-signal design into the Verdi platform. Compile your design into the Verdi KDB with both digital and analog hierarchy/signals. Use the `spicom` utility to compile the SPICE/Spectre/Eldo or any other analog models into the netlist model.

The following sections describe how to compile your design with compilers, and how to load KDB and FSDB files into the Verdi platform in detail.

Compiling a Mixed-Signal Design

The following sections describe how to compile your design with `spicom` and how to load KDB into Verdi.

- [Compiling and Importing a SPICE Design](#)
- [Compiling With the SPICE Unified Front-End Flow](#)
- [Support Spectre Format Probes in Unify Dumping](#)

1. For details on the configuration commands, see the *CustomSim User Guide*.

2. For details on the compilation and simulation options, see the *Mixed-Signal Simulation User Guide*.

Compiling and Importing a SPICE Design

When your mixed-signal design includes SPICE and Spectre format, use the `spicom` utility to compile the analog modules of your design. If your design files include Verilog formats (such as, Verilog-A), `spicom` automatically invokes `vericom` to compile these files. The compile log files of `spicom` and `vericom` are separated.

To compile and load your design into Verdi, perform the following steps:

1. Use the `vericom` utility to compile Verilog modules in your design. Its syntax is as follows:

```
%> vericom <your_verilog_design> [vericom_options]
```

2. Use the `vhdlcom` utility to compile VHDL modules in your design. Its syntax is as follows:

```
%> vhdlcom <your_vhdl_design> [vhdlcom_options]
```

3. Use the `spicom` utility to compile analog modules in your design.

You need to add the following option to the `spicom` command line:

- `-ad`: Loads the simulation setup file.

Additionally, the following options are optional:

- `-topname`: Specifies a name for the top cell. The default value is *topcell*.
- `-lib`: Specifies the library to save your design. The default value is *work*.

For example, to use the AD file as the input, use the following command:

```
% spicom -ad vcsAD.init
```

This command creates the netlist type data in the design created. The analysis statement for simulation (for example, `ALTER`, `PRINT`, `DATA`, and so on) is not compiled.

4. Use the `vhdlcom` utility to compile VHDL modules in your design. For example:

```
% vhdlcom <your_vhdl_design> [vhdlcom_options]
```

5. Add the `-msv` option into the Verdi command line to enable mixed-signal debug features. For example:

```
% verdi -top <your_design_top> -msv
```

6. To load the specified report file of interface elements, use the `-msvDir` option if you have the report file of the interface elements. For example:

```
%> verdi -top <your_design_top> -msv -msvDir simv.msv
```

Note:

Verdi supports loading only one report file when loading a design into Verdi.

Compiling With the SPICE Unified Front-End Flow

You can compile SPICE designs in Verdi by invoking `vlogan`, `vhdlan`, or `vcs`, and specifying the `-kdb` option along with the AD file.

Add the `-kdb` option on the VCS command line as follows:

```
%>vcs -kdb -ad=vcsAD.init
```

VCS internally invokes `spicom` with this command line to analyze the `vcsAD.init` file, and the KDB model of SPICE is generated.

When you specify the `-kdb` option, the `$VERDI_HOME` variable must be set.

Support Spectre Format Probes in Unify Dumping

Verdi unify dumping supports spectre format. Dump spectre format only when the user specifies it. You get this information from the simulator. Then analog simulator provides spectre format probes to dumper and dumper then selects dump corresponding probes with the dumping option into FSDB.

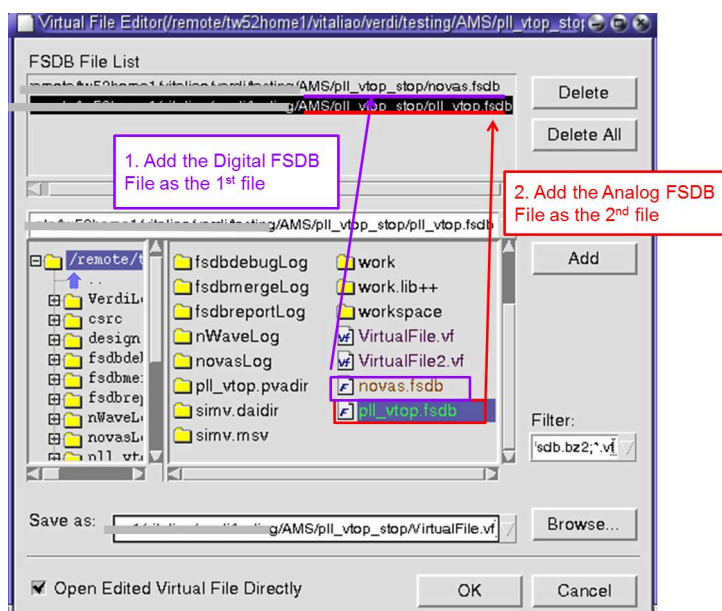
Loading FSDB Files

To debug the mixed-signal design with the simulation result in Verdi, you need to load a virtual file that includes digital and analog FSDB files into the *nWave* window.

To create the virtual file which includes digital and analog FSDB files, perform the following steps:

1. Click the *nWave*  icon to invoke the *nWave* window.
2. Use the **File -> Edit Virtual File** option to invoke the *Virtual File Editor* form.
3. In the *Virtual File Editor* form, double-click the digital FSDB file as the first FSDB in the virtual file. Then, double-click the analog FSDB file as the second FSDB file.

Chapter 2: Setting Up the AMS Debug Setting Up the AMS Cosimulation Debug



4. Specify the virtual file name in the **Save as** field and click **OK**.
5. Both digital and analog FSD files are loaded into the *nWave* window through the virtual file. The time-units of the digital and analog FSD files are synchronized. The master scope is the digital FSD file.

Setting Up the AMS Cosimulation Debug

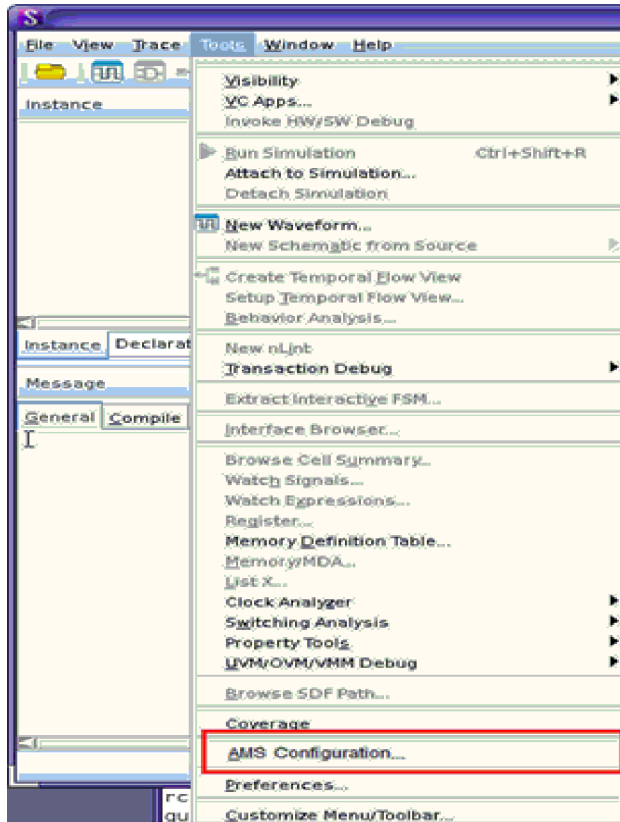
You can set up the cosimulation debugging in the Verdi AMS using the GUI with the following capabilities:

1. Swap view for instances which have multiple views.
2. Port mapping in case of port mismatches between views.
3. Set up parameters for interface elements.
4. Save the configuration in the AMS control file. (*vcsAD.init* as default)

All these features are discussed in detail in the following sections. To avail this capability, set the environment variable `VERDI_ENABLE_AMS_SETUP`.

You can edit the control file for a design; use the menu option **AMS Configuration** under the **Tools** menu as shown in the following figure:

Figure 1 AMS Configuration Menu Item



You can also use the AMS toolbar item to access the configuration widget as shown in the following figure:

Figure 2 AMS Configuration Toolbar Item



Select the **AMS Configuration** to open the widget which is used to setup the control file to perform AMS co-simulation in Verdi.

There are two different configuration widgets, one for a pure digital design and another for a mixed digital and analog design as described below.

AMS Configuration Widget for Pure Digital Designs

If you have a pure digital design, Verdi displays an **AMS Configuration** widget containing the following five tabs which appear as shown in the following figure.

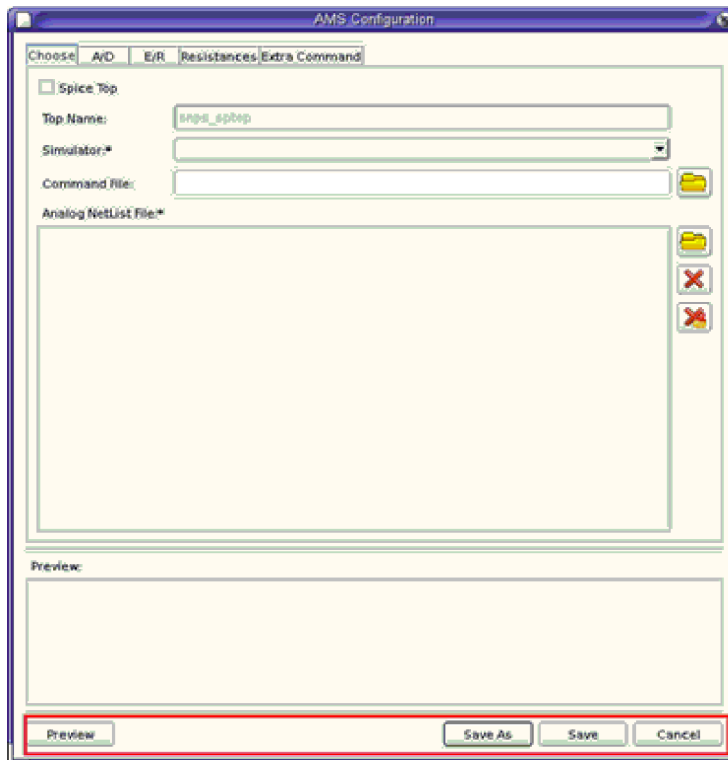
For more information on each tab and its functionality, click on the each tab name in the following list:

- [The Choose Tab](#)
- [The A/D Tab](#)
- [The E/R Tab](#)
- [The Resistances Tab](#)
- [Extra Options](#)

The following functions are common to all the five tabs in the configuration widget as shown in the following figure:

- **Cancel:** Discards current settings and closes the widget.
- **Save/Save As:** Saves the current settings to a file after performing some error checks. A warning message is displayed in the following scenarios:
 - **Simulator** field is empty
 - **Analog Netlist File** field is empty
 - **Spice Top** checked but the **Top Name** field is empty
 - **Cell**, **port**, and **node** fields are empty for the A/D interface element
 - **Node** field is empty for the E/R interface element
 - **Node** or **r** field is empty for node resistance override
 - **Node** or **file** field is empty for the resistance map file
- **Preview:** Shows the commands on the preview widget.

Figure 3 AMS Configuration Widget

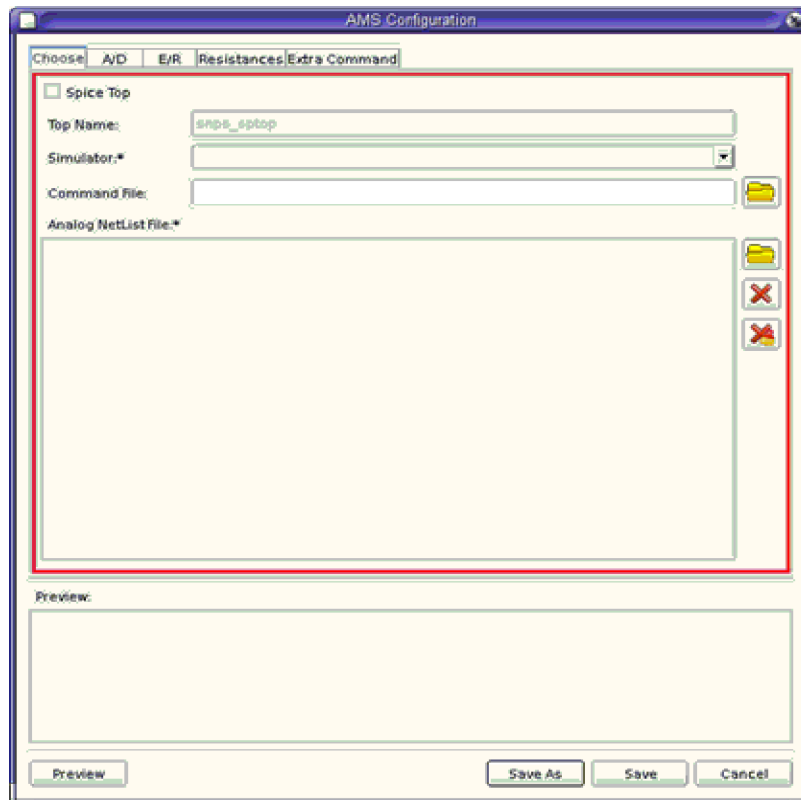


The Choose Tab

The *Choose* tab enables you to setup the `choose` and `spice_top` options as shown in figure [The Choose Tab](#). The * symbol indicates the mandatory fields. Following are the descriptions of each field in the tab:

- **Simulator:** Specifies analog simulator type.
- **Analog Netlist File:** Specifies analog netlist file.
- **Command File:** Specifies analog simulator command file.
- **Spice Top:** Specifies whether the top-level of a design is a SPICE design.
- **Top Name:** Specifies the SPICE top name, the default name is `snps_sptop`.

Figure 4 The Choose Tab



The A/D Tab

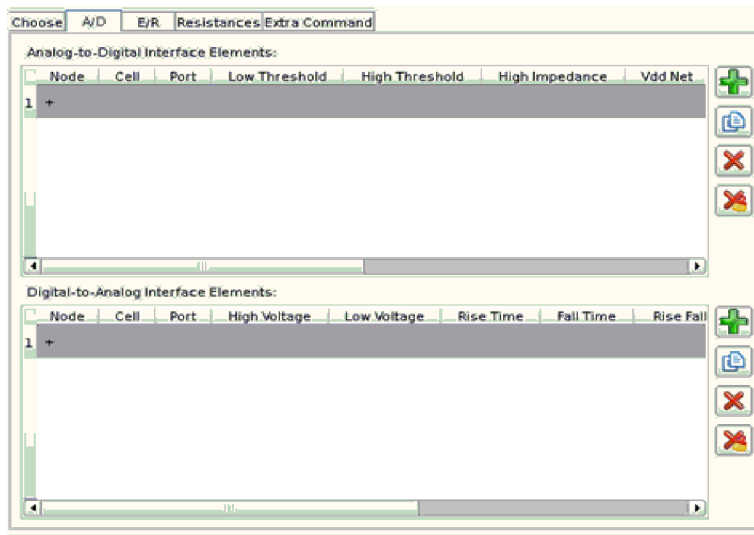
The **A/D** (Analog to Digital and Digital to Analog) tab enables you to set up `a2d` and `d2a` commands.

The **A/D** tab contains two kinds of interface elements: `a2d` and `d2a` as shown in figure [The A/D Tab](#). The definition of each of these interface elements is as follows:

- **Analog to Digital Interface Elements:**
 - **Node:** Specifies hierarchical path of the instance.
 - **Cell:** Specifies the cell name.
 - **Port:** Specifies the port name.
 - **Low Threshold:** Specifies the low threshold values.
 - **High Threshold:** Specifies high threshold values.
 - **High Impedance:** Specifies the high impedance.

- **Vdd Net:** Specifies the dynamic supply.
- **Minimum Voltage:** Specifies minimum voltage for **VDD**.
- **Minimum Voltage Logic:** Specifies the logic value passed to the digital engine when the conversion is disabled.
- **Cap.Load:** Specifies the load capacitance to be inserted at interface net.
- **Digital to Analog Interface Elements:**
 - **Node:** Specifies the hierarchical path of the instance.
 - **Cell:** Specifies the cell name.
 - **Port:** Specifies the port name.
 - **Power Net:** Specifies the PowerNet name.
 - **Rise Fall Times:** Specifies the rise and fall times.
 - **Rise Time:** Specifies the rise time.
 - **Fall Time:** Specifies the fall time.
 - **Delay:** Specifies the delay before the analog transition starts.
 - **Logic-X:** Specifies rules on how a logic X must be translated to a voltage level.
 - **High Voltage:** Specifies the high voltage.
 - **Low Voltage:** Specifies the low voltage.
 - **Vdd:** Specifies the dynamic supply.

Figure 5 The A/D Tab



The E/R Tab

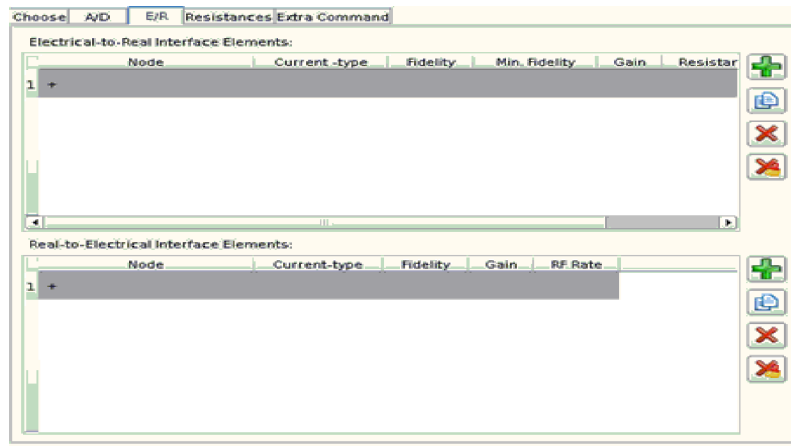
The **E/R (Electrical to Real and Real to Electrical)** tab enables you to set up `e2r` and `r2e` commands.

The **E/R** tab contains two kinds of interface elements: `e2r` and `r2e` as shown in figure [The E/R Tab](#). The definition of each of these interface elements is as follows:

- **Electrical-to-Real Interface Elements (e2r):**
 - **Node:** Specifies hierarchical path to the `e2r` interface.
 - **Type:** Specifies if `e2r` is converting voltage values to digital, or the current through a resistor.
 - **Fidelity:** Specifies the absolute voltage/current threshold value for `e2r` events.
 - **Min.Fidelity:** Specifies the voltage difference between two `e2r` events.
 - **Gain:** Specifies multiplier when converted to digital.
 - **Resistor:** Specifies value or the resistor.
- **Real to Electrical Interface Elements (r2e):**
 - **Node:** Specifies hierarchical path to the `e2r` interface.
 - **Type:** Specifies if the `e2r` converting voltage values to digital, or the current through a resistor.
 - **Fidelity:** Specifies the absolute voltage/current threshold value for `e2r` events.

- **Gain:** Specifies multiplier when converted to digital.
- **RF Rate:** Specifies value of the resistor.

Figure 6 The E/R Tab



The Resistances Tab

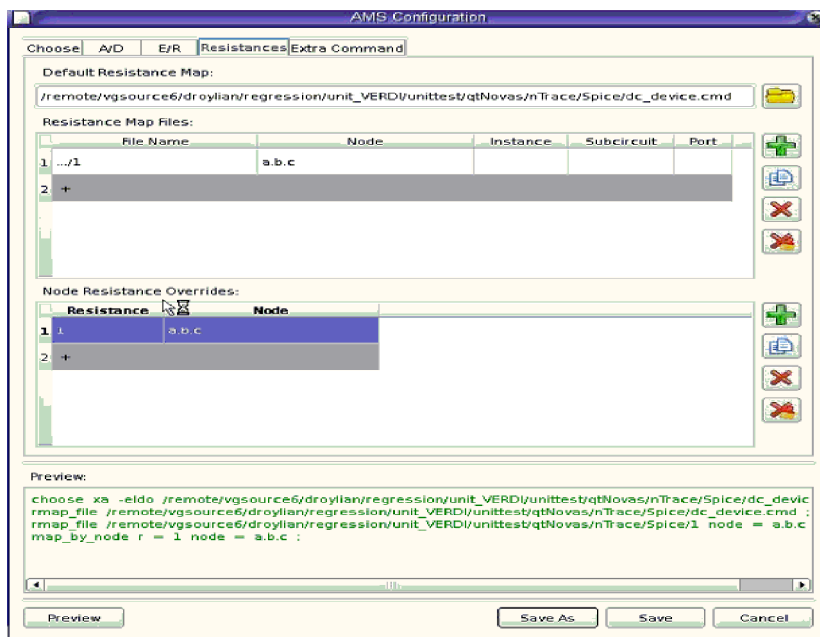
The **Resistances** tab enables you to set up `rmap_file` and `map_by_node` commands.

The following are the descriptions of each field in the tab:

- **Default Resistance Map:** Specifies the default resistance map.
- **Resistance Map Files:** Specifies the resistance map file.
 - **File Name:** Specifies resistance map file.
 - **Node:** Specifies node name.
 - **Instance:** Specifies instance name.
 - **Subcircuit:** Specifies subcircuit name.
 - **Port:** Specifies port name.
 - **Node File:** Specifies node file.
 - **Instance File:** Specifies instance file.
 - **Subcircuit File:** Specifies subcircuit file.
- **Node Resistance Overrides:** Specifies resistance value to the interface resistor.

- **Resistance:** Specifies resistance map file.
- **Node:** Specifies node name.

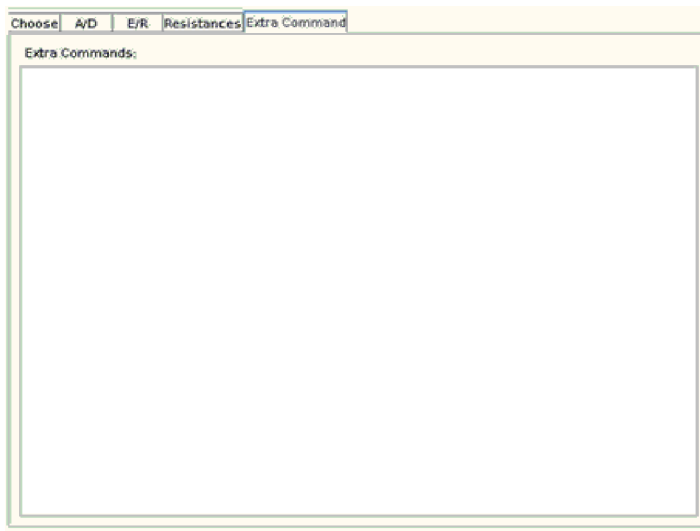
Figure 7 The Resistances Tab



Extra Options

You can specify extra options in the control file using the following tab.

Figure 8 *The Extra Options Tab*



An AMS Configuration Widget for Mixed Signal Designs

If you have a digital and analog mixed design, you can edit the control file using the **Swap View** menu option as shown in the figure [The Swap View Menu Item](#).

You can see this menu options when you right-click on an instance of a mixed signal design that has multiple views as shown in figure [Instances With Multiple Views](#).

The following icons indicate instances having multiple views:

- : Verilog instance has multiple views.
- : Subckt instance has multiple views.
- : VHDL instance has multiple views.

Note:

Module/Cell based swap view is currently not supported.

Selecting the **Swap View** menu option displays the **AMS Configuration** widget containing the following three tabs as shown in figure [The AMS Configuration](#):

- **Swap View**
- **A/D**
- **E/R**

Figure 9 The Swap View Menu Item

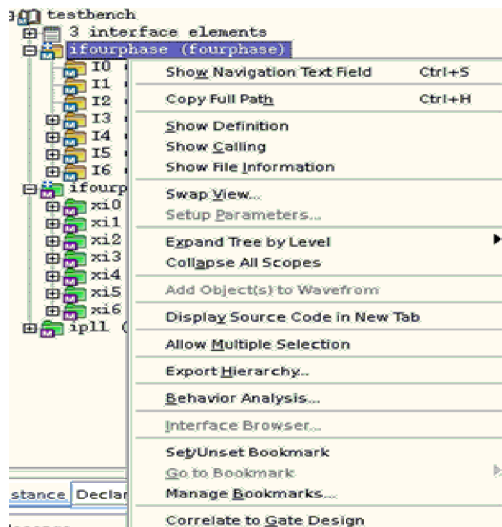


Figure 10 Instances With Multiple Views

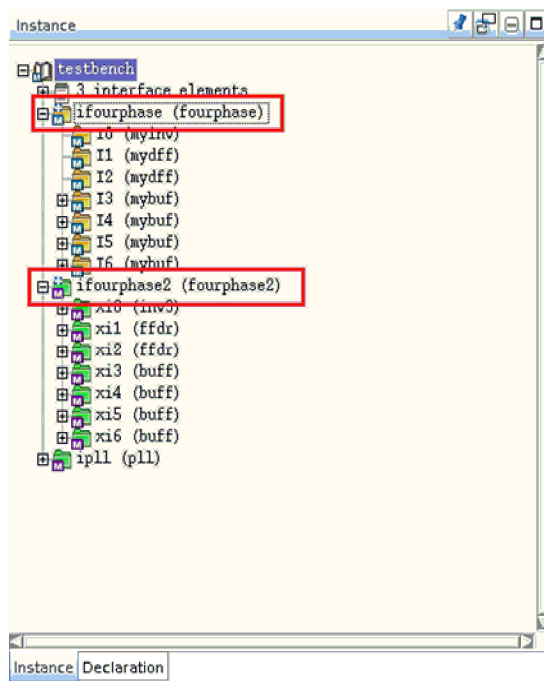
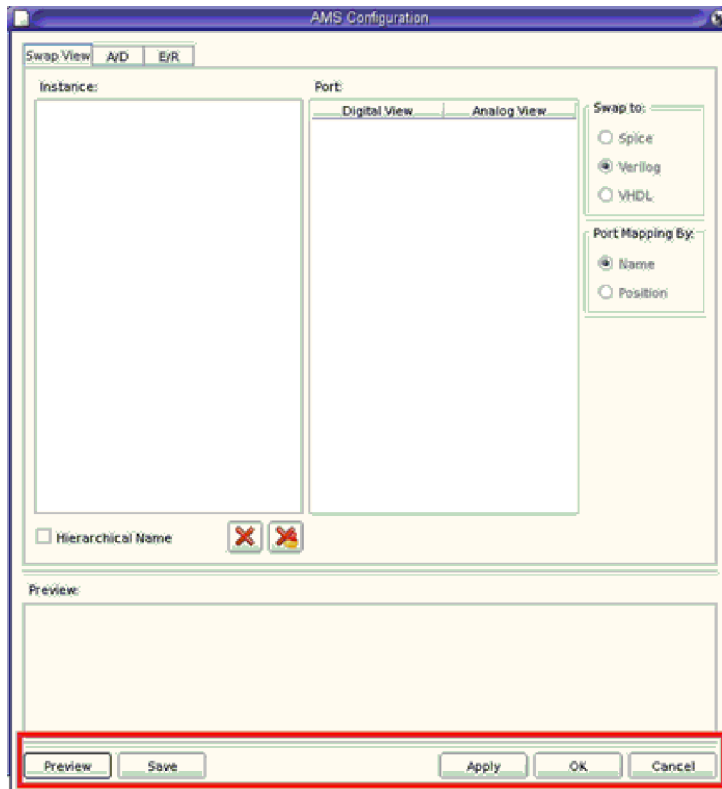


Figure 11 The AMS Configuration



The following functions are common to all the three tabs in the configuration widget:

- **Preview:** Displays a preview of the updates you have made to the control file. The updates are highlighted in blue and additions are highlighted in green and deletions are highlighted in gray as shown in figure [The Preview Tab](#).
- **Add:** Adds the command with the green font color as shown in figure [The Preview Tab](#).
- **Change:** Comments the original command and adds the new command instead of the original one.
- **Delete:** Comments the command with the gray font color as shown in figure [The Preview Tab](#).

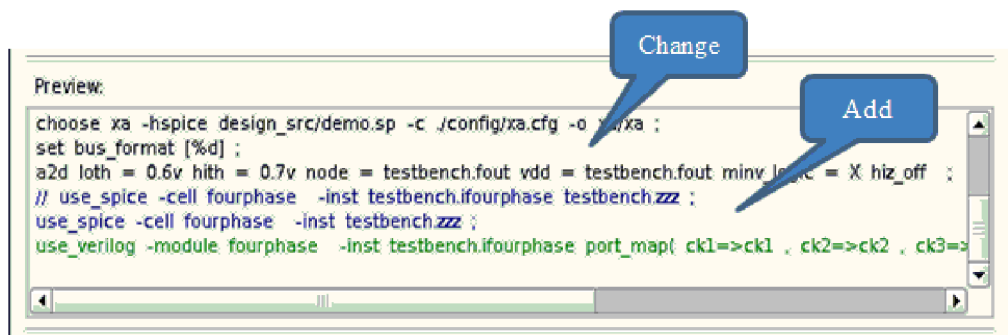
Note:

The command style may differ from the original one. For example, the original is `use_spice -inst x -cell M`, the new option is `use_spice -cell M -inst x`.

- **Save:** Saves the current settings to a file.
- **Apply:** Applies the current setting and then reloads the file.

- **OK:** Applies the new settings and then closes the form.
- **Cancel:** Closes the form.

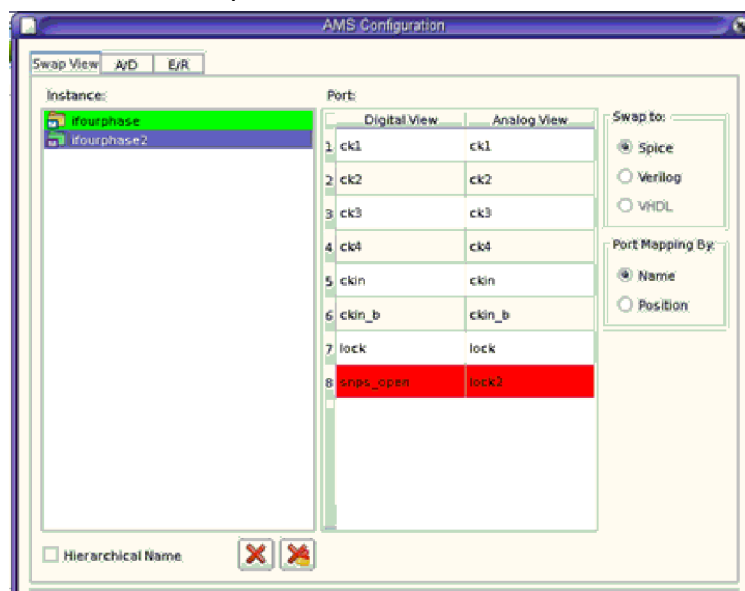
Figure 12 The Preview Tab



The Swap View Tab

The **Swap View** tab enables you to change the view of an instance. You can view SPICE, Verilog, and VHDL views of an instance.

Figure 13 The Swap View Tab



Following are the descriptions of each field in the tab:

- **Instance:** Displays instances with swapped views. It supports the drag and drop operation. If an instance is highlighted in green, that is, no port is mismatching. If it is highlighted in red, that is a port has mismatches. You can also delete the instance. The icon of an instance indicates the status of the current view.
 -  Spice view to Verilog/VHDL View
 -  Verilog view to Spice view
 -  VHDL view to Spice View
- **Hierarchical Name:** Displays the hierarchical name of an instance in the list.
- **Spice:** Swaps to the Spice view of the selected item. The instance is selected if the current item is already in the Spice view.
- **Verilog:** Swaps to the Verilog view of the selected item. The instance is selected if the current item is already in the Verilog view. The instance is disabled if the instance does not have the Verilog view.
- **VHDL:** Swaps to the VHDL view for the selected item. The instance is selected if current item is already in the VHDL view. The instance is disabled if the instance does not have the VHDL view.
- **Port:** Lists the ports of digital and analog views. It supports the drag and drop operation inside the list. You can drag a port and drop it to the same column port to swap its position.
- **Position:** Ports in the same row are mapped; other ports are opened.
- **Name:** Ports that have the same name are mapped; other ports are opened.

The **A/D** and **E/R** tabs are the same as the configuration tabs for digital designs. For more information on **A/D** and **E/R** tabs, see [The A/D Tab](#) and [The E/R Tab](#) sections respectively.

3

Interactive Simulation Debugging of Mixed-Signal Designs

Verdi provides an approach to perform Analog Mixed-Signal (AMS) Interactive Simulation Debugging that enables you to debug complex AMS designs during the simulation.

To perform AMS debugging in the **Interactive Simulation Debug** mode, you must compile mixed-signal designs with VCS and Verdi, configure the simulation setting, and then enable the **Interactive Simulation Debug** mode.

The following sections describe how you can debug analog mixed-signal designs in Verdi:

- [Performing Interactive Simulation Debug of Mixed-Signal Designs](#)
- [Dumping Analog Signals in FSDB File in VCS-CustomSim Co-simulation Flow](#)
- [Dumping Automatic Inserted Connect Modules \(AICM\) Signals in the Verilog-AMS Flow](#)
- [Debugging in Source Code Pane](#)
- [Debugging in Waveform View](#)
- [Debugging in the Schematic View](#)
- [Debugging in WaveView](#)
- [Tracing Signals in the Temporal Flow View](#)
- [Filtering Signals Based on Signal Types in nTrace](#)
- [Mapping the Port Configuration](#)
- [Modifying the Analog Configuration During Runtime \(Using ace reread\)](#)
- [Tracing Results of Connect Modules in the AMS Designs](#)
- [IE_All_Only Option for MSV Common Dump](#)

Performing Interactive Simulation Debug of Mixed-Signal Designs

After the mixed-signal designs are compiled in Verdi, you can interactively debug the design.

The following features are supported with the **AMS Interactive Debug** mode for AMS designs:

- Provides various methods to dump voltage of analog signals.
- Shows the voltage value in the annotation for analog signals in interactive debug mode.
- Shows analog signals in the interactive *Watch* tab.
- Forces/releases the voltage of nodes with the slope that can be specified.
- Saves/restores the simulation session/state.
- The waveform provides to save/restore a signal *.rc* file with analog signals in form of *v(...)* being dumped and restored.
- Interactive console to input CustomSim interactive commands with *ace* preceding and then load UCLI command file to execute it.

The following sections describe the supported interactive simulation debug features:

- [Getting Started](#)
- [Setting the Simulator Type in Preference Page](#)
- [Using the Interactive Watch Tab](#)
- [Annotating Values](#)
- [Dumping Analog Signals](#)
- [Using Force/Release Node Voltage](#)
- [Saving or Restoring Simulation Session or State](#)
- [Using the Interactive Console](#)
- [CustomSim Signal Name Format](#)
- [Limitations](#)

Getting Started

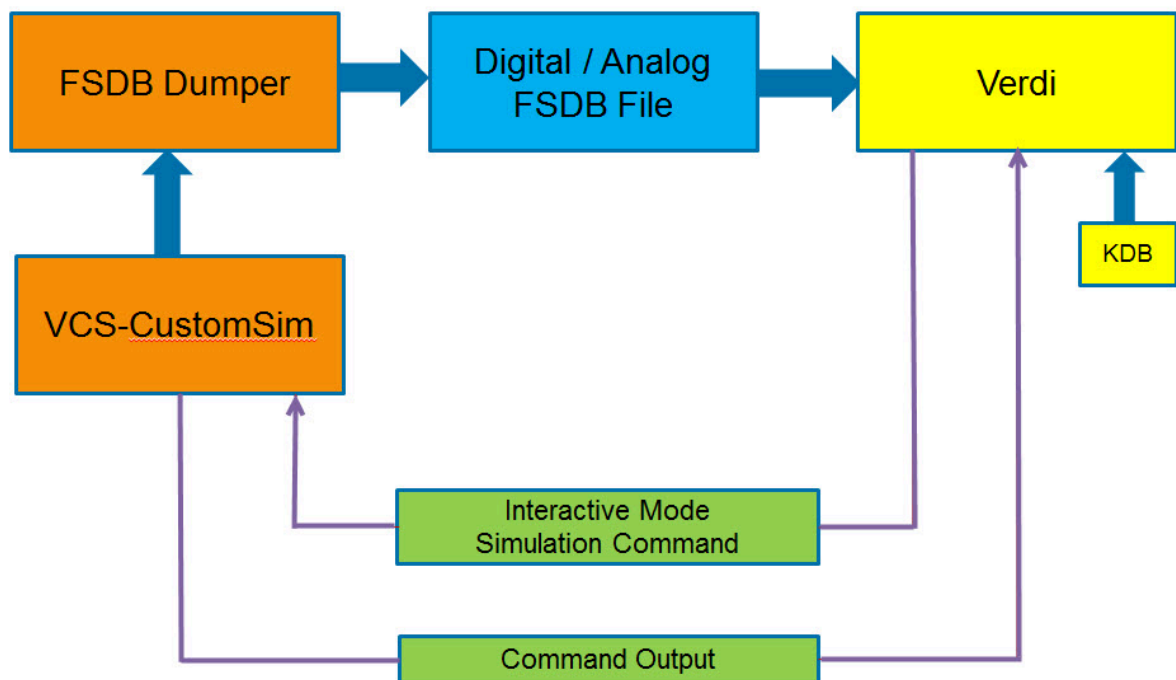
Verdi supports AMS interactive debug mode with the following flow:

1. Prepare the AMS design.
2. Compile the AMS design with VCS and acquire the `simv` that has the debug capability.
For more information, refer to the document of VCS/CustomSim/PLI.

3. Use Verdi to load the AMS design with `cosim` option specified by as follows:
 - Specify `-msv` when importing the design from the command line.
 - In `nTrace`, select **File -> Import Design -> From library -> Options**, toggle on the **Cosim Debug** command and load the AMS design.
4. Select **Tools -> Run Simulation**.

Verdi creates and opens a single FSDB to include the digital and analog signals dumped by the FSDB dumper. The following figure provides an overview of the **Verdi AMS Interactive Mode**.

Figure 14 Overview of Verdi AMS Interactive Mode



You can run AMS interactive debug with the AMS design. However, the simulation is controlled by VCS and following are the limitations that exists in VCS/CUSTOMSIM when running AMS interactive debug:

- Step or next for an analog design is not supported.
- Breakpoints cannot be added into analog design.
- Rewind is blocked for an AMS interactive debug mode.

These limitations cause some menu options and right-click menu options to be disabled.

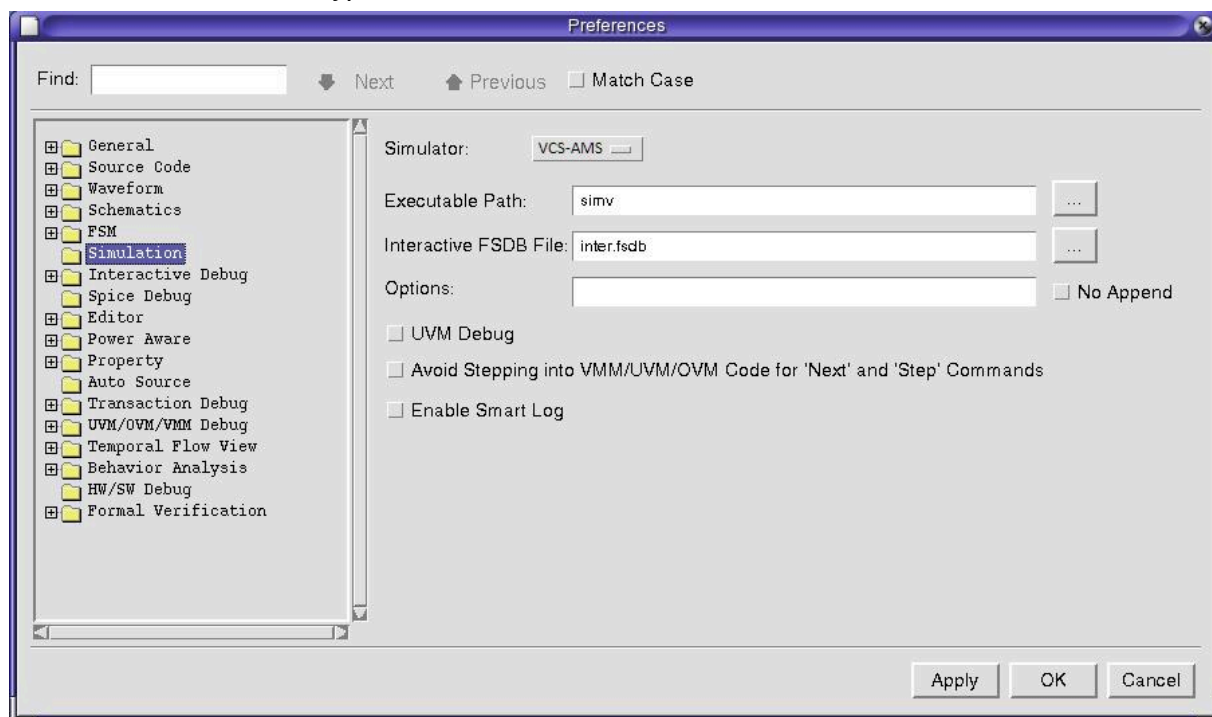
For more information on the simulation and interactive debug commands, see the *Verdi and Siloti Command Reference Guide*.

Setting the Simulator Type in Preference Page

You can specify the simulator in the **Simulator** field of the *Simulation Preferences* form (invoked with **Tools -> Preference -> Simulation -> Simulator** option).

If you specify the `-msv` option in command line, the simulator type in preference is changed to *VCS-AMS*.

Figure 15 New Simulator Type in Preference



- Previously, Verdi had `-simType` option to indicate the simulator type when invoking Verdi. If you specify both the `-msv` and the `-simType` options, then Verdi issues a warning message and honors the `-msv` option.
- Verdi issues a warning message when you invoke the simulation by different simulator types when running AMS interactive debug. For example,
 - You can use the simulator type VCS and run interactive mode with AMS `simv`.
 - You can use the simulator type VCS-AMS and run interactive mode with digital only `simv`.

Points to Note

- Verdi does not support runtime options for the dumper with the command line. You must specify them using the `$fsdbDumpvars()` task; for example, `$fsdbDumpvars("+msv+v")`.
- The Verdi options that must be specified after the `-verdi_opts` option, for example:

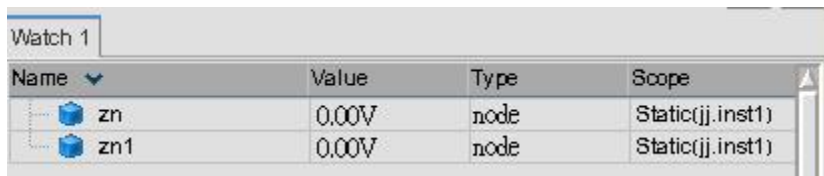
```
./simv -do temp.do -verdi -verdi_opts "-ssf xx.fsdb"
```
- Specifying `-ad_runopt=a2dopt+d2aopt` or `-ucli -ucli2Proc` option is not required in **Interactive Debug** mode.
- The following is an example command to invoke interactive mode for mixed signal co-simulation in Verdi:

```
verdi -simBin simv -i ./simv -verdi
```

Using the Interactive Watch Tab

The Interactive *Watch* tab displays analog signals, ports/nodes, and their voltage value at the current simulation time. You can add analog signals, ports/nodes to the *Watch* tab with the drag-and-drop or the **Add to Watch** right-click menu option, as illustrated in the following figure:

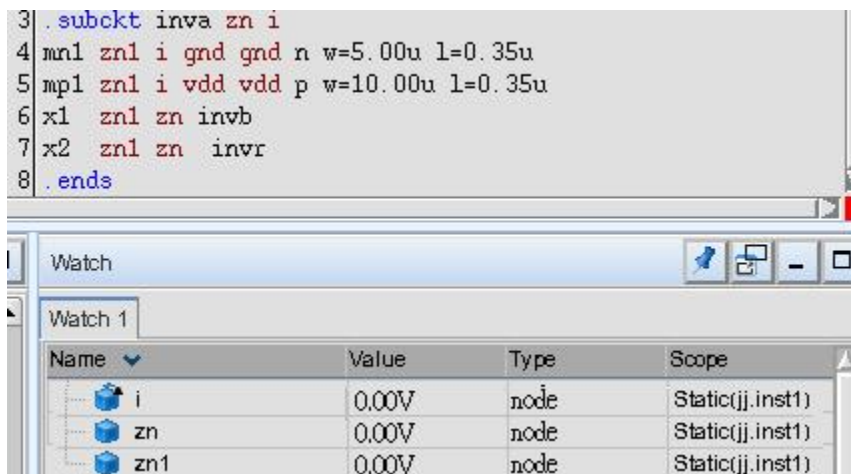
Figure 16 Signals Added to Interactive Watch Tab



Name	Value	Type	Scope
zn	0.00V	node	Static(jj.inst1)
zn1	0.00V	node	Static(jj.inst1)

You can also drag SPICE instance to the interactive *Watch* tab. All analog ports and nodes that are defined in this SPICE instance are added to the *Watch* tab. For example, `jj.inst1` is a SPICE instance. You can drag it to the *Watch* tab. All its ports and nodes are added to the *Watch* tab.

Figure 17 Ports and Nodes Added to Watch Tab



The *Watch* tab provides the following information for analog variables:

- **Name:** The variable name.
- **Value:** The voltage value of the current simulation time. For example, 3.30V.
- **Type:** Analog signal, port, or node.
- **Scope:** Full hierarchical scope.

You can summarize the effectiveness of each right-click options in the interactive *Watch* tab in an analog design.

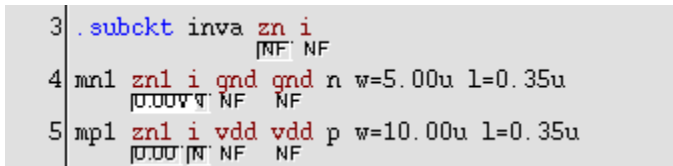
- The following options are disabled with analog signals:
 - Set the Breakpoint
 - Set Radix
 - Add Object to *Watch* Tab
 - Show Declaration, Show in Class Browser, View Object References
 - Dump Object to FSDB file, Dump, Add Object to Waveform
- **Limitation:**
 - The *Watch* tab displays the voltage value. The current value is not supported in the *Watch* tab.

Annotating Values

Voltage values can be annotated to the analog signals in *Source Code* and *nSchema* panes by using the **Source Active Annotation** toggle command (the corresponding shortcut key is 'x').

The voltage value is shown as 3.30V.

Figure 18 Voltage Value Annotation



```
3 .subckt inva zn i
      [NF] NF
4 mnl zn1 i gnd gnd n w=5.00u l=0.35u
      [0.00V] NF NF
5 mp1 zn1 i vdd vdd p w=10.00u l=0.35u
      [0.00V] NF NF
```

Limitation:

This feature has the following limitations:

- The **Tools -> Preference -> Spice -> Annotation** option does not work in interactive debug mode.
- Changing the analog format and precision in the annotation is not supported, if the signal is not dumped earlier. If it is dumped, you can change it from *nWave*.

Dumping Analog Signals

Verdi allows you to dump voltage and current of analog signals. Similar to interactive debug mode with digital design, you can dump the voltage value of an analog signal in AMS interactive debug mode in the following ways:

- Drag the specified analog signal from the *Source Code* window and drop it to *nWave*.
- In *nTrace*, right-click the signal and select the **Add Signal(s) to Waveform** option.
- In *nTrace*, right-click the signal/instance and select the **Dump to FSDB File** option.

In *nTrace*, right-click the signal/instance and select the **Drag** option. In *nWave*, right-click the signal pane and select the **Drag** option. If you select analog signals, Verdi dumps the voltage value of the selected analog signal. If you select analog instances, Verdi dumps the voltage value of the analog signal under selected analog instance.

Figure 19 Dump Menu Item

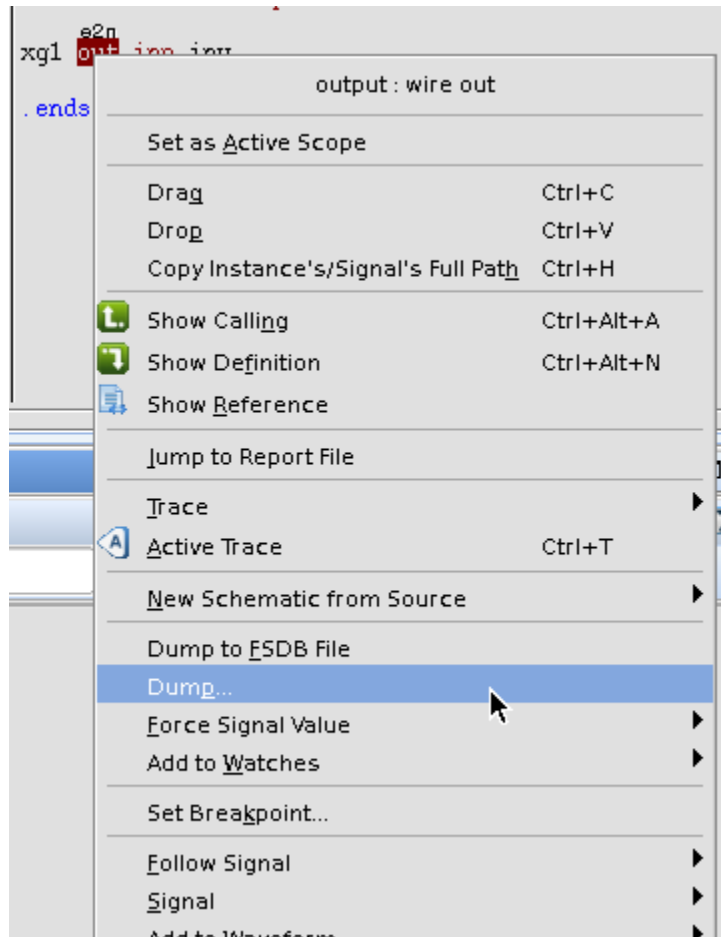


Figure 20 AMS Signal Dump Form

Net/Signal	Device/Circuit	Port	Depth
top.idut.out	top.idut.g1	1	1

☒ Dump Voltage
☒ Dump Current

Add Delete Dump Cancel

Limitation

This feature has the following limitation:

- If you dump analog signals using SPICE statements, CustomSim configuration file, or CustomSim interactive dumping command in split mode, the analog signals are not dumped into the Verdi interactive FSDB files including `.print`, `probe`, `probe_waveform_voltage`, and `iprobe_waveform_current`.

Dumping Analog Scope

You can dump the analog scope by dragging and dropping the sub-circuit or device from *nTrace* to *nWave*. For example, if you drag the `jj.inst1.mn1` device from *nTrace* to *nWave*, all voltage values of analog signals under `jj.inst1` are dumped. The port of `jj.inst1.mn1` is added to *nWave* as before.

You can also dump the analog scope by dragging and dropping the instance from the instant pane to *nWave*. For example, if you drag the `jj.inst1` device from *nTrace* to *nWave*, all voltage values of analog signals under `jj.inst1` are dumped. The port of `jj.inst1` is added to *nWave* as before.

Dumping Batch Signals

You can dump batch signals with the **nWave -> File -> Restore Signals** option that is same as interactive debug mode with the digital design.

Using Force/Release Node Voltage

Verdi provides the *force value* form for SPICE signals to force voltage values. Right-click the specified signals to select the **nTrace -> Force Signal Value -> Set Force** menu option. The *force value* form includes the following fields:

- **Signal name:** Specify the signal to force by entering the name of the specified signal to this field.
 - If the *force value* form is invoked with the **nTrace -> Force Signal Value -> Set Force** right-click menu option, the name of the signal is filled in this field automatically.
- **Value:** Specify the forced voltage value in the field. The unit is *Volt*. The default is the current value.
- **slope value:** When this option is turned *on*, specify the slope value for the analog signal by entering the value to this field. The unit is *second/volt*. The default value is *1ps/volt*. You can specify the time unit such as 1ms, 1us, or 1s. The default of the slope value option is *off*.

Note:

If the time unit is not specified in this field, the time unit is in *seconds*.

Figure 21 Force Value Form



The image shows a screenshot of a software dialog box titled "force value". It contains three input fields: "Signal Name:", "Value:", and "slope value". The "slope value" field has a checked checkbox next to it. The dialog box has a standard Windows-style title bar with a close button.

Saving or Restoring Simulation Session or State

Verdi supports saving/restoring the simulation session/state in interactive mode with the digital design. This feature provides the capability to save the current debug process

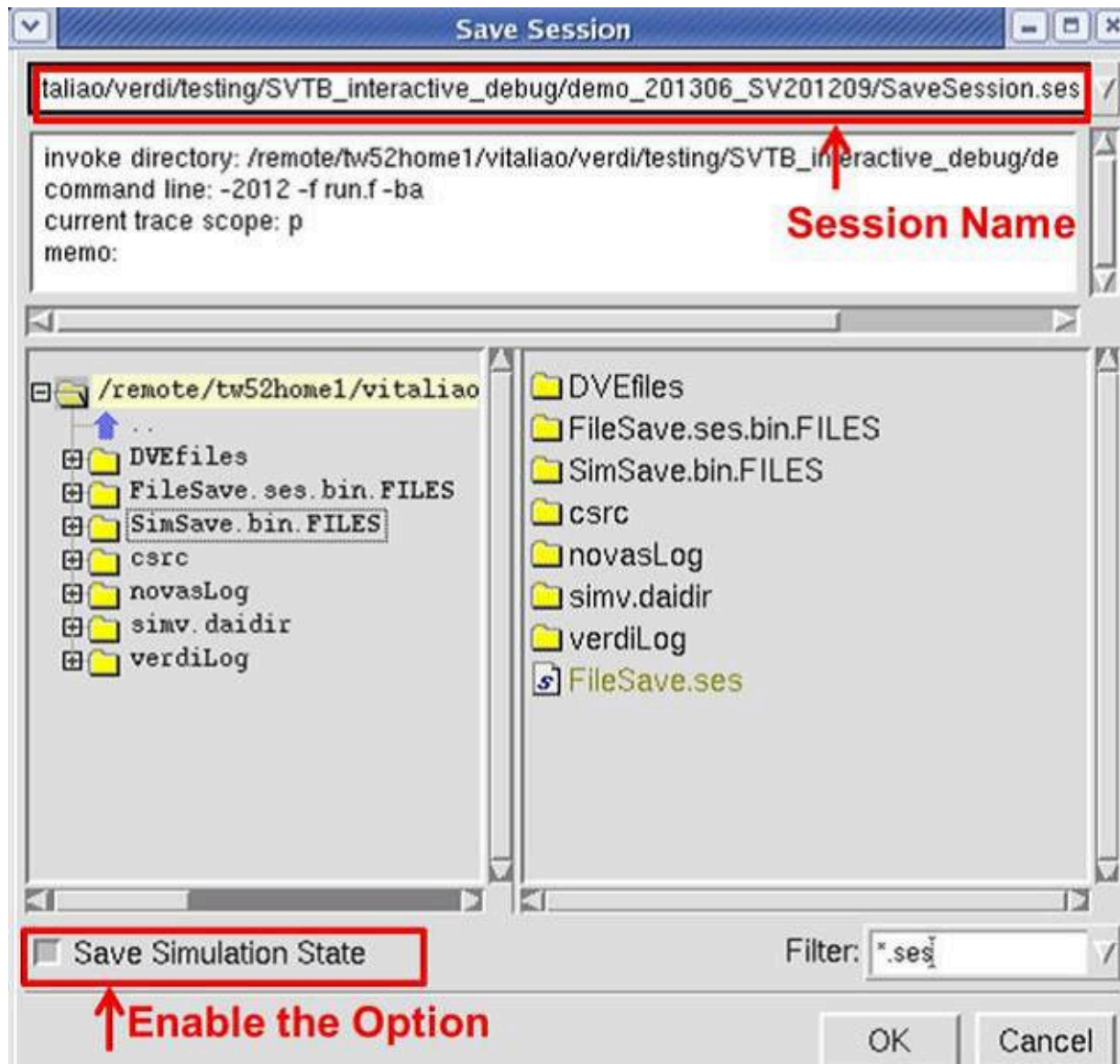
or restore the point to continue the unfinished debug process. Verdi also supports this feature for AMS interactive debug mode. Both simulation states in digital and analog can be saved/restored.

The usage is as same with the digital design. You can use this feature with the following options:

- **nTrace -> Simulation -> Save State**
- **nTrace -> File -> Save Session**

Then, toggle on the **Save Simulation State** option.

Figure 22 Save/Restore Session/State



Use the following environment variable to turn *on* this feature:

```
setenv VCS_ENABLE_ASIR_SUPPORT 1
```

- **Limitation:**

- This feature restores the voltage value of the analog signals that are in the form of `v(...)`.

Using the Interactive Console

Verdi supports to specify the CustomSim interactive command by specifying *ace* before the command from the interactive console. Some of the CustomSim interactive commands is not valid in AMS interactive debug mode. The CustomSim interactive commands that affect the simulation progress are disabled in interactive debug mode.

Note:

This feature does not provide the information to notify the command.

Verdi also provides the `source <ucli command file>` command to load the command file and execute.

Note:

The GUI is not updated when executing commands in the file until *source* is finished.

CustomSim Signal Name Format

Following is the format of the analog signal name with value types:

Table 1 The Format and Description of the Signal:

Signal Name Format	Description
v(node)	Voltage of a node.
vn(instance)	Voltage of a terminal n of an instance where n is an integer greater than or equal to 1; or d, g, s, b for MOSFET and c, b, e, s for BJT.
v(node1, node2)	Expression of the difference voltage of two nodes. The utilities do not do any arithmetic calculations. V(node1, node2) must be presented in the original waveform file already.
i(instance)	Current flowing into the first terminal of an instance.
in(instance)	Current flowing into an instance at terminal n where n is an integer greater or equal to 1; or d, g, s, b for MOSFET and c, b, e, s for BJT.
isub(subckt_port) x(subckt_port)	Total current flowing into the subcircuit port.

Supporting Verilog-AMS in the Interactive Debug Mode

Interactive *Watch* tab supports to show Verilog-AMS signals when running AMS interactive mode. Verilog-AMS signals includes variable, electrical nodes (electrical, voltage, and current) and branch.

Interactive *Watch* tab supports to show analog signals, ports/nodes, and their voltage value at the current simulation time. You can add Verilog-AMS signals to the *Watch* tab with the drag-and-drop or the **Add to Watch** right-click option.

Limitations

This section summarizes all limitations for the AMS debug interactive mode with co-simulation.

VCS/CustomSim

- Unable to run **step** or **next** into analog design.
- **Breakpoints** cannot be added into the analog designs.
- **Rewind** is blocked for AMS interactive debug mode.

CustomSim/PLI

- GUI only supports dumping of the voltage value.
- The **Dump** right-click menu option dumps different value types, such as, the current is not supported.
- If you specify the SPICE dumping task by source code, the CustomSim configuration file, or the CustomSim interactive dumping command, the analog signals are not dumped into the Verdi interactive FSDB file.
- Following commands are not supported:
 1. `.print (...)`
 2. `probe_waveform_voltage`, `iprobe_waveform_current`, and so on.
- No force/release icons are shown for analog signals in *nWave*.

Verdi AP

- The **Tools -> Preference -> Spice -> Annotation** option is not valid in interactive debug mode.
- Changing the analog format and precision in annotation is not supported.

Dumping Analog Signals in FSDB File in VCS-CustomSim Co-simulation Flow

The UCLI dump command enables you to dump analog signals in the FSDB file in the VCS-CustomSim co-simulation environment. The UCLI dump command supports dumping of the hierarchy scope with mixed digital and analog modules, for an object specified in the design.

This is described in detail in the following sections:

- [Dumping of the Analog/Digital Signals into an FSDB File](#)
- [Performing Merge Dumping](#)
- [Limitation](#)

Dumping of the Analog/Digital Signals into an FSDB File

This section describes how you can dump digital signals, analog signals, or both analog and digital signals in the FSDB file. The following steps are prerequisites for FSDB dumping:

1. Set the `VERDI_HOME` variable as follows:

```
% setenv VERDI_HOME <verdi_path>
```

2. Compile the design with the `-debug_access` option, as follows:

```
% vcs -debug_access <file_name>
```

You can now use one of the following ways to invoke Verdi dumper on analog signals:

```
ucli% dump -msv[=on|off]
ucli% dump -file analog_mixed_signal.fsdb -type fsdb
```

OR

```
ucli% dump -file analog_mixed_signal.fsdb -type fsdb -msv[=on|off]
```

You can use the `-msv` option to enable (on) or disable (off) dumping of analog signals throughout the simulation. By default, this option is enabled if on or off is not specified.

The analog targets are ignored if the `-msv` option is not specified.

Once an analog scope is enabled with the dump `-msv on` command, it cannot be disabled for dumping throughout the simulation using the dump `-msv off` command.

If `-type` is not specified, you can use the following command to set the default dump type as FSDB:

```
% setenv SNPS_SIM_DEFAULT_GUI verdi
```

Use the dump `-add` UCLI command to dump analog signals, digital signals, or both analog and digital signals in the FSDB file.

Example-1: dump `-msv (on|off)` is not specified.

The `-msv` option is enabled by default when on or off is not specified. Consider the following example:

```
ucli% dump -msv -type fsdb -file analog_mixed_signal.fsdb
ucli% dump -add top.a -fid FSDB0
```

This example dumps all the analog and digital signals of the `top.a` scope.

You must specify the `-fid` argument if multiple dump files are open, if not VCS issues an error message.

Example-2: dump `-msv off` is specified

```
ucli% dump -msv off -type fsdb -file analog_mixed_signal.fsdb
ucli% dump -add top.U0 -fid FSDB0
```

This example dumps all the digital signals of the `top.U0` scope and all the hierarchies under it, excluding all analog signals in the hierarchy.

For more information on the `dump -add` and `dump -file` UCLI commands, see *Integrating VCS with Verdi* chapter in the *VCS User Guide*.

Usage Example

If the `-msv` option is set to on, the `dump -add a.b.c -type` command exhibits the following behavior:

- If `a.b.c` is an analog net, dumps its voltage.
- If `a.b.c` is an analog sub-circuit, dumps all ports and internal nets of the sub-circuit.
- If `a.b.c` is digital net, dumps its digital value.
- If `a.b.c` is digital instance, dumps the signal inside this scope.
- If `a.b.c` is digital or analog instance where `c` contains mixed-signal hierarchies, then both digital and analog signals of `c` and its hierarchies are dumped.

Dumping Current Probes Using the UCLI Command

You can dump current probes using the UCLI command or Verdi dump tasks by specifying the corresponding command in CustomSim configuration file. If not, Verdi issues error messages and signals are not dumped.

The following table lists UCLI and CustomSim Probe Commands that you can use to dump the current probes:

UCLI Commands	CustomSim Probe Commands
<code>dump -add top -i -fid FSDB0</code>	<code>iprobe_waveform_current -i top.* -limit 99</code>
<code>dump -add top -i2 -fid FSDB0</code>	<code>iprobe_waveform_current -i2 top.* -limit 99</code>
<code>dump -add top -iall -fid FSDB0</code>	<code>iprobe_waveform_current -iall top.* -limit 99</code>
<code>dump -add top -isub -fid FSDB0</code>	<code>iprobe_waveform_current -isub top.* -limit 99</code>

Note:

The values in the table are examples only, the scope name and current option can be changed as required.

Performing Merge Dumping

You can dump analog (spice) signals/scopes into the first opened FSDB file, that is, the default FSDB file. You can also use `fsdbDumpvars` to dump digital signals into the same FSDB.

Perform the following steps:

1. Specify the following command in the *CustomSim* configuration file (xa.cfg):

```
set_waveform_option -format FSDB -file merge
```

2. Set the `VERDI_HOME` variable as follows:

```
% setenv VERDI_HOME <verdi_path>
```

3. Compile the design with the `-debug_access` option, as follows:

```
% vcs -debug_access <file_name>
```

4. The spice signal specified to be dumped in the configuration file is dumped to the first FSDB file opened. If there is no FSDB file opened, the FSDB dumper creates an FSDB

with the default file name which is `novas.fsdb`, and dumps the spice signal into this FSDB file.

5. If you do not want to use `novas.fsdb` as the default FSDB file, you can use the following command to specify the default FSDB file name to `xxx.fsdb`. This file becomes the default FSDB file. If this is not specified, then the default file name is `novas.fsdb`.

```
simv +fsdb+default_filename+xxx.fsdb
```

or

```
setenv FSDB_DEFAULT_FILE_NAME xxx.fsdb
```

Note:

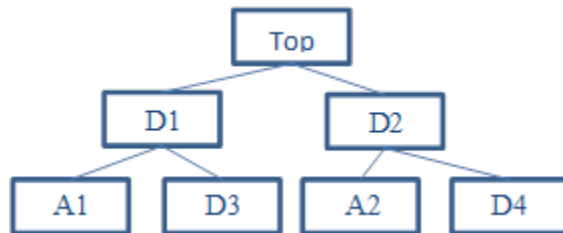
If any *CustomSim* probe command is invoked on a SPICE signal, its wave is dumped in the target FSDB file. For more information on the *CustomSim* configuration commands, refer to the *CustomSim Command Reference User Guide*.

Usage Example for Merge Dumping

The following example describes how the common dump flow and the single merged FSDB file flow work together.

Consider that you have the following design:

Figure 23 Sample Design



In this design, `Top`, `D1`, `D2`, `D3`, `D4` are digital scopes and `A1`, `A2` are analog scopes.

Consider that the configuration file `xa.cfg` has the following settings:

```
set_waveform_option -format fsdb -file merge
```

The following *CustomSim* command dumps a signal's voltage under the scope `Top.D1.A1`:

```
probe_waveform_voltage Top.D1.A1.* -port 1 -limit 3
```

The VCS command (UCLI command) file has the following setting:

```
fsdbDumpvars Top.D2 +msv+v
```

The following table shows how the signals are dumped for this design with the current setting. The signals in green are dumped by the `fsdbDumpvars` command and the signals in blue are dumped by the `probe_waveform_voltage` command.

-file merge	+msv option	Scope of Dumped Signals
V	V	A1, A2, D2, D4
V	X	A1, D2, D4
X	V	A2, D2, D4 (signals in scope A1 go to xa.fsdb, is not merged in FSDB)
X	X	D2, D4 (signals in scope A1 goto xa.fsdb, is not merged in FSDB)

When you use `-file merge` in the configuration file, the configuration file provides the target probes to the dumper and then the dumper dumps these probes in the FSDB file.

When you use the `+msv` option, the dumper queries the configuration file for the probes under the target scope. If not, the dumper only dumps digital signals under the target scope.

Limitation

This feature has the following limitation:

If an instance or a signal name in a SPICE scope contains a SystemVerilog hierarchy delimiter ".", for example, a signal is named as `xx.bb`, in merged single FSDB flow, the signal may not be put into right hierarchy in the FSDB as expected.

Dumping Automatic Inserted Connect Modules (AICM) Signals in the Verilog-AMS Flow

This feature enables you to dump the AICM (*_aicm) signals into the Fast Signal Database (FSDB).

To enable the dumping, do the following:

1. Set the following environment variable:

```
setenv FSDB_MSV_D_AICM 1
```

2. Use explicit dump commands as follows:

```
dump -add <aicm_signal_name> -fid $digitalDumpFile
```

Examples

The following command dumps the `snps_sptop.rst1__aicm` signal into the FSDB.

```
dump -add snps_sptop.rst1__aicm -fid $digitalDumpFile
```

The following command dumps the `snps_sptop.in2__aicm` signal into the FSDB.

```
dump -add snps_sptop.in2__aicm -fid $digitalDumpFile
```

Enabling Through Net Dumping Information in FSDB File

The analog view of `a2a` through net in digital scope and the digital view of `d2d` through net in analog scope can be dumped through dump command without extra environment variable setting. The digital view of `a2a` through net is filtered to avoid the confusion caused by the `z` value.

In the *nWave* window, for `a2a`, `a2d`, `d2d`, and `d2d` tags to appear, for example, `v(clk)` (`a2a`), you need to import the design and interface element report file. Also, when multiple interface element types are available for the signals at same time, the tags are separated by "/" without duplication.

Use Model

In analog scope, to enable the digital view of `d2d` through net, use the following simulation command:

- `dump -add *`

In digital scope, to enable `a2a` through net dumping of the current in the analog view, use the following simulation commands:

- Dump analog current signals

For example, `dump -add * -i`

- Dump the total current flow

For example, `dump -add * -isub`

- Dump analog current signals

For example, `dump -add * -vaI`

In digital scope, to enable `a2a` through net dumping of the voltage in the analog view, use the following simulation commands:

- Dump analog voltage signals

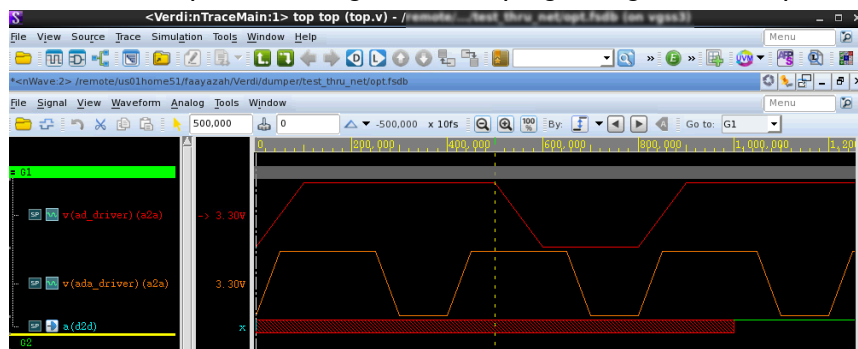
For example, `dump -add * -v`

- Dump analog voltage signals

For example, `dump -add * -vaV`

Following figure is an example of the net dumping using the `+msv` options:

Figure 24 Example of Through Net Dumping Using the `msv` Options



Limitations

This feature has the following limitations:

- Analog view of the `a2a` through net cannot be dumped when only the digital simulator simulates the design.
- The through net interface does not support VHDL design.
- The through net interface does not support the union data type.
- The digital view of `a2a` through net is dumped and displays the value as `z` when the `a2a` net is one of the following types:
 - Partial bits of vector/packed type.
 - Member of array or unpacked type.

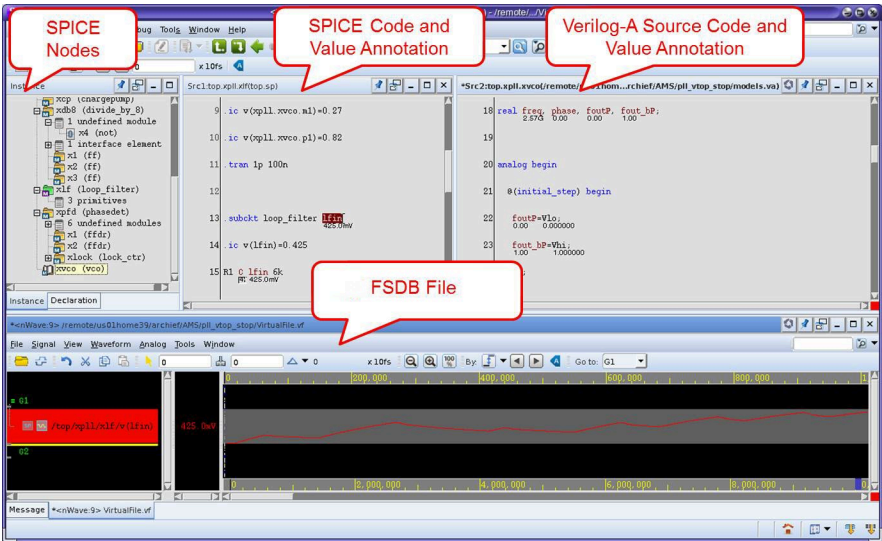
Debugging in Source Code Pane

After loading mixed-signal design into Verdi, you can view the hierarchy with both analog and digital scopes in the *Instance* pane. You can also check the SPICE and Verilog-

A source code in the *Source Code* pane as illustrated in the following figure. For more information about debugging Verilog-A designs, see [Interactive Simulation Debugging of Verilog-A Designs](#).

If your design includes SPICE source code, the SPICE nodes are also shown in the *Instance* pane.

Figure 25 Showing SPICE Information



The following table shows the SPICE nodes shown in the *Instance* pane.

Node Figure	Node Meaning
	A primitive device
	An interface element
	A subckt instance

You can double-click a node in the *Instance* pane to jump to the corresponding definition source code in the *Source Code* pane. Right-click menu options are available for SPICE nodes.

Tracing Signals

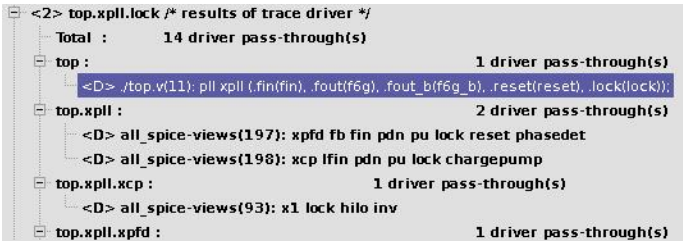
In the *Source Code* pane, the features like, coloring, range, selection, drag and drop, double-click and right-click menu options are available. The trace driver, load, and connectivity features are also available for a SPICE signal. You can double-click a signal

to trace the drivers or use the following right-click menu options to trace the selected SPICE signal:

- **Driver:** Shows the results of the trace driver in the *Message* pane.
- **Load:** Shows the results of the trace load in the *Message* pane.
- **Connectivity:** Shows the results of the trace connectivity in the *Message* pane,

You can also double-click a result in the *Trace* window as illustrated in the following figure to check the source code in the *Source Code* pane:

Figure 26 Trace Driver for a SPICE Signal



Note:

The **Active Trace** command is not supported in SPICE scope.

Active Annotation

After the virtual file containing digital and analog data files is loaded into the Verdi platform, you can use the **Source -> Active Annotation** option to see the current SPICE value annotated in the *Source Code* and *Signal List* panes as shown in the following figures:

Figure 27 Active Annotation in Source Code



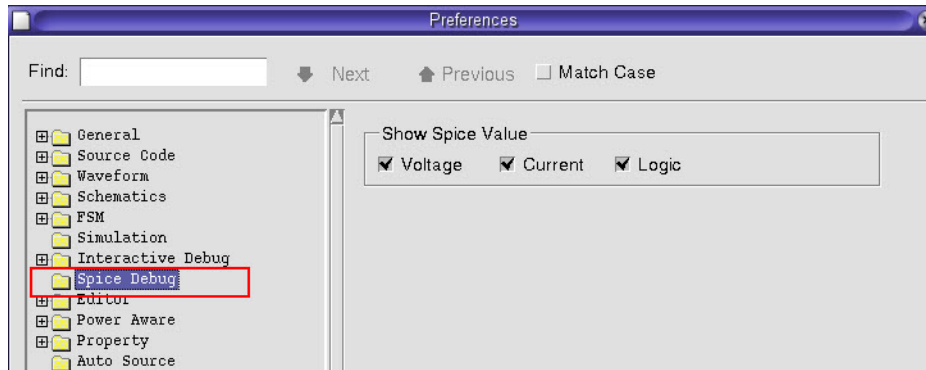
Figure 28 Analog Value in Signal List

Signal	Value	
bias2		
c1	344.1mV	wire
c2	259.8mV	wire
gnd	NF	wire
hilo	0.00	wire

The type of the attached value is the voltage value by default. You can use the following steps to add or remove other types for the attached value:

1. Use the **Tools -> Preferences** option to invoke the *Preferences* form.
2. Select the **Spice Debug** page and enable the types. Showing multiple-types is supported.

Figure 29 Showing SPICE Value



Note:

Only v(<signal>) and i(<signal>) values are supported for the annotation of the voltage and current types.

3. Click **OK** to apply the setting.

Interface Elements Enhancements

This section contains the following subsections:

- [Interface Element Browser in nTrace](#)
- [Interface Element Nets in nSchema](#)
- [Interface Element in nWave](#)
- [Interface Element Property Table](#)

Interface Element Browser in nTrace

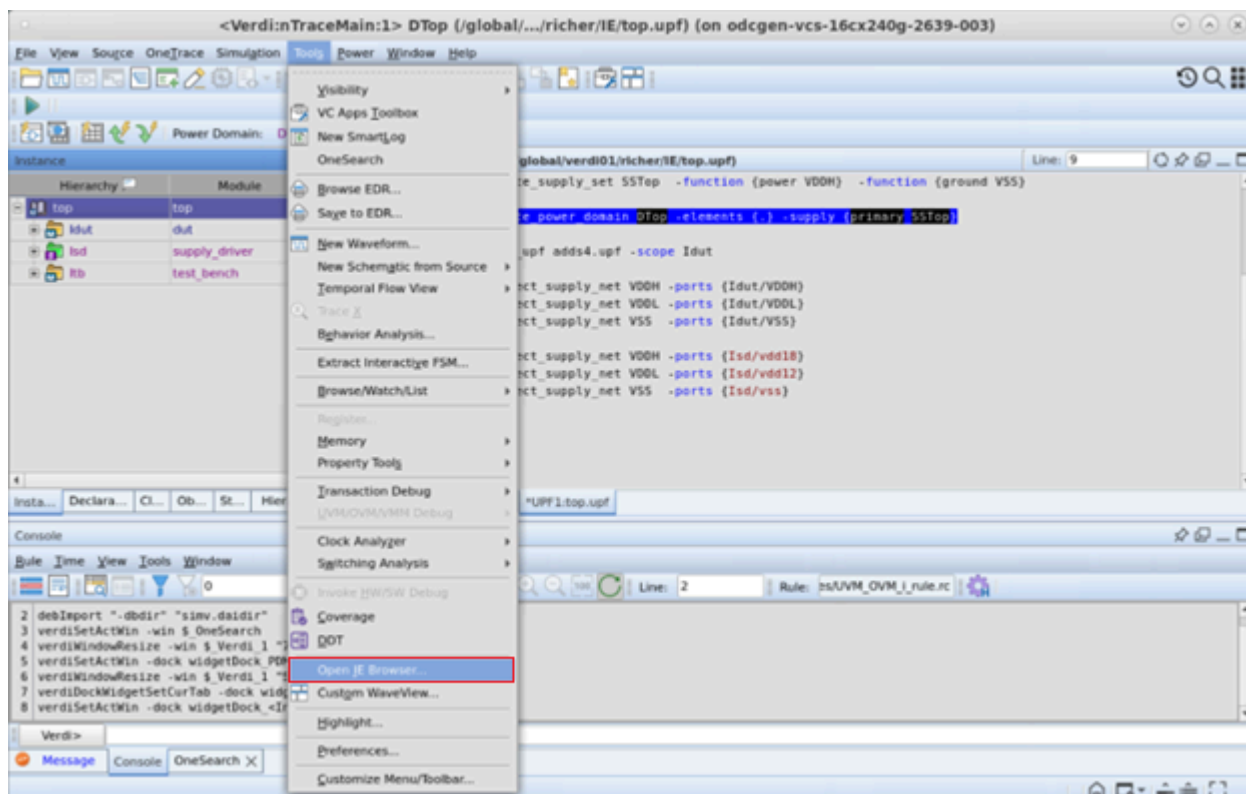
nTrace provides Interface Element filter types and pie chart based browser is used to display the proportion of the Interface Element in the design.

Use Model

The following are the steps to open Interface Element (IE) Browser:

1. Select **Tools > Open IE Browser** command. The command is only available for AMS designs.

Figure 30 Open IE Browser

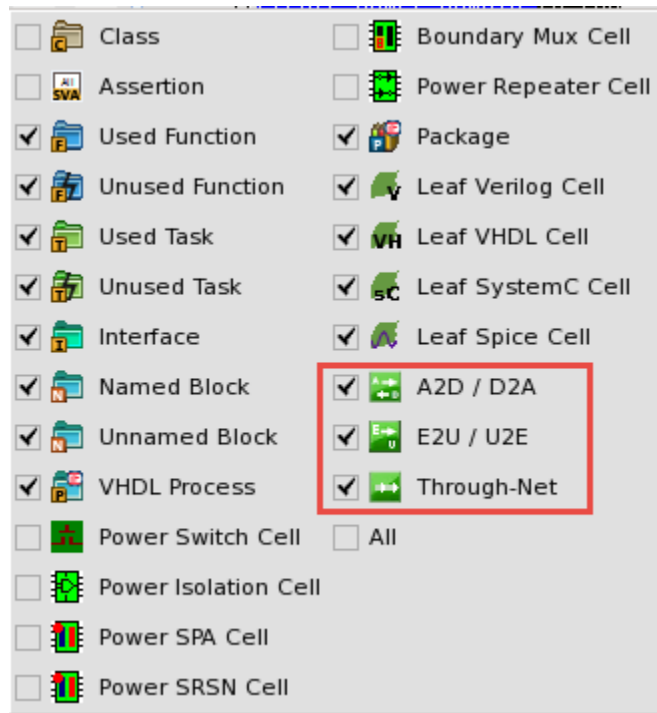


Filter Types

Following are the new filter types as shown in the figure below:

- A2D/D2A
- E2U/U2E
- E2R/R2E
- E2N/N2E
- Through-Net

Figure 31 Filter Types



Interface Element Nets in nSchema

nSchema window can highlight IE nets with color and then show IE type in the toolbar.

Use Model

Following are the features supported by *nSchema*:

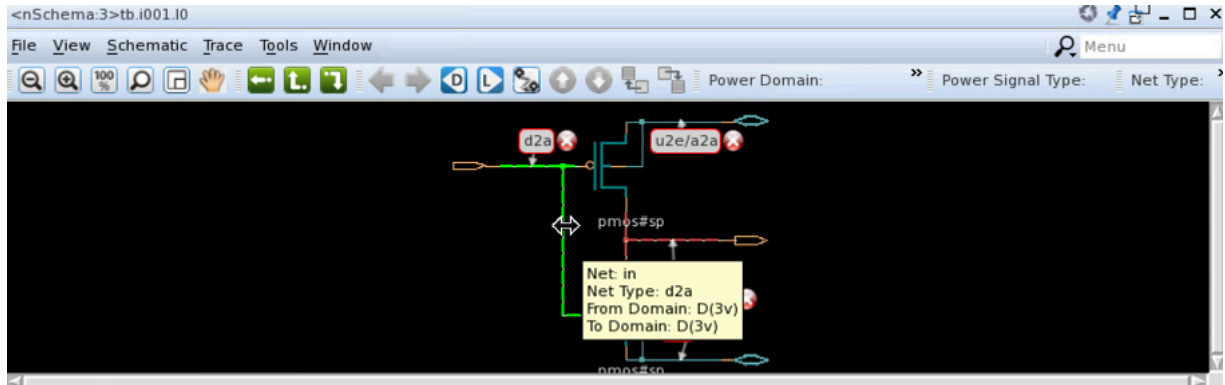
- When the mouse hover is on the IE net, the tooltip shows domain, voltage, and net type information.
- To list IE net information for the schematic window into a table.

Tooltip Format

- Net Name: `in`
- Net Type: `d2a`
- From Domain: domain name (voltage)
- To Domain: domain name (voltage)

The following figure shows how tooltip appears in *nSchema* window:

Figure 32 Tooltip in *nSchema* Window



RMB Menu Interface Element Information

Click Interface Element Information from the RMB menu. It lists the from/to domain list and the domain voltage for each interface element as shown in the following figures:

Figure 33 Interface Element Information



Figure 34 *Interface Element Information Table*

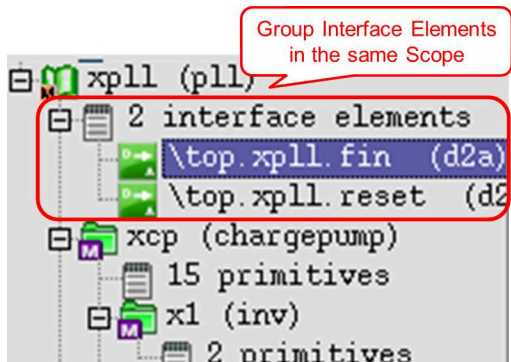
Interface Element Information_nSchema3 ()		
Interface Element	From Domain List(voltage)	To Domain List(voltage)
tb.i001.i0.in	D(3v)	D(3v)
tb.i001.i0.vdd	VDD(3v)	D(3v)
tb.i001.i0.vss	VSS(0v)	D(3v)

When you click the IE net name, it gets highlighted in the schematic window. It is supported only with UPF. The voltage changes if the timing is changed.

Viewing Interface Elements

When the report file of the interface elements is loaded into the Verdi platform, the interface elements in the same scope are grouped in the *Instance* tab as illustrated in the following figure:

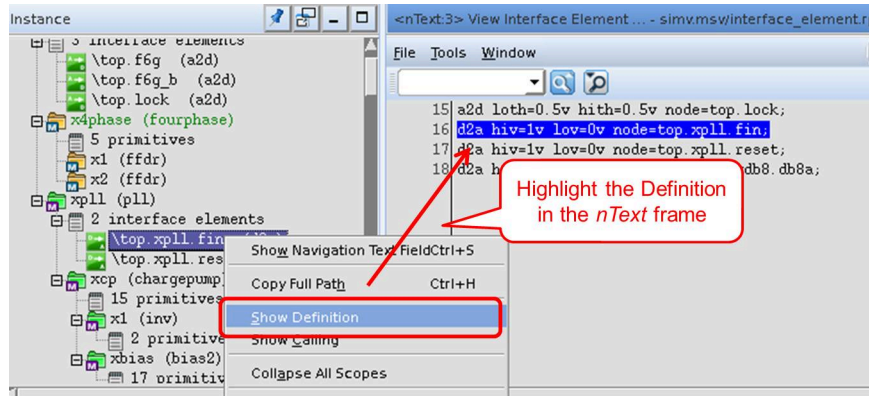
Figure 35 *Group Interface Elements*



In the *Instance* tab, you can use the **Show Definition** right-click menu option for a selected interface element or double-click it to see the definition. The report file is opened

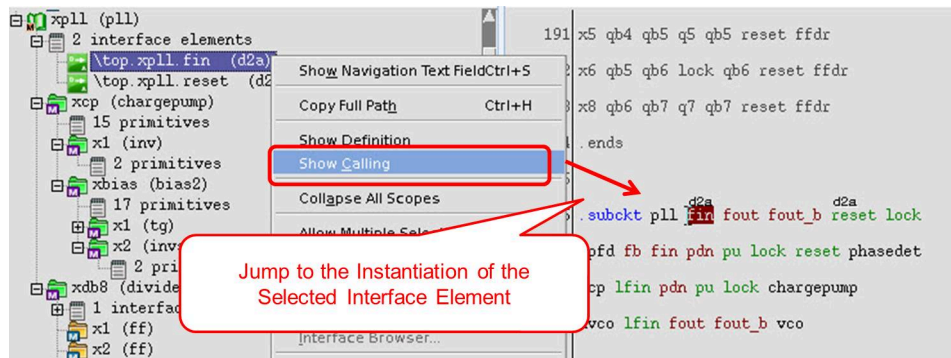
in the automatically invoked *nText* pane and the line of the interface element definition is highlighted as illustrated in the following figure:

Figure 36 Show Definition



You can use the **Show Calling** right-click menu option for a selected interface element to check and highlight the instantiation in the *Source Code* pane, as shown in the following figure:

Figure 37 Show Calling



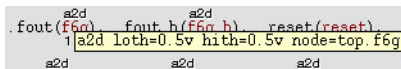
In the *Source Code* pane, the type of the interface element is shown above the interface element. The following table lists the interface net types:

String	Interface Net Types
a2d	Analog to digital interface node
d2a	Digital to analog interface node
e2r	Real interface node with analog to digital direction
r2e	Real interface node with digital to analog direction

e2n	Nettype interface node with analog to digital direction
n2e	Nettype interface node with digital to analog direction
inout	Bidirectional interface node

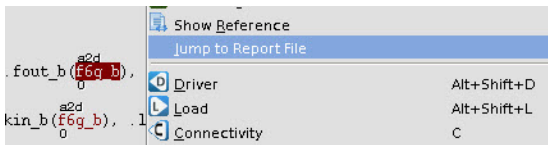
When the cursor is placed on the interface element, a hint dialog is shown with the information as shown in the following figure:

Figure 38 Show Hint



You can use the **Jump to Report File** right-click menu option for the selected interface element to open the interface element report file and highlight the line that defines the current interface node as shown in the following figure:

Figure 39 See Definition



When a tracing result passes through an interface element, <IE> is prefixed to the result.

Figure 40 Trace Result With Interface Element



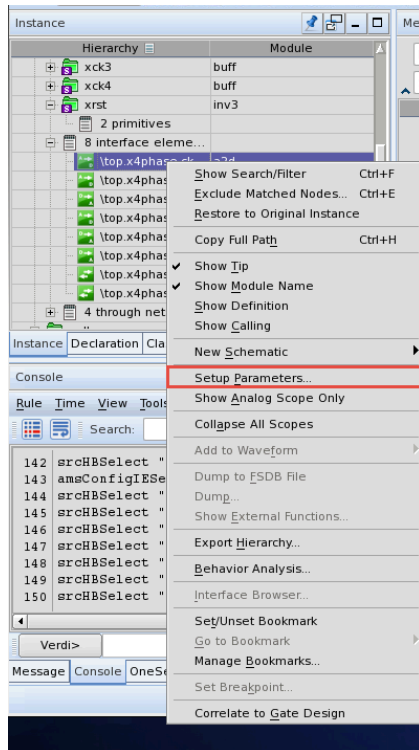
Editing the Interface Elements in Verilog-AMS Under Interactive Debug Mode in the nTrace Window

Verdi supports editing the Interface Elements (IE) in Verilog-AMS under Interactive mode.

To edit the IE elements, perform the following steps:

1. Right-click `a2d`, `d2a`, `e2n`, `n2e`, `e2r`, `r2e` node(s) in the *Instance* pane and select **Setup Parameters** option, as illustrated in the following figure:

Figure 41 Setup Parameters



You can also drag-and-drop the nodes in the in the *Instance* pane.

2. The *AMS Configuration* form appears containing the three tabs (**A/D**, **E/R**, and **E/N**), as illustrated in the following figure:

Figure 42 *The AMS Configuration Form*

AMS Configuration (on vgss3)

A/D E/R E/N

Analog to Digital interface node:

	primary_name	node	loth	hith	hiz	minv	minv_logic	mid
1	top.xpll.fout_b	top.x4phase.ckin_b	0.5v	0.5v				
2	+							

Digital to Analog interface node:

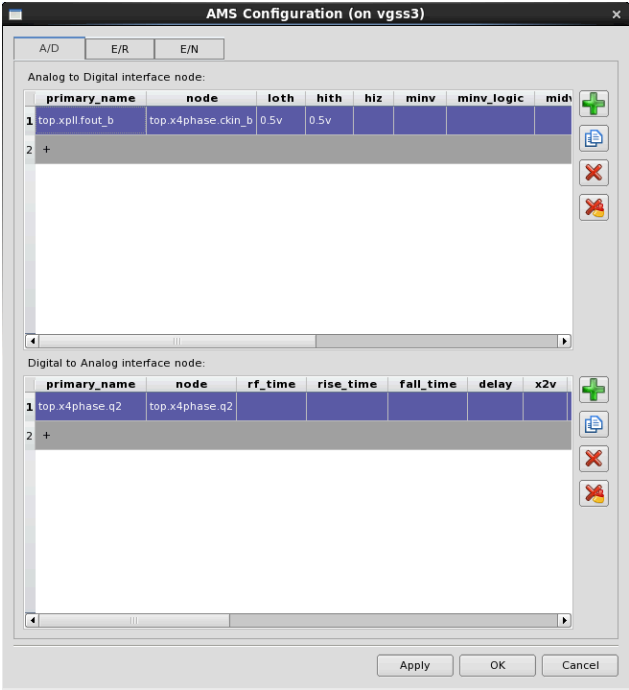
	primary_name	node	rf_time	rise_time	fall_time	delay	x2v
1	top.x4phase.q2	top.x4phase.q2					
2	+						

Apply OK Cancel

The A/D Tab

The **A/D** (Analog-to-Digital and Digital-to-Analog) tab enables you to set up the `a2d` and `d2a` commands, as illustrated in the following figure:

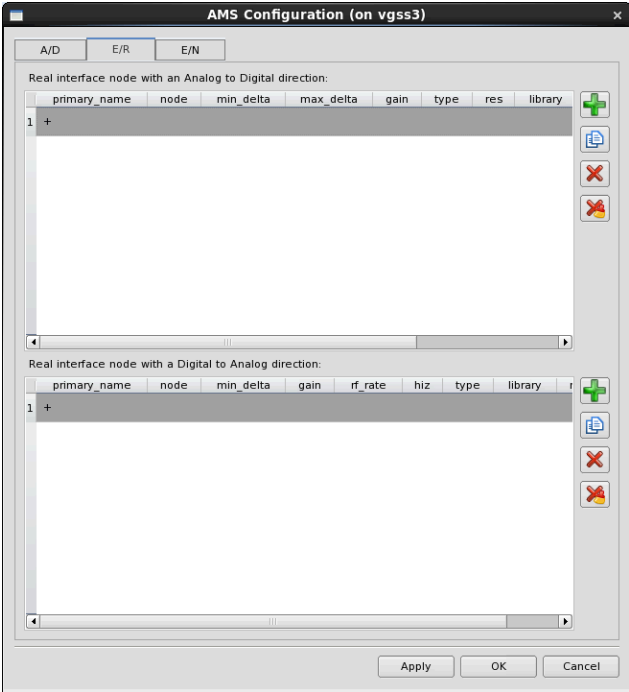
Figure 43 The A/D Tab



The E/R Tab

The **E/R** (Electrical-to-Real and Real-to-Electrical) tab enables you to set up the `e2r` and `r2e` commands, as illustrated in the following figure:

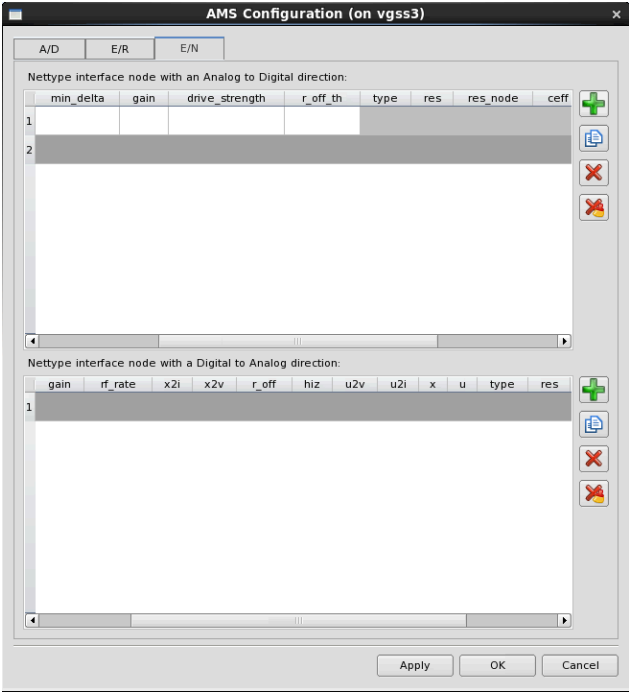
Figure 44 The E/R Tab



The E/N Tab

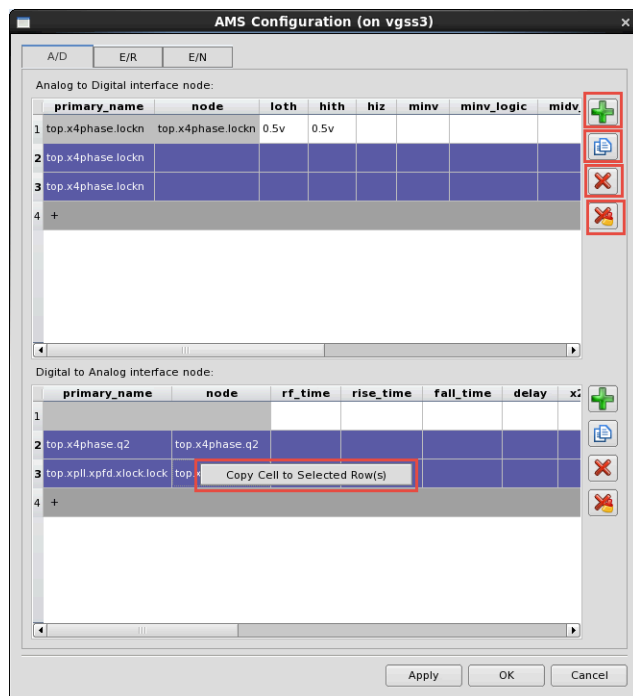
The **E/N** (Electrical-to-NetType and NetType to Electrical) tab enables you to set up the `e2r` and `r2e` commands, as illustrated in the following figure:

Figure 45 E/N Tab



The following operations are common to all the three tabs in the configuration form:

Figure 46 AMS Configuration Form



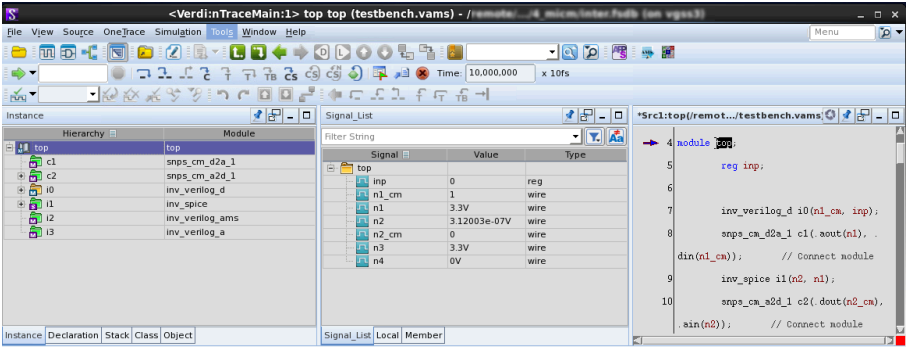
- **Add** : Adds a new row.
- **Copy** : Creates a new row which copies content from the selected row.
- **Delete** : Deletes the selected row(s).
- **Delete All** : Deletes all the row(s) selected.
- **Copy Cell to Selected Row(s)**: Copies the cell contents to all the selected row(s).
- **Apply**: Applies the current setting.
- **OK**: Applies the new settings and then closes the dialog.
- **Cancel**: Cancels the setting and closes the form.

Supporting Verilog-AMS in the nTrace Window

After loading Verilog-AMS designs into Verdi, you can see the hierarchy with the analog scopes in the *Instance* pane. You can also check the Verilog-AMS source code in the *Source Code* pane and the *Signal List* pane.

The Verilog-AMS instances are displayed in the *Instance* pane, as illustrated in the following figure:

Figure 47 Verilog-AMS Instances



The following table describes the Verilog-AMS instances shown in the *Instance* pane.

Node Figure	Node Meaning
	The Verilog-AMS instance
	The instrumented Verilog-AMS instance
	The instrumented Verilog-AMS signal

You can double-click a node in the *Instance* pane to jump to the corresponding definition source code in the *Source Code* pane.

Interface Element in nWave

You can drag an interface element from an instance browser to *nWave*. Both digital and analog values (two waves) for either side of the interface element are displayed as shown in the following figure.

The interface element can be **A2D**, **D2A**, **E2R**, **R2E**, **N2E**, **E2N**, and **INOUT**. The signals that are dropped are included in a group named as IE Group.

Note:

There is no change in the *nWave* behavior when you drag an interface element from the source code.

Chapter 3: Interactive Simulation Debugging of Mixed-Signal Designs Interface Elements Enhancements

Figure 48 Drag an Interface Element From Instance Browser

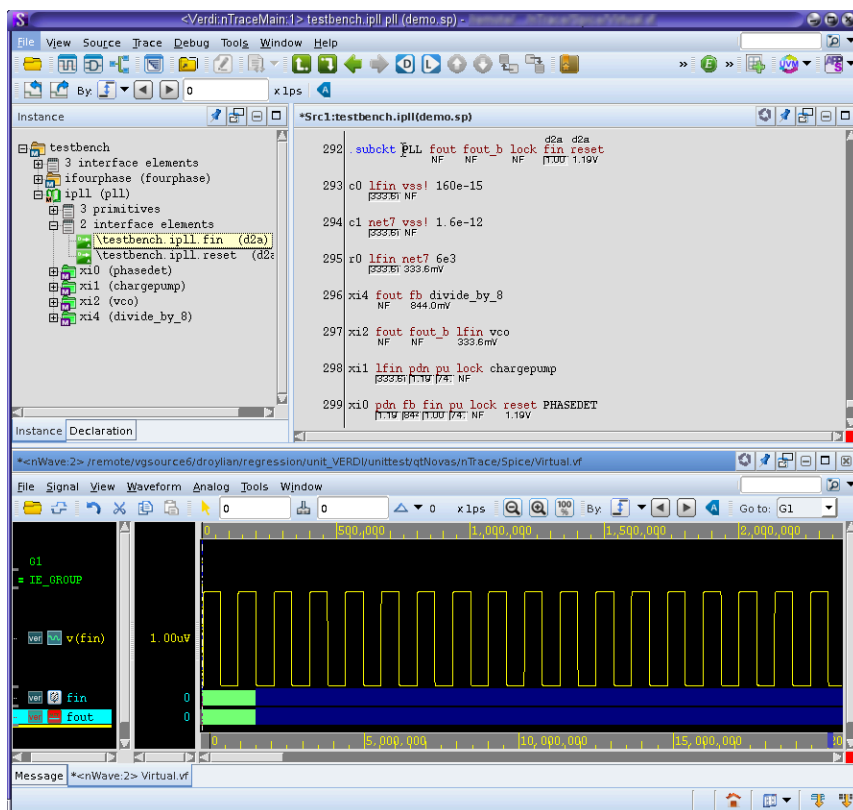


Figure 49 Drag and Drop an Interface Element From nTrace to nWave

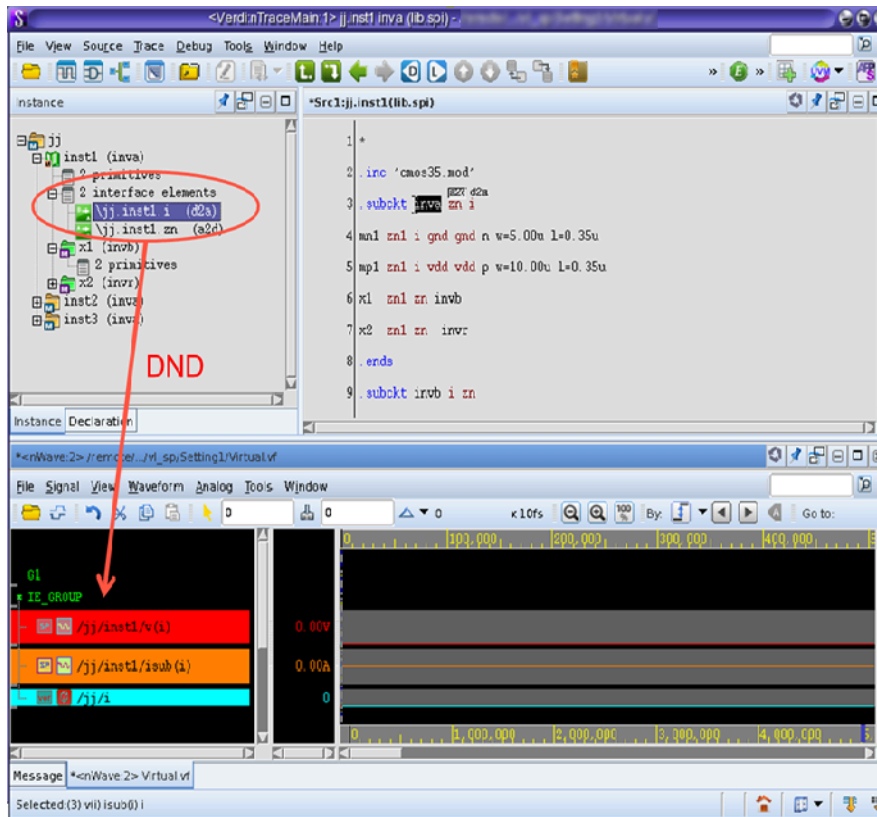
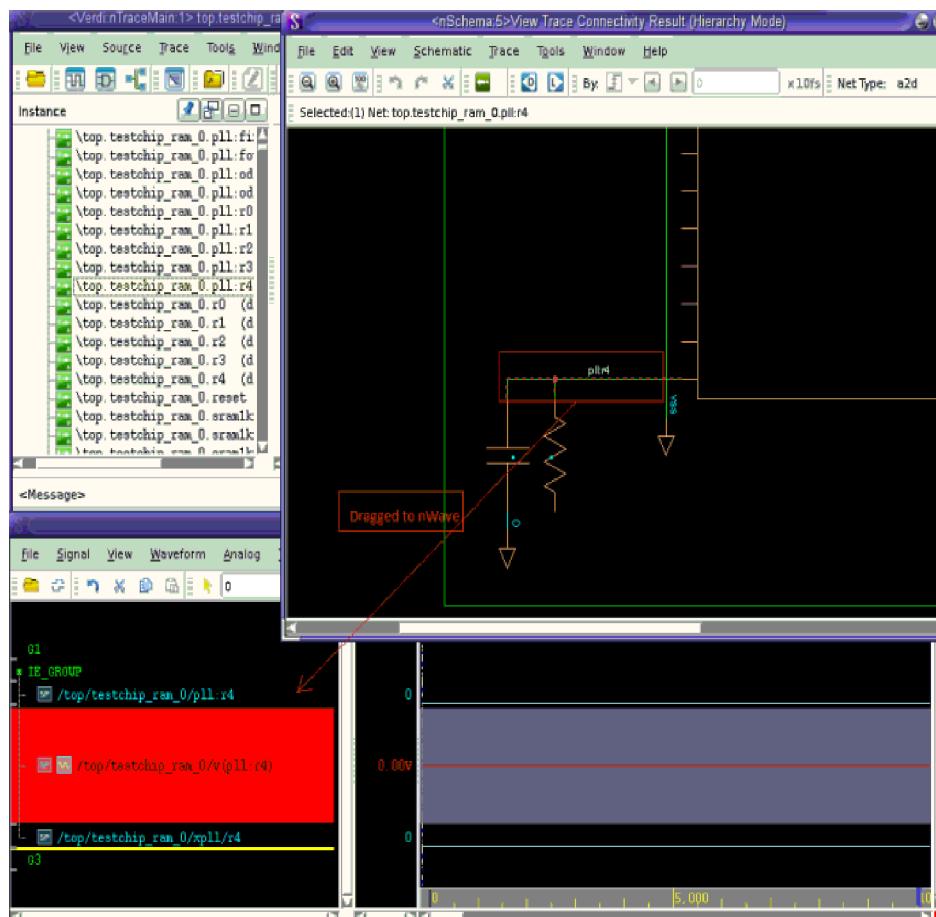


Figure 50 Drag and Drop an Interface Element From nSchema to nWave



Interface Element Property Table

An interface element (IE) property table provides a unified view of all interface elements in Verdi. This table provides the information included in the following reports:

- IE report (interface_element.rpt, and interface_connectivity.rpt)
- UPF report (upf_power_scope.rpt, upf_supply_source.rpt, and ams_tunnel_thru_upf.rpt)

Figure 51 Interface Element (IE) Property Table

Node	Type	Hiv	lov	loth	vdd	vss	rf_time	minv	minv_logic	minv_analog	top_net	dig_drv	ana_port	VDDSupplyNet	VDDSupplyNet	VDDSupplyNet	VDDSupplyNet	VDDSupplyNet
top_hut_bu444_133_0	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_0	VDDHtop	VSStop	3	1	top_hut_bu444_133_0
top_hut_bu444_133_1	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_1	VDDHtop	VSStop	3	1	top_hut_bu444_133_1
top_hut_bu444_133_2	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_2	VDDHtop	VSStop	3	1	top_hut_bu444_133_2
top_hut_bu444_133_3	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_3	VDDHtop	VSStop	3	1	top_hut_bu444_133_3
top_hut_bu444_133_4	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_4	VDDHtop	VSStop	3	1	top_hut_bu444_133_4
top_hut_bu444_133_5	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_5	VDDHtop	VSStop	3	1	top_hut_bu444_133_5
top_hut_bu444_133_6	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_6	VDDHtop	VSStop	3	1	top_hut_bu444_133_6
top_hut_bu444_133_7	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_7	VDDHtop	VSStop	3	1	top_hut_bu444_133_7
top_hut_bu444_133_8	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_8	VDDHtop	VSStop	3	1	top_hut_bu444_133_8
top_hut_bu444_133_9	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_9	VDDHtop	VSStop	3	1	top_hut_bu444_133_9
top_hut_bu444_133_10	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_10	VDDHtop	VSStop	3	1	top_hut_bu444_133_10
top_hut_bu444_133_11	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_11	VDDHtop	VSStop	3	1	top_hut_bu444_133_11
top_hut_bu444_133_12	a2a	100%	0%		top_hut_vdd10	top_hut_vss	25-10				top_hut_sump101		top_hut_bu444_133_12	VDDHtop	VSStop	3	1	top_hut_bu444_133_12

The IE table has the following tabs:

Tab Name	Column Name
a2d_d2a	- type - node - hiv - hith - lov - loth - vdd - vss - rf_time - minv - minv_logic - minv_analog - dig_drv - top_net - ana_port [upfList]
a2a_d2d_n2n	- node - type - top_net - ana_port

Tab Name	Column Name
e2u_u2e	• node • type • top_net • ana_port • Rth • Vth • Vth_Low • Glitch_time • Res_Tth • Supply_Type • rise_time_rate • fall_time_rate • gain • [upfList]
e2r_r2e_e2n_n2e	• node • type • rf_rate • hiz • gain • min_delta • max_delta • r_off_th • drive_strength

The *[upfList]* column appears in the IE table only when the information in the table includes the UPF report.

The *[upfList]* includes the following columns:

- UPF Supply Net Vdd (*upfSupplyNet_vdd*)
- UPF Supply Net Vss (*upfSupplyNet_vss*)
- Vdd Group ID (*VddGroupId*)
- Vss Group ID (*VssGroupId*)
- UPF Supply Source Vdd (*upfSupplySource_vdd*)
- UPF Supply Source Vss (*upfSupplySource_vss*)

IE Property Table Functions

The IE table enables you to perform the following functions:

- [Jump to the nTrace or nSchema Window](#)
- [Jump to the UPF File View](#)

Chapter 3: Interactive Simulation Debugging of Mixed-Signal Designs Interface Elements Enhancements

- [Add to the nWave Window](#)
- [Create a New Schematic](#)
- [Open the Percentage Pie Chart](#)
- [Sort and Filter](#)
- [Set Preferences](#)

Jump to the nTrace or nSchema Window

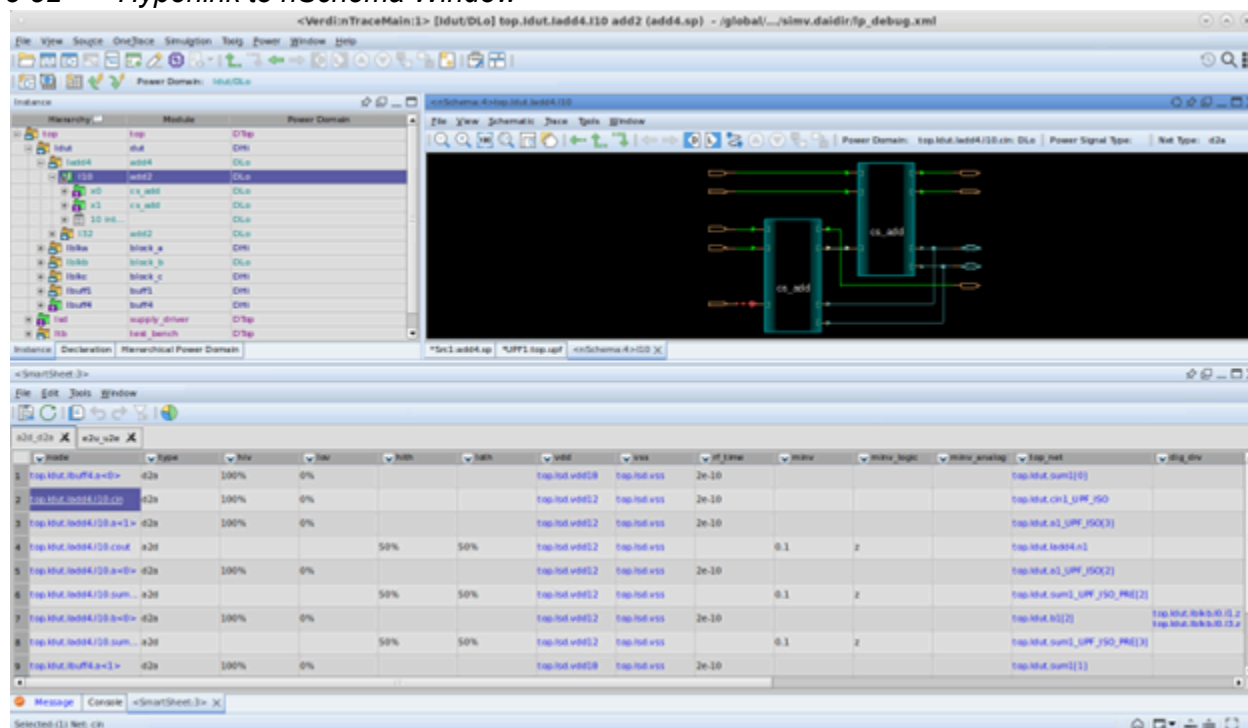
The columns in the IE table that support jump to *nTrace* and *nSchema* windows are: *node*, *vdd*, *vss*, *top_net*, *dig_drv*, and *ana_port*.

Click the hyperlink in the column to jump to the *nTrace* source window or the *nSchema* window. This function is available only when the source window or the *nTrace* window is open.

Note:

If you click a hyperlink when both *nTrace* source window and *nSchema* window are open, then *nSchema* window becomes the active window.

Figure 52 Hyperlink to nSchema Window

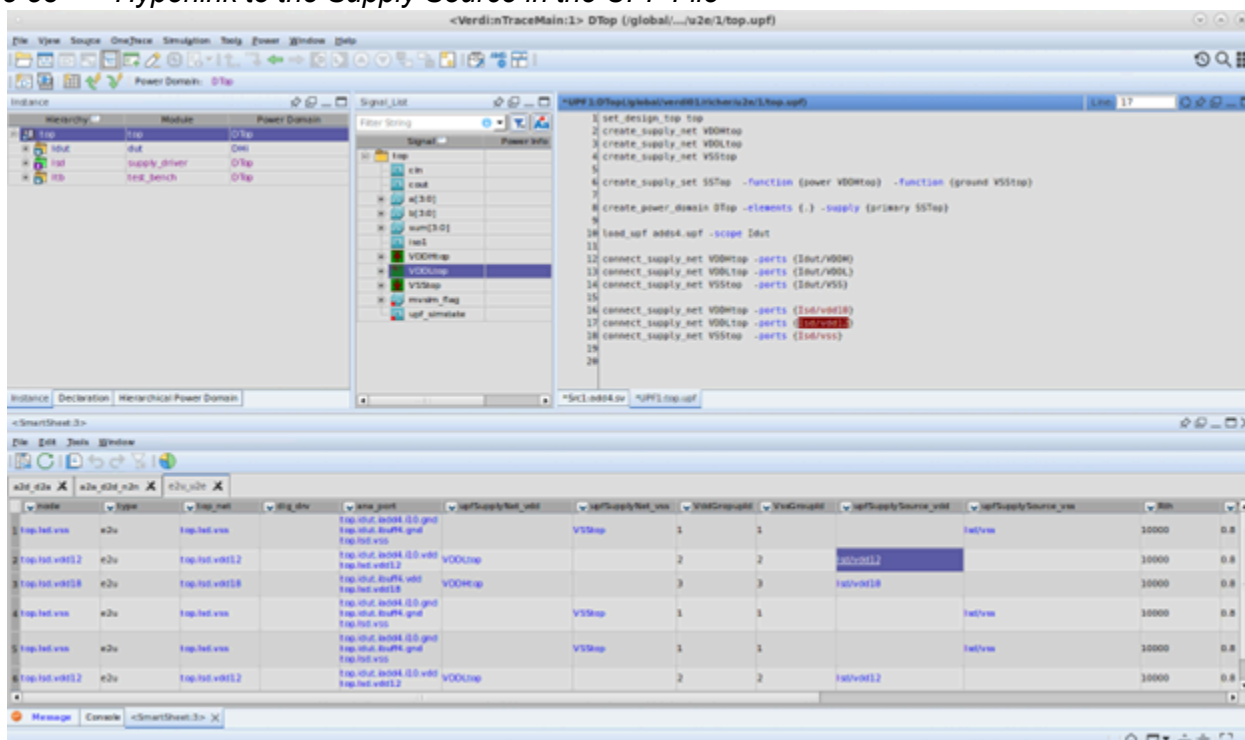


Jump to the UPF File View

Click the hyperlink in the *upfSupplyNet_vdd* and *upfSupplyNet_vss* columns, to jump to the supply net defined in the UPF file.

Click the hyperlink in the *upfSupplySource_vdd* and *upfSupplySource_vss* columns to jump to the supply source and find the first string that matches with the content in the cell.

Figure 53 Hyperlink to the Supply Source in the UPF File



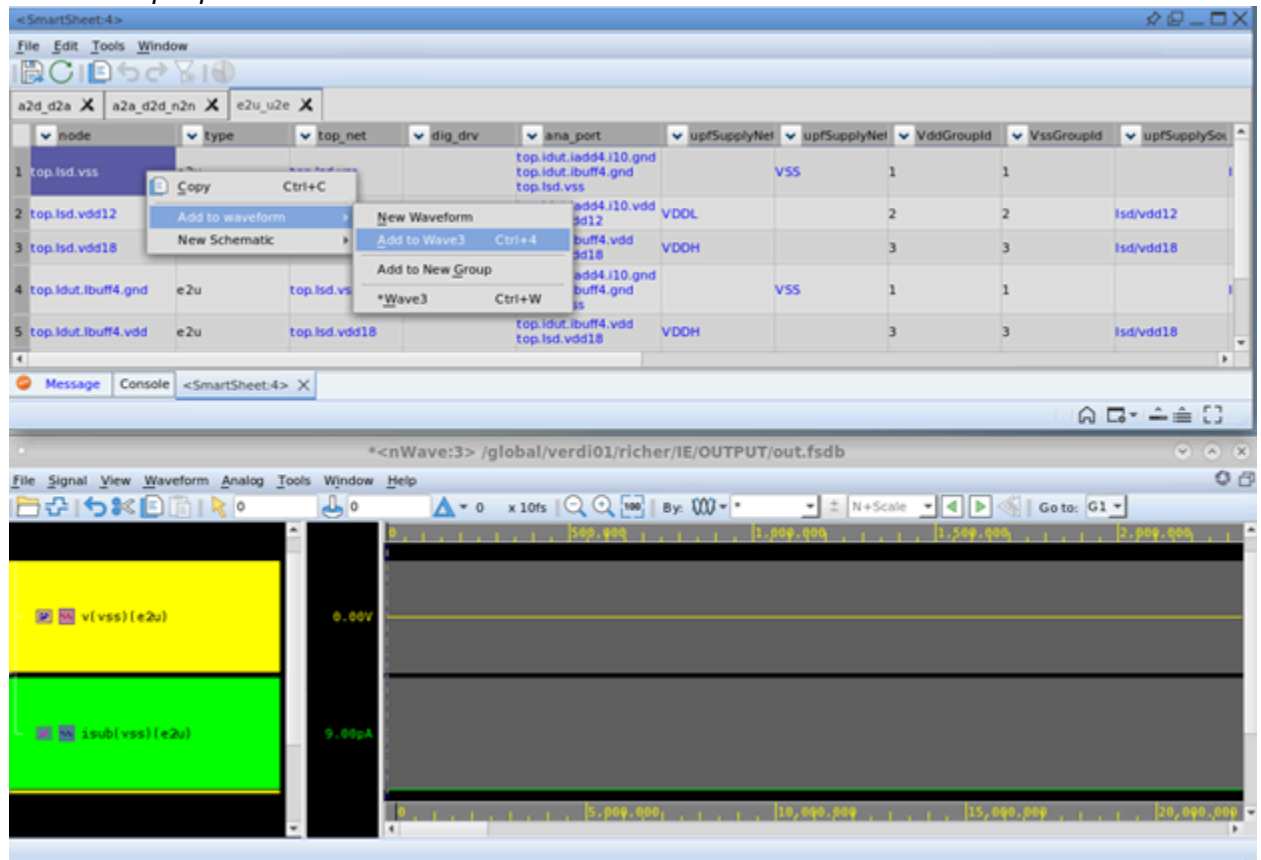
Note:

The Supply Source window does not support jump to the TCL command in the UPF file view. Use the **Search** option to find and select the first *Isd/vdd18* that appears in the UPF file view.

Add to the nWave Window

Right-click a cell and select **Add to waveform**.

Figure 54 Pop-Up Menu to Add Waveform



Note:

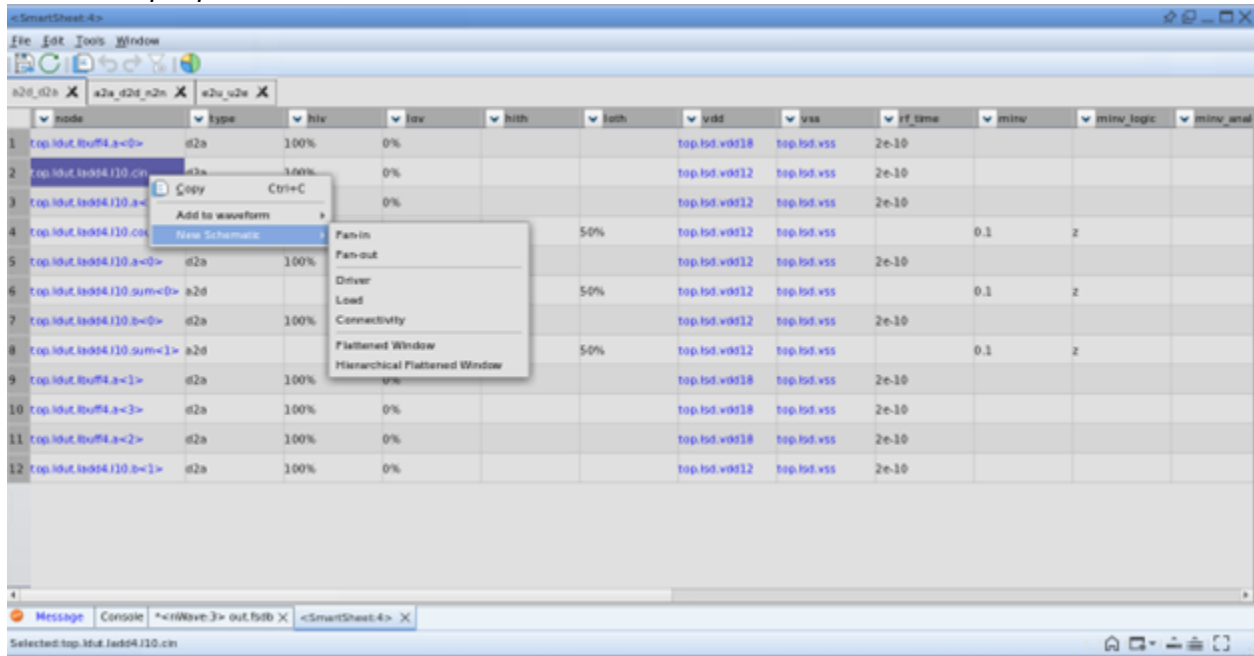
If the signal is a Spice or Verilog signal, find and add all its dumped signals to the waveform.

Add to the *nWave* feature is available in the columns: *node*, *vdd*, *vss*, *top_net*, *dig_drv*, *ana_port*, *upfSupplyNet_vdd*, *upfSupplyNet_vss*, *upfSupplySource_vdd*, and *upfSupplySource_vss* only and the table cell cannot be empty.

Create New Schematic

Right-click a cell, select **New Schematic**, and then select schematic type from the submenu .

Figure 55 Pop-Up Menu to Create Schematic

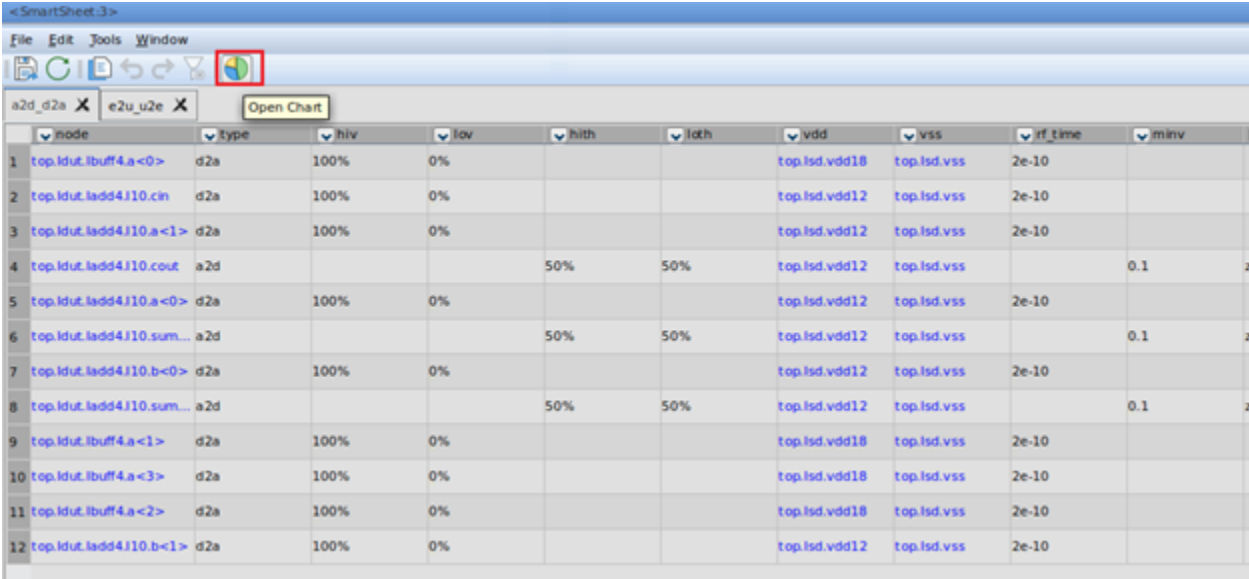


The new Schematic option is available in the columns: *node*, *vdd*, *vss*, *top_net*, *dig_drv*, *ana_port*, *upfSupplyNet_vdd*, and *upfSupplyNet_vss* only. The table cell cannot be empty.

Open a Percentage Pie Chart

On the toolbar, click the **Open Chart** icon to open the percentage pie chart.

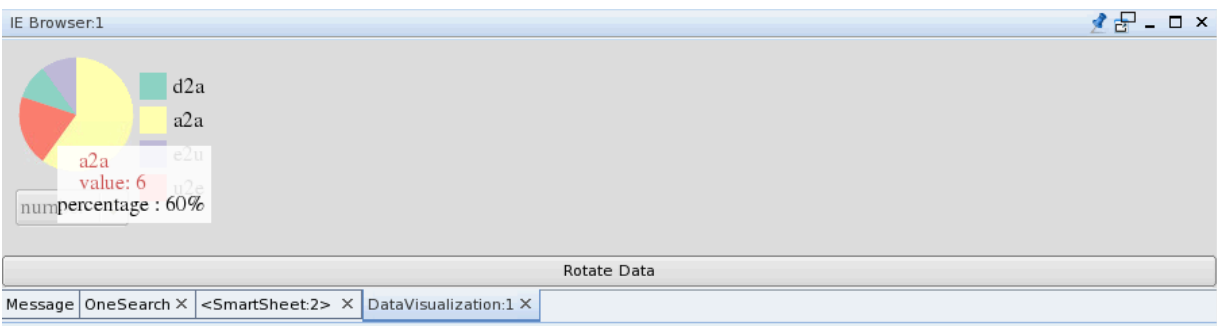
Figure 56 Open Chart Icon on Toolbar



Select a pie chart to view its interface elements as shown in the following figure.

The IE Browser displays the proportion of IE nets. When you move the cursor to any slice of the pie, a tool-tip is displayed as shown in the following figure. It contains the IE type, the value of IE, and the IE proportion.

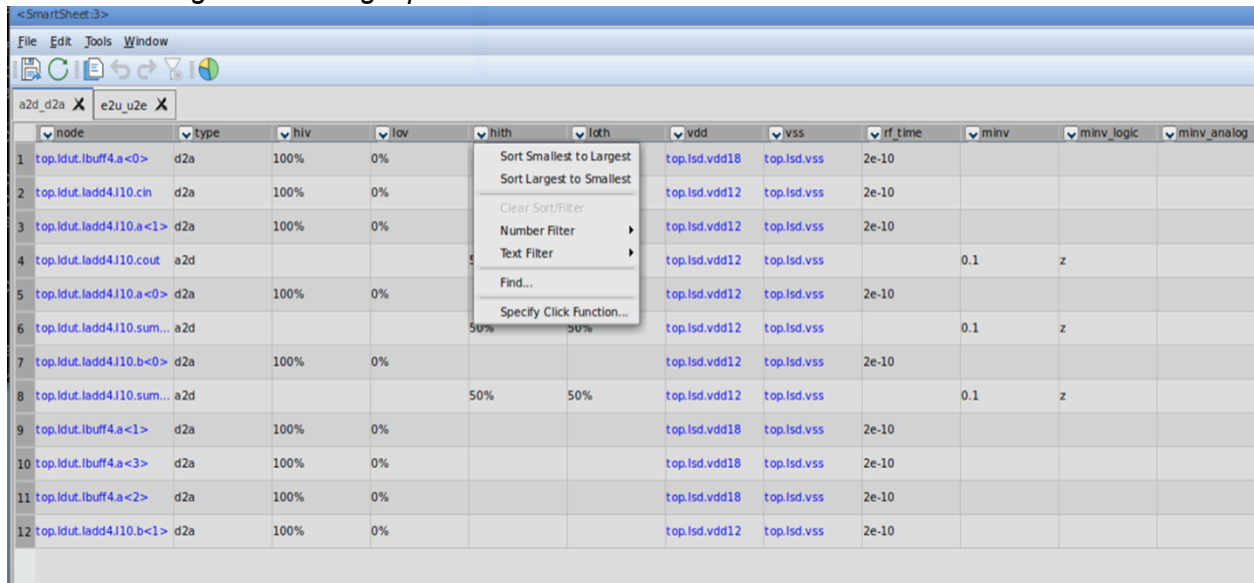
Figure 57 Display the Proportion of IE Nets



Sort and Filter

The IE table supports sorting and filtering function in each column.

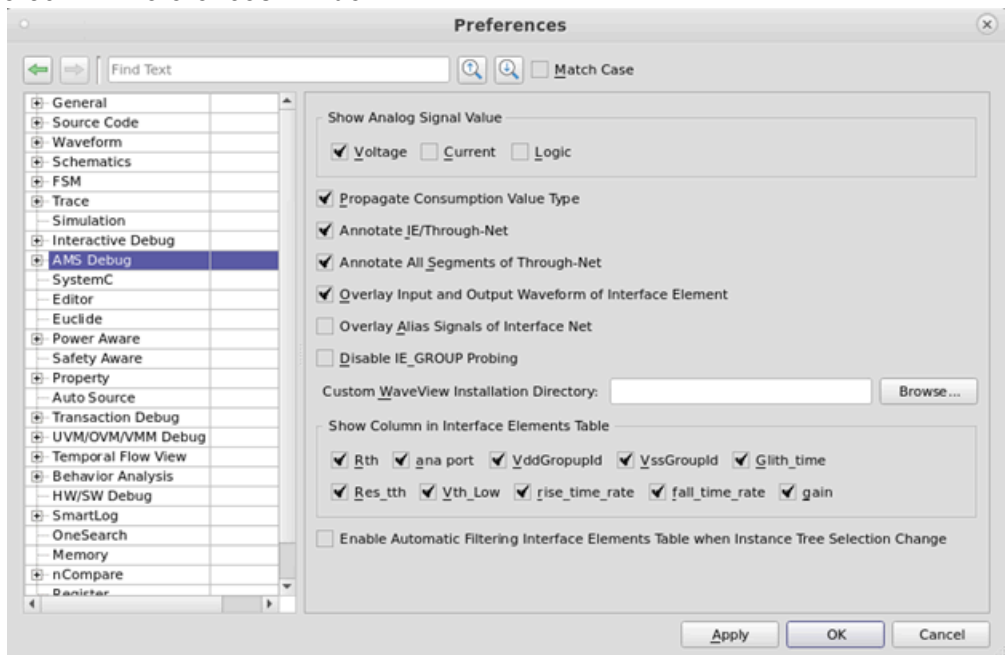
Figure 58 *Sorting and Filtering Options*



Set Preferences

You can set the preferences for the IE table in the **Preferences** window.

Figure 59 *Preferences Window*



To set the display preference settings, perform the following steps:

1. In the **Preferences** dialog box, expand **AMS Debug**.
2. In the **Show Column in Interface Elements Table** section, click the column type check box to select or deselect.

The columns that can be selected or deselected are: *Rth*, *ana_port*, *VddGroupId*, *VssGroupId*, *Glitch_time*, *Res_tth*, *Vth_Low*, *rise_time_rate*, *fall_time_rate*, and *gain*.

3. Select the **Enable Automatic Filtering Interface Elements Table when Instance Tree Selection Change** to filter the node column by instance tree selection.

By default value, this filter is not selected.

4. Turn on or off the filter by scope.
5. Click **Apply** to save the preferences.

Tcl Commands

The following Tcl Commands are used:

`verdiSetAMSOption -EnableAnaPort on/off` : This setting is stored in `novas.rc` file.
Set the as `EnableAnaPort = TRUE`.

`amsIEBrowserOpenPieChart`: This command opens the percentage pie chart.

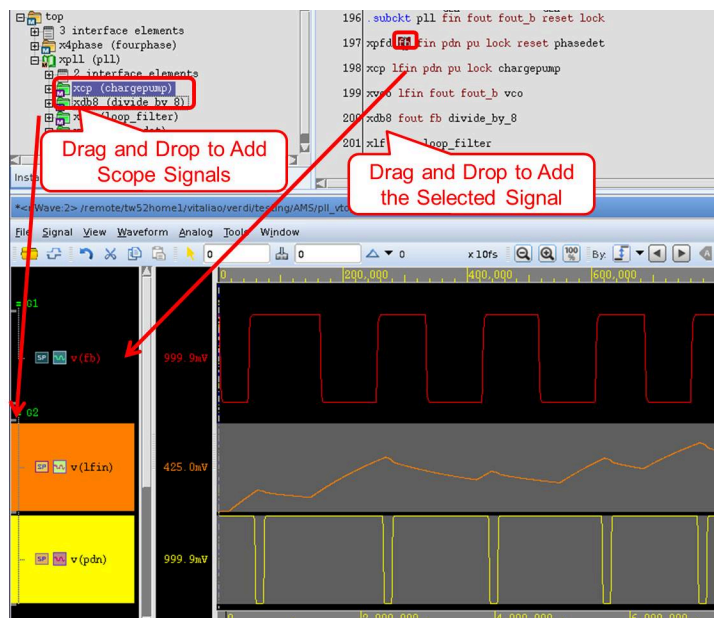
`amsOpenIEBrowser`: This command opens the `smartSheet` form (IE table).

Debugging in Waveform View

When you load a file containing digital and analog data files into Verdi, you can add both digital signals and analog signals into the *nWave* window that opens that file. You can also use the drag-and-drop and synchronization features to add these signals.

Similar to digital scopes, you can drag a SPICE instance in the *Instance* tab and drop it to the *nWave* window to add the ports of the selected scope. You can also drag a SPICE signal (or multiple signals) in the *Source Code* pane and drop it to the *nWave* window to add the selected signal as shown in the following figure. When you add a port/signal, all the value types of the ports/signals are also added automatically.

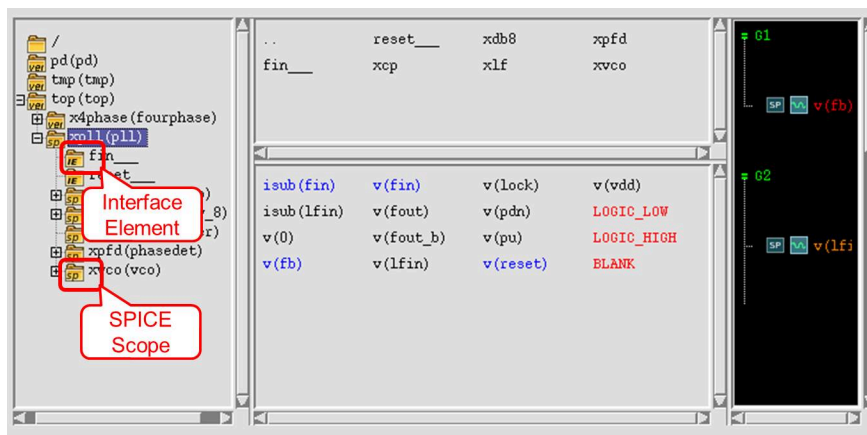
Figure 60 Add Signals by Dragging and Dropping



If the selected signal is not dumped into the FSDb file, the *List of Unrecognized Signal(s)* form opens and lists the selected signal as an unrecognized signal.

You can also use the **Signals -> Get Signals** option to select the dumped signals in the invoked *Get Signals* form. Select a scope or an interface element in the *Design Browser* pane, you can then select the signals in the *Signal List* pane to add them into the *nWave* window (refer to the following figure):

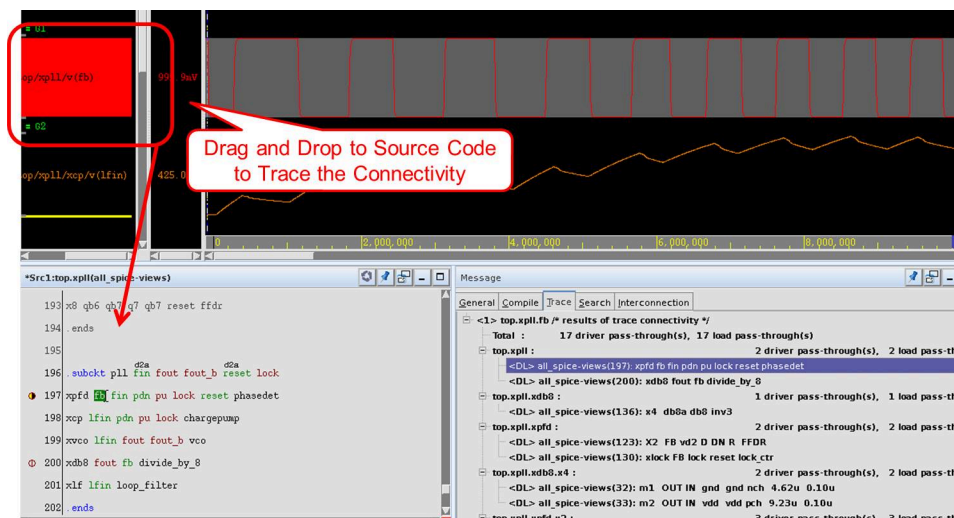
Figure 61 Get Signals Form



After the signals are added into the *nWave* window, as shown in the following figure, you can drag and drop a signal to the *Source Code* pane to trace the connectivity.

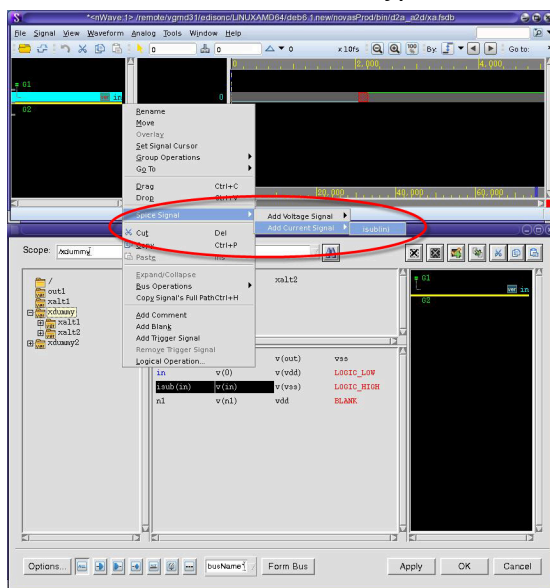
Chapter 3: Interactive Simulation Debugging of Mixed-Signal Designs Debugging in Waveform View

Figure 62 Trace Connectivity



If an added SPICE signal having other value type(s) get dumped, you can use **Spice Signal -> Add Voltage Signal**, **Spice Signal -> Add Current Signal** or **Spice Signal -> Add Logic Signal** right-click menu options to add the corresponding value type as shown in the following figure:

Figure 63 Add Related Value Types



Note:

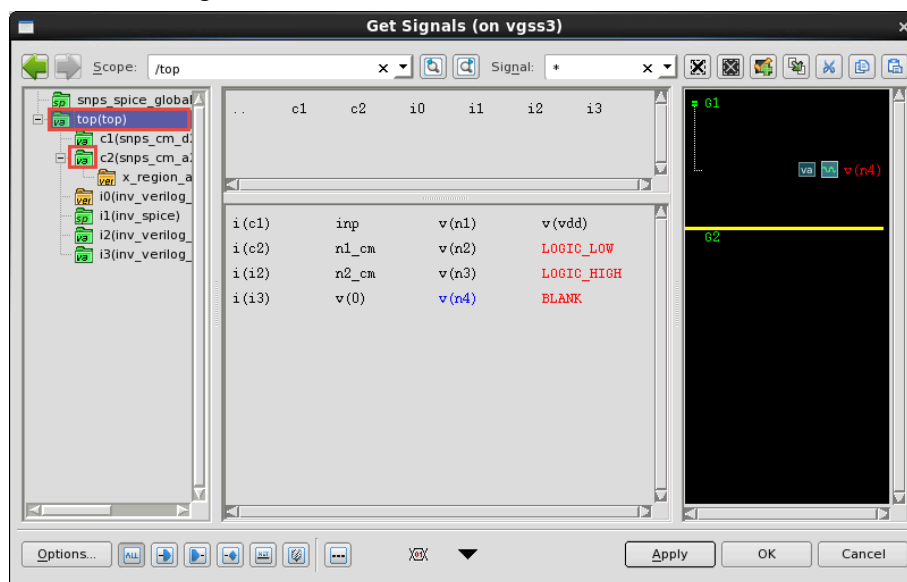
Add Voltage Signal, **Add Current Signal**, or **Add Logic Signal** commands are shown in the right-click menu only when the corresponding value types of the selected signal have been dumped into the FSDB files.

You can click the  icon in the different windows to synchronize the selected signals between different windows.

Supporting Verilog-AMS in the nWave Window

The icon for Verilog-AMS in the *Get Signal* Form is a green folder and the icon for Verilog-AMS signals in *Signal List* pane is **vA**, as illustrated in the following figure:

Figure 64 Get Signals



Verilog-AMS signals can be categorized into the following:

- **Nodes:** The current node should be different according to their position and usage in the Source Code pane.
- **Branches:** There are two types of branches in Verilog-AMS, **unnamed branch** and **named branch**. Compared to node, the signals to be added for branch is easy since a position issue does not exist in the branch.

Dumping of nWave Signals

If you drag-and-drop or right-click to add Verilog-AMS signals to *nWave*, *nWave* triggers the corresponding signals to be dumped. The signals to be dumped and be added should be same as drag-and-drop in post-simulation.

However, dumper does not support to dump specific port current if a Verilog-AMS instance is a leaf instance. That is, all nodes, current branch is dumped if you dump port current of leaf Verilog-AMS instance.

In stand-alone *nWave* window, *nWave* displays IEs and through nettypes through the information that the simulator dumped in the FSDB file on analog signals. Also, for *a2a*, it shows on both analog and digital signals. For the other tags it is shown, only on analog signals. Thus, the IEs and through nettype tag only label on the analog side signal.

Note:

- The *n2n* is dumped as *d2d* with the previous scenario.
- When signals are aliased to through net or IE, they contain a tag, even if they are not through net or IEs.
- When signals are not aliased as through net or IE they do not have a tag, even if they are through net or IE.

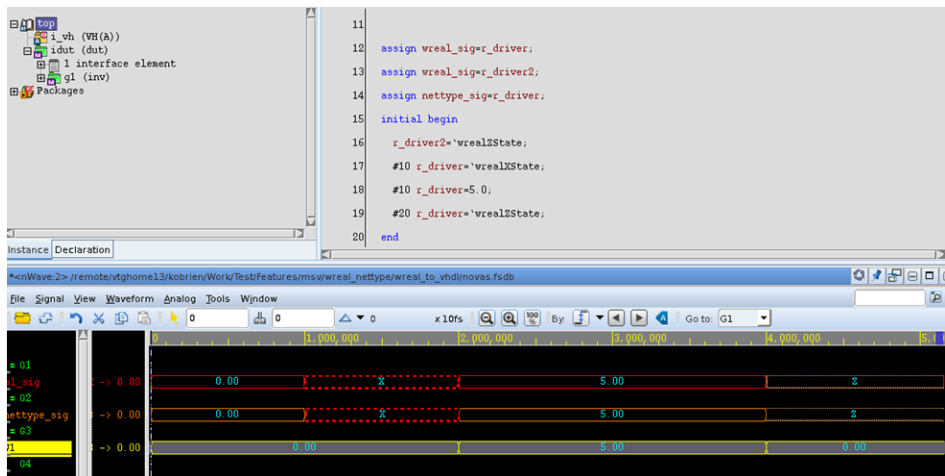
Supporting Unknown (X) and Inactive (Z) States for WReal Objects

nWave supports predefined nettypes that mimic Verilog-AMS *wreal* behavior, thus providing the capability to use a nettype-based flow along with the *wreal*-based flow.

Accordingly, *nWave* supports **Unknown** and **Inactive** states (represented as an X and a Z in 1-bit, 4-state logic) that are represented with *wreals* using the predefined macros ``wrealXState` and ``wrealZState`.

Similar to *wrealX* and *wrealZ* signals, *nWave* also supports *real* (*realX* and *realZ*) signals and displays them as shown in the following figure. If ``wrealXState` is assigned to a real object, it is shown as an X in the waveform. Also, if a ``wrealXState` is assigned to a real variable, it is shown as an X in the waveform. Also, if ``wrealZState` is assigned to a real variable, it is shown as a Z in the waveform.

Figure 65 Support X and Y States



The ``wrealXState` and ``wrealZState` macros also behave differently compared with other real numbers in arithmetic operations. Consider that you are adding two 4-state logic values and would like the sum `1'b1+1'bx` to equal `1'bx`. Similarly with ``wrealXState`, the sum `3.14+`wrealXState` must be equal to ``wrealXState`.

The following table provides some examples of results of logic and wreal arithmetic operations:

Table 2 Examples of Logic and Wreal Arithmetic Operations

Logic		Wreal	
Expression	Result	Expression	Result
<code>1'b1+1'bx</code>	<code>1'bx</code>	<code>3.14+`wrealXState</code>	<code>`wrealXState</code>
<code>1'b1+1'bx</code>	<code>1'bx</code>	<code>3.14+`wrealXState</code>	<code>`wrealXState</code>
<code>1'b1+1'bx</code>	<code>1'bx</code>	<code>`wrealXState+`wrealXState</code>	<code>`wrealXState</code>
<code>1'b1 && 1'bz</code>	<code>1'bx</code>	<code>3.14 && `wrealZState</code>	<code>`wrealXState</code>

Viewing Runtime Interface Elements in nWave

Verdi supports several features for force/release for analog designs, similar to the features available for digital designs. For example, Verdi supports the symbol `^` prefixed before the annotation value and the force/release icon.

Note:

This feature is supported for analog signals only if the simulator dumps the information to FSDB for the merged FSDB flow; split mode is not supported.

For analog designs, you can use following commands to perform force/release operations.

Note:

To dump the information, set the environment variable `FSDB_FORCE` to 1.

These commands instruct the simulator to dump force/release events on analog signals. However, this information is only recorded on voltage signals whose format is `v(net)`.

- `force/release`
- `hdl_xmr/hdl_xmr_force/hdl_xmr_release`
- `force_node_voltage/release_node_voltage`
- `iforce_node_voltage/irelease_node_voltage`

Limitations

This feature only supports `v(net)`; other value format such as `vn(device)`, `vsub(net)`, `in(device)`, `isub(net)`, `net`, `ln(device)`, `lsub(net)` are not supported.

The subsequent sections discuss the feature in detail:

- [Viewing Force/Release Events in nWave](#)
- [Searching by Forced/Deposited/Released Events](#)
- [Reporting Selected Signals](#)

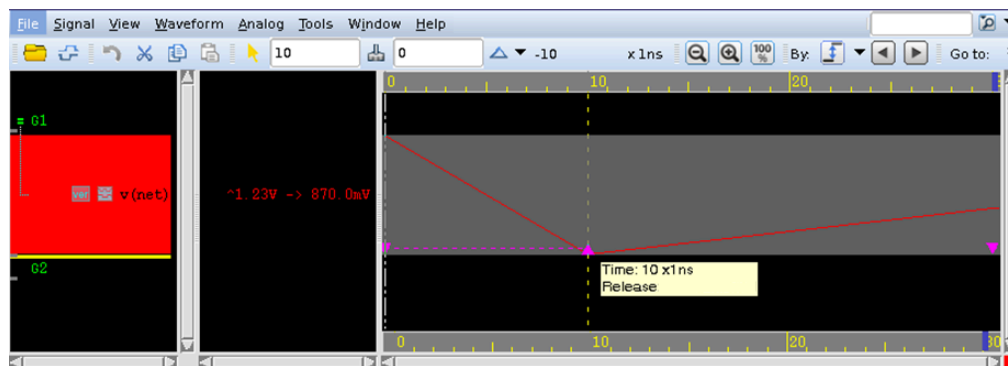
Viewing Force/Release Events in nWave

You can use *nWave* to view force/release information in *Value* and *Waveform* panes, once this information is dumped in FSDB.

Displaying Force/Release Events in the Value Pane

The *Value* pane displays **Forced Time Range** with the symbol '^' prefixed as shown in the following figure:

Figure 66 Force/Release Support in nWave



Viewing Force/Release Events in the Waveform Pane

The *Waveform* pane displays the **Force/Release** icon on the waveform similar to how it is displayed for digital waveforms. The following icons are displayed on the waveform based on the event:

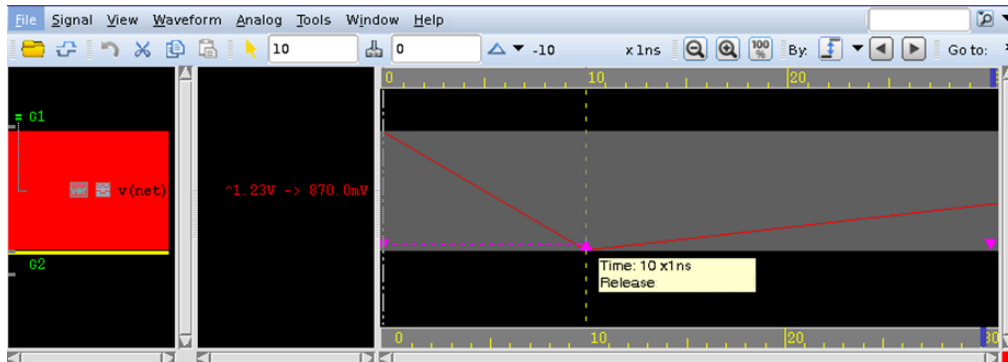
- **Force** is denoted as inverted solid triangle ▼.
- **Release** is denoted as a solid triangle ▲.
- **Forced** status is denoted as a line ———.
- Normal pair of *forced to released events* is denoted with **Force** and **Release** icons connected with the **Forced** status line ▼———▲.
- **Force/Release** can be denoted at the same time as ✕.

You can view the force/release event time and type using the **Enable Tip** option. To enable cursor tip information on the **Force/Release** icon, invoke the command **Tools -> Preferences -> View Options -> Waveform Pane -> Enable Tip**.

When this option enabled, the **Force/Release** event time and type is displayed in the *Waveform* pane as shown in the following figure.

You can double-click on the waveform to trace the **Force/Released Value** of an analog signal similar to digital signals. *nWave* triggers **Active Trace** when you double-click on the **Force/Release** icon in the waveform.

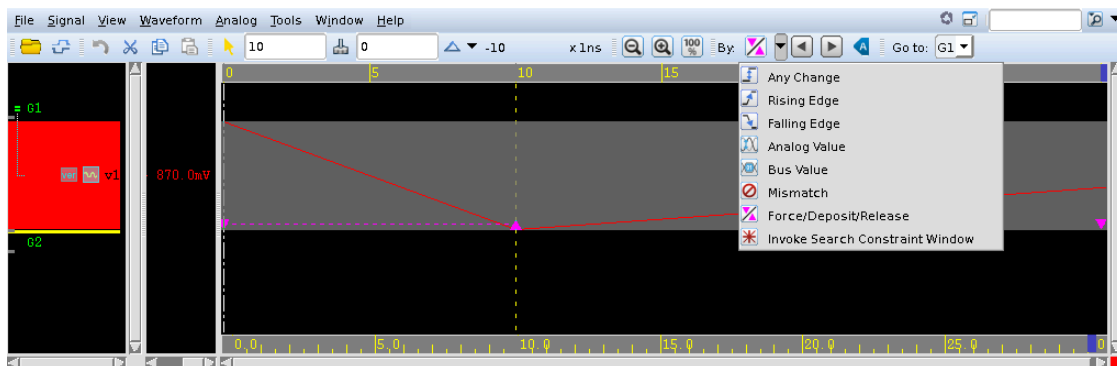
Figure 67 Force/Release Support in nWave



Searching by Forced/Deposited/Released Events

Similar to digital signals, you can search by **Forced/Deposited/Released** events for analog signals as shown in the following figure:

Figure 68 Search by Force/Deposited/Release



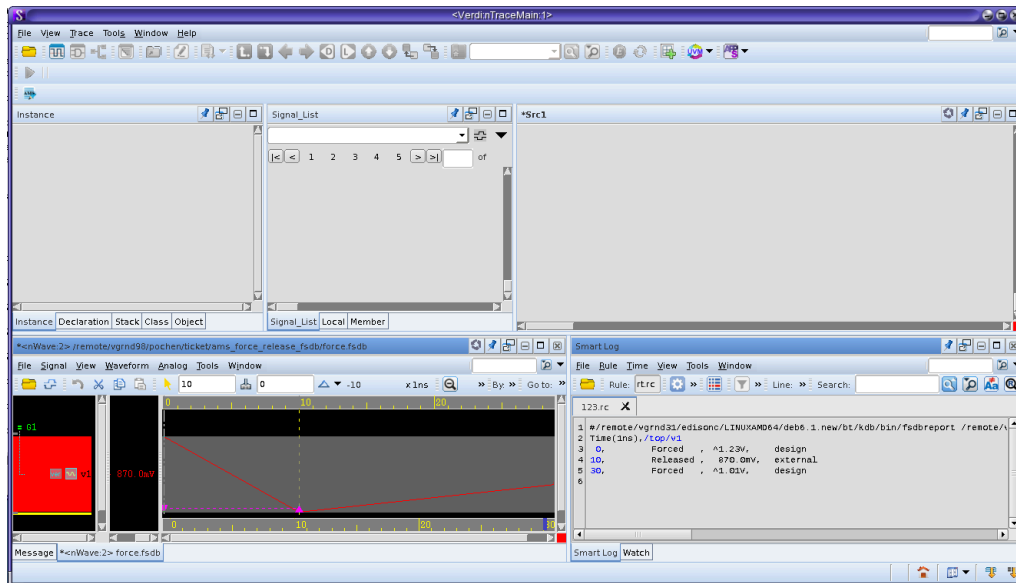
Reporting Selected Signals

You can report selected signals using the **File -> Report Selected Signals...** option. You can specify the option `-find_forces` in the **Options** field. Verdi reports forced/ released time and value in an output text file.

You can open this output text file using the **Output Form**. You can click the signal name (indicated by the blue text) to add the signal to *nWave* if it has not been added previously.

You can also click the time (indicated by the blue text) to change the cursor time as shown in the following figure:

Figure 69 Report Selected Signals' Force/Release Events and Open in Output Form



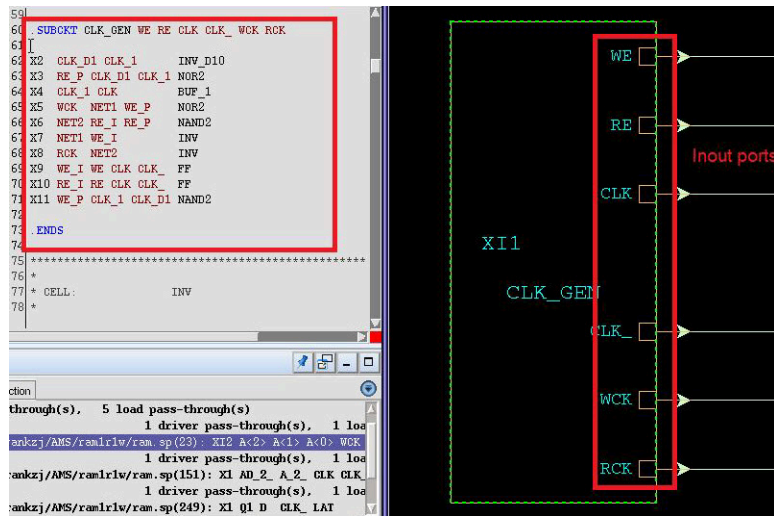
Debugging in the Schematic View

You can load and debug a mixed signal in *nSchema* similar to *nWave*. The following sections discuss these operations in detail:

- [Show Focus Connection](#)
- [Show Current Debug Focus Connection](#)
- [Viewing Interface Elements in nSchema](#)
- [Viewing Runtime Interface Elements in nSchema](#)
- [Viewing Through-Nets in nSchema](#)
- [Color-Coding Signals and Blocks in nSchema](#)
- [Mapping Symbols and Models in nSchema](#)

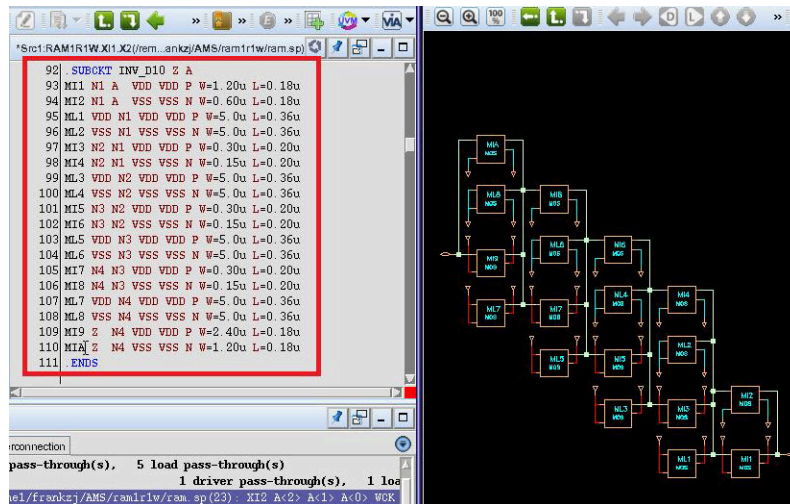
To debug in the schematic view, click the  to invoke the *nSchema* window for the selected scope.

Figure 70 Ports of SPICE Cell



You can double-click a subckt cell to push view into it:

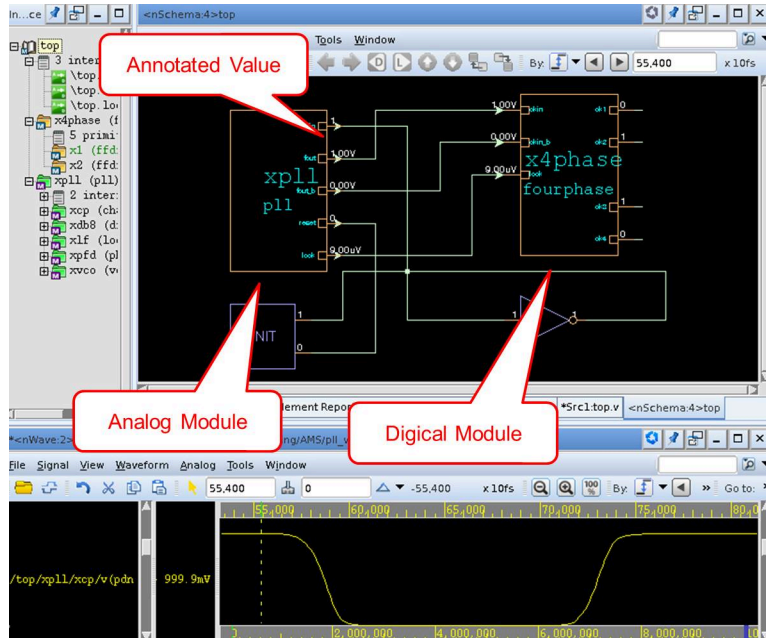
Figure 71 Inout Ports of SPICE Cell



After the file containing digital and analog FSDb files is loaded into the Verdi platform, you can also see the analog values in the *nSchema* window. In the *nSchema* window, use the **Schematic -> Active Annotation** option to annotate the current value as shown in the

following figure. The settings in the *Preference* form (see section [Active Annotation](#)) are also applied in the *nSchema* window:

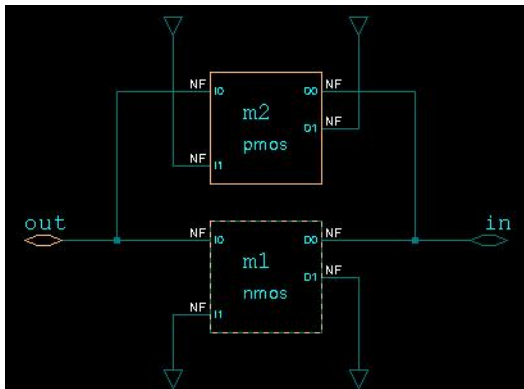
Figure 72 Active Annotation in *nSchema*



Drag and drop features are available for SPICE signals. For example, you can drag and drop a SPICE signal between the *nSchema* and *Source Code* panes to highlight corresponding SPICE cell/signals.

All non-active tracing features are available for SPICE signals, that is, **Active Fan-in** tracing and Trace-X features are not supported in the *nSchema* window. For example, you can select an analog signal and use the **Trace -> Fan-out** option to highlight the result of the fan-out as shown in the following figure:

Figure 73 Trace Fan-out



Show Focus Connection

In some scenarios, a debug session starts by adding the base signal to *nWave*. To debug it further, you can drag and drop the signal from *nWave* to *nSchema*. The schema thus created may be quite complex as the current signal is often dissipated to many sub-blocks. It is therefore difficult to find the block that you might want to debug. You can use the menu **Schematic -> Show Focus Connection** option to find the appropriate blocks in *nSchema* as shown in the following figure:

The **Show Focus Connection** option traces the connections (driving instance to loading instances with connecting net) of the selected net, port, or instance pin and summarizes the results in the **Show Focus Connection** form as shown in the following figure.

Note:

This option is available in full hierarchical and flattened schematic windows. It is not available in partial hierarchical windows.

If another net, port, or instance pin is selected in the current schematic window, the **Show Focus Connection** form is automatically updated to display the new connection results. If a net, port, or instance pin is selected in another schematic window, the form is not updated automatically and the option must be invoked again.

Note:

If the **Focus Connection** option is invoked while the **Show Focus Connection** form is open, the form is cleared and the results are highlighted only in the schematic window.

Figure 74 Show Focus Connection

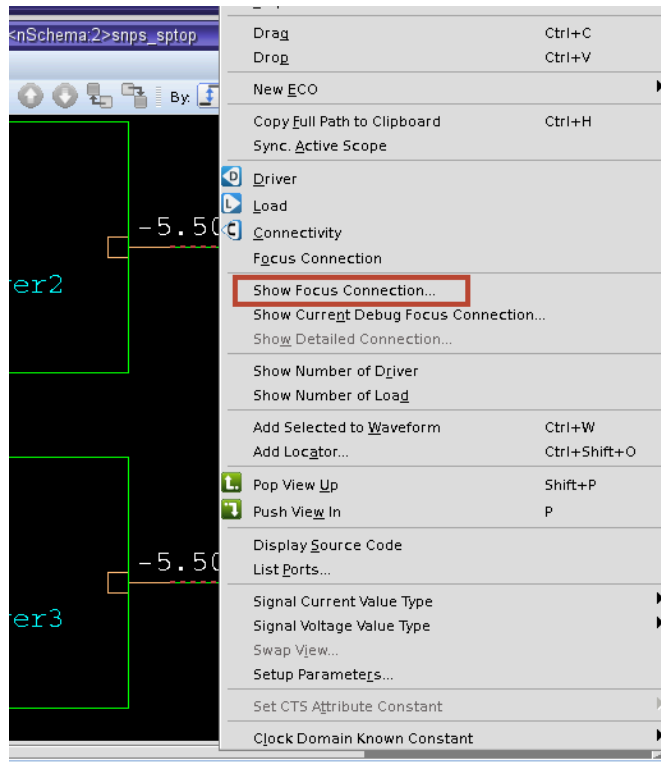
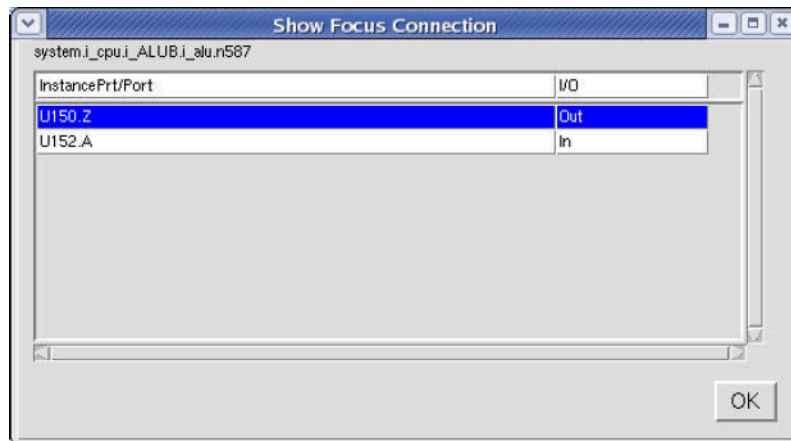


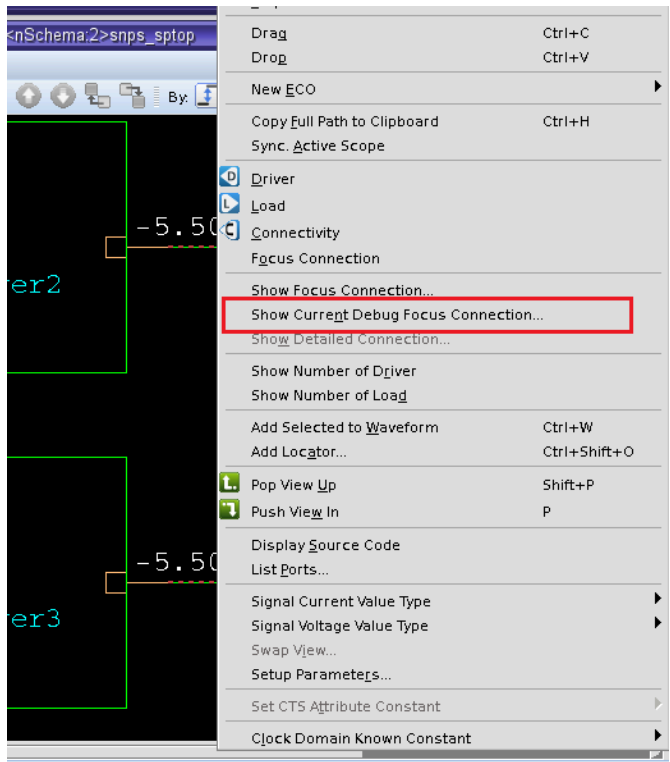
Figure 75 Show Focus Connection Dialog Box



Show Current Debug Focus Connection

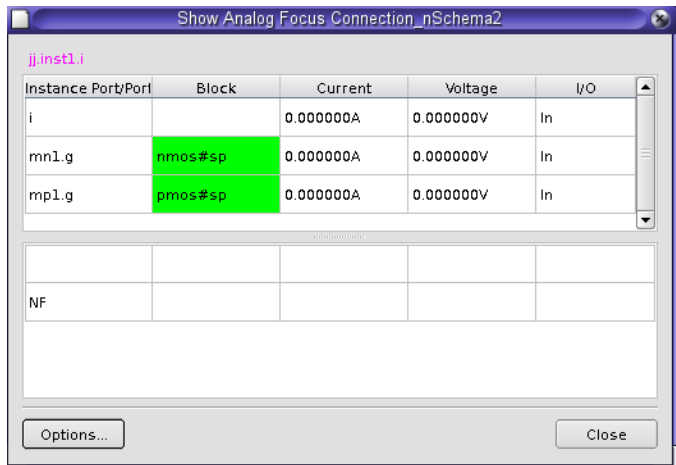
nSchema provides the **Schematic -> Show Current Debug Focus Connection** option that shows the current/voltage annotations of a selected net, port, or instance pin.

Figure 76 Show Current Debug Focus Connection Menu Item



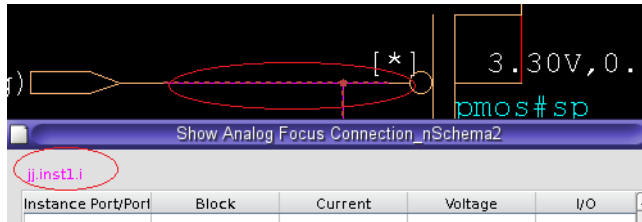
On selecting this option, the **Show Analog Focus Connection** dialog box displaying the block, current, voltage and I/O pin information is displayed as shown in the following figure.

Figure 77 Connections for a Net



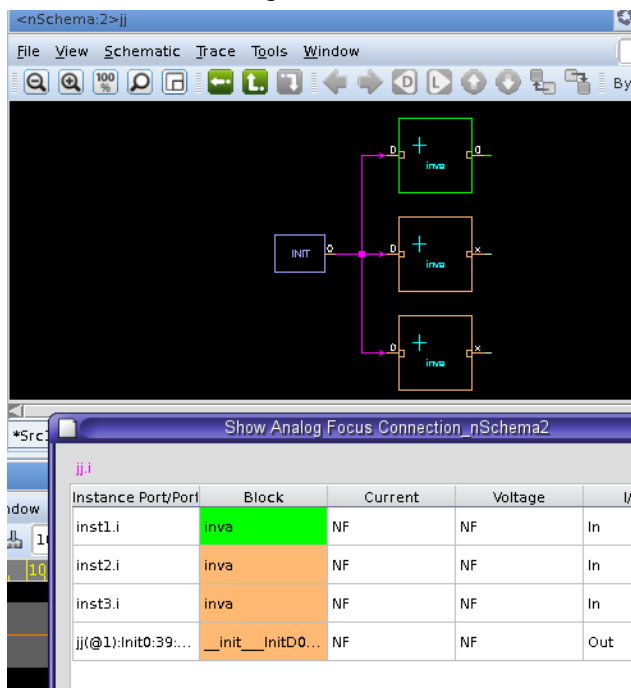
The name of the net has the same color in both the *nSchema* window and the **Show Analog Focus Connection** form, as shown in the following figure:

Figure 78 Color-coding of Nets



Similarly, the color of the net in the **Block** column of the **Show Analog Focus Connection** form is the same as color of the net in the *nSchema* window, as shown in the following figure:

Figure 79 Color-coding on Nets



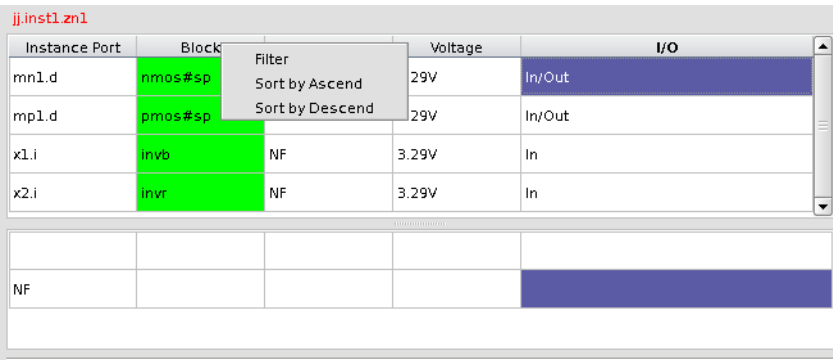
The dialog box displays the following information for a selected connection:

- [Block Information](#)
- [Current Information](#)
- [Voltage Information](#)
- [I/O Port Information](#)

Block Information

The block information associated to a connection is displayed as shown in the following figure. You can filter the block information (**Filter**) or sort in ascending (**Sort by Ascend**) or descending (**Sort by Descend**) order.

Figure 80 Sort and Filter Option for Block Column

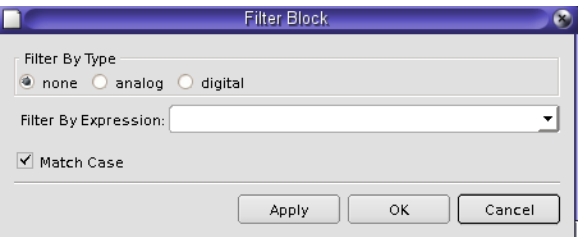


The screenshot shows a table titled 'jj_inst1.zn1' with columns: Instance Port, Block, Filter, Voltage, and I/O. A context menu is open over the 'Block' column, showing options: Filter, Sort by Ascend, and Sort by Descend. The table contains the following data:

Instance Port	Block	Filter	Voltage	I/O
mn1.d	nmos#sp	Sort by Ascend	29V	In/Out
mp1.d	pmos#sp	Sort by Descend	29V	In/Out
x1.i	invb	NF	3.29V	In
x2.i	invr	NF	3.29V	In
NF				

On selecting the **Filter** option, the **Filter Block** form is displayed as shown in the following figure. The dialog offers various filtering criteria, such as filtering analog or digital signals using the **Filter By Type**. You can also filter using an expression using the **Filter By Expression** option. You can specify whether the search operation should consider the case or not by selecting the **Match Case** option.

Figure 81 Filtering Criteria for Block Column

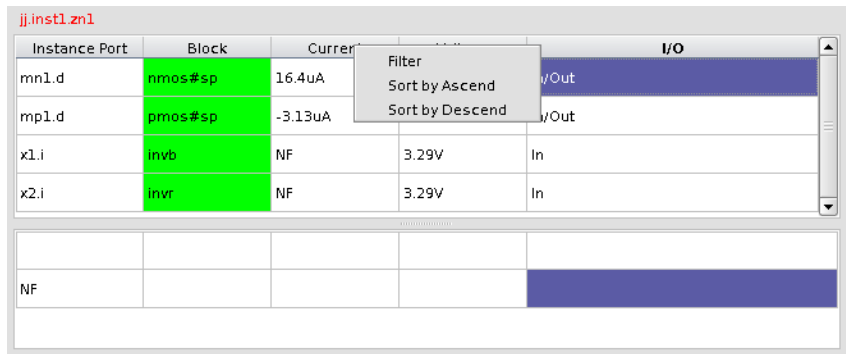


The screenshot shows the 'Filter Block' dialog box. It has a 'Filter By Type' section with radio buttons for 'none' (selected), 'analog', and 'digital'. Below it is a 'Filter By Expression' dropdown menu. There is a checked checkbox for 'Match Case'. At the bottom are 'Apply', 'OK', and 'Cancel' buttons.

Current Information

The current information associated to a connection is displayed as shown in the following figure. You can filter the current information (**Filter**) or sort in ascending (**Sort by Ascend**) or descending (**Sort by Descend**) order.

Figure 82 Sort and Filter Option for Current Column

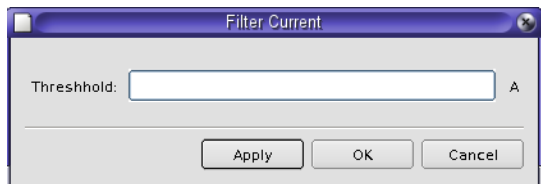


The screenshot shows a table with columns: Instance Port, Block, Current, Voltage, and I/O. The 'Current' column is highlighted, and a context menu is open with options: Filter, Sort by Ascend, and Sort by Descend. The table data is as follows:

Instance Port	Block	Current	Voltage	I/O
mn1.d	nmos#sp	16.4uA		Out
mp1.d	pmos#sp	-3.13uA		Out
x1.i	invb	NF	3.29V	In
x2.i	invr	NF	3.29V	In
NF				

On selecting the **Filter** option, the **Filter Current** dialog is displayed as shown in the following figure. You can enter the threshold value using which the current values are to be filtered in the **Threshold** field.

Figure 83 Filtering Criteria for Current Column

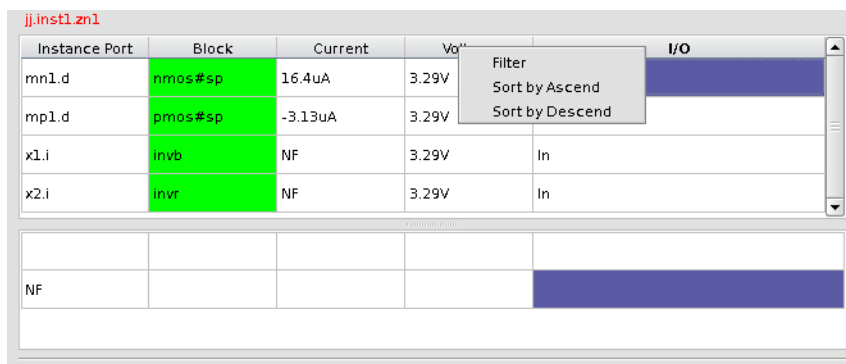


The screenshot shows the 'Filter Current' dialog box. It has a 'Threshold:' label followed by a text input field and a unit 'A'. Below the input field are three buttons: 'Apply', 'OK', and 'Cancel'.

Voltage Information

The voltage information associated to a connection is displayed as shown in the following figure. You can filter the voltage information (**Filter**) or sort in ascending (**Sort by Ascend**) or descending (**Sort by Descend**) order.

Figure 84 Sort and Filter Option for Voltage Column

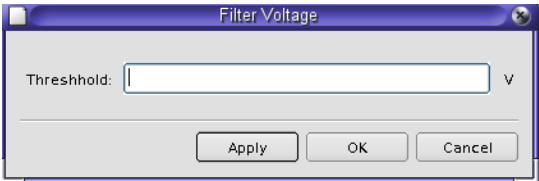


The screenshot shows a table with columns: Instance Port, Block, Current, Voltage, and I/O. The 'Voltage' column is highlighted, and a context menu is open with options: Filter, Sort by Ascend, and Sort by Descend. The table data is as follows:

Instance Port	Block	Current	Voltage	I/O
mn1.d	nmos#sp	16.4uA	3.29V	Out
mp1.d	pmos#sp	-3.13uA	3.29V	Out
x1.i	invb	NF	3.29V	In
x2.i	invr	NF	3.29V	In
NF				

On selecting the **Filter** option, the **Filter Voltage** form is displayed as shown in the following figure. You can enter the threshold value using which the voltage values are to be filtered in the **Threshold** field.

Figure 85 Filtering Criteria for Voltage Column



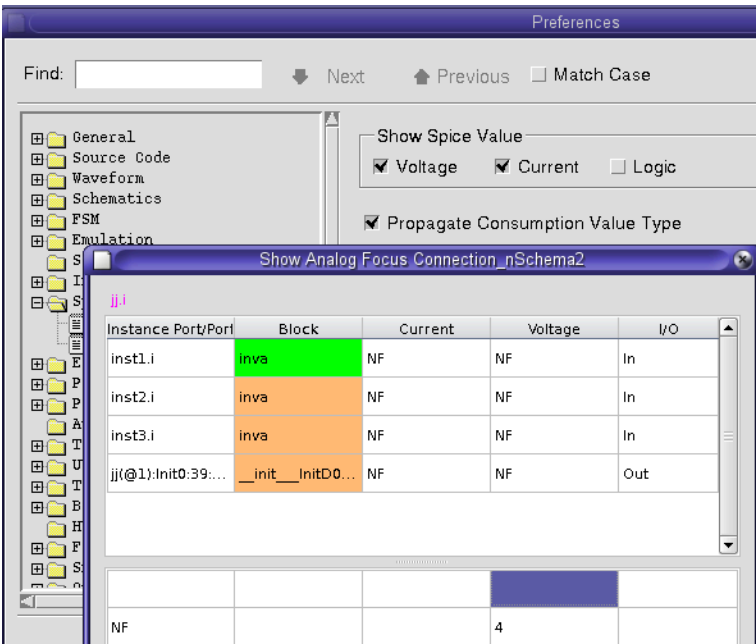
I/O Port Information

Displays the in/out/in-out port information.

Preference Setting for Show Current Debug Focus Connection

You can show or hide the current/voltage column by selecting or clearing the **Voltage** and **Current** values in the *Preferences* form, as shown in the following figure:

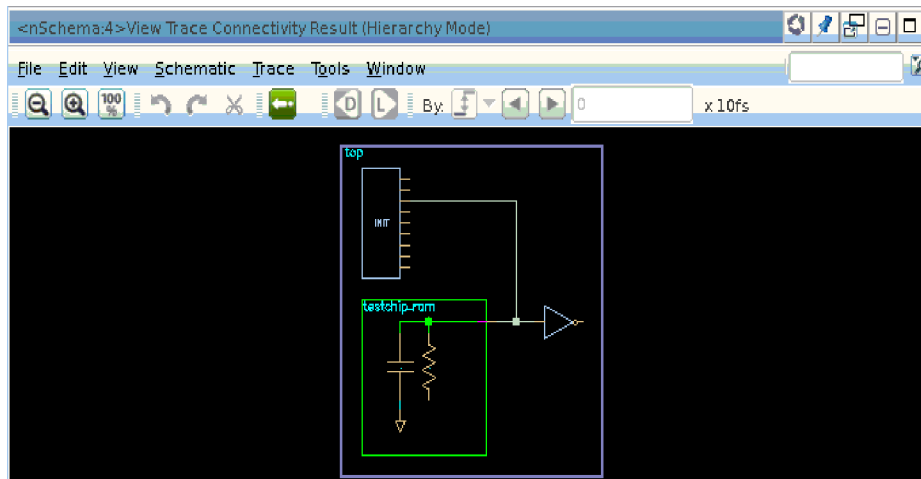
Figure 86 Preference Setting for Voltage/Current



Viewing Interface Elements in nSchema

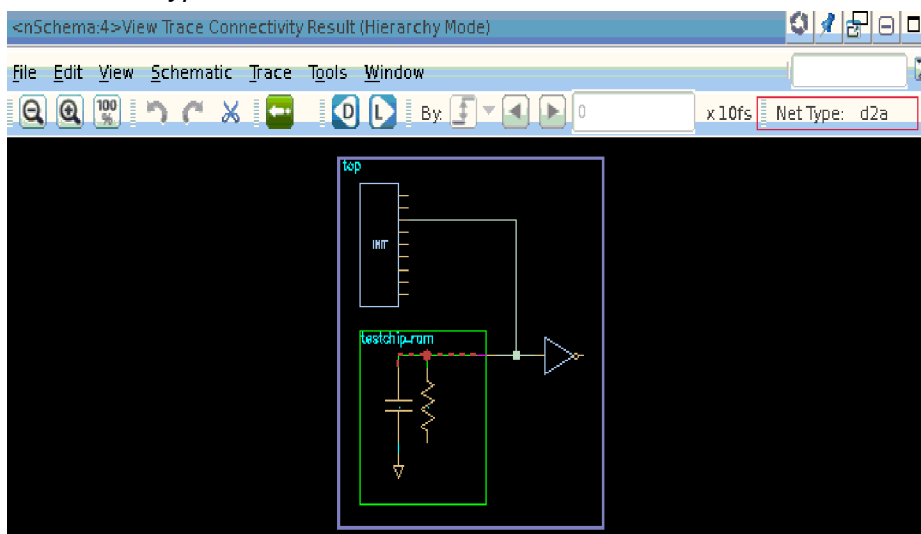
nSchema displays interface elements in *nSchema*, as shown in the following figure. The default color of the interface element is green.

Figure 87 Interface Element in nSchema



If the interface element is selected or clicked, the complete element is highlighted and the type of this interface element is displayed in the toolbar, as shown in the following figure. The nettype of this element is d2a.

Figure 88 Nettype of the Interface Element



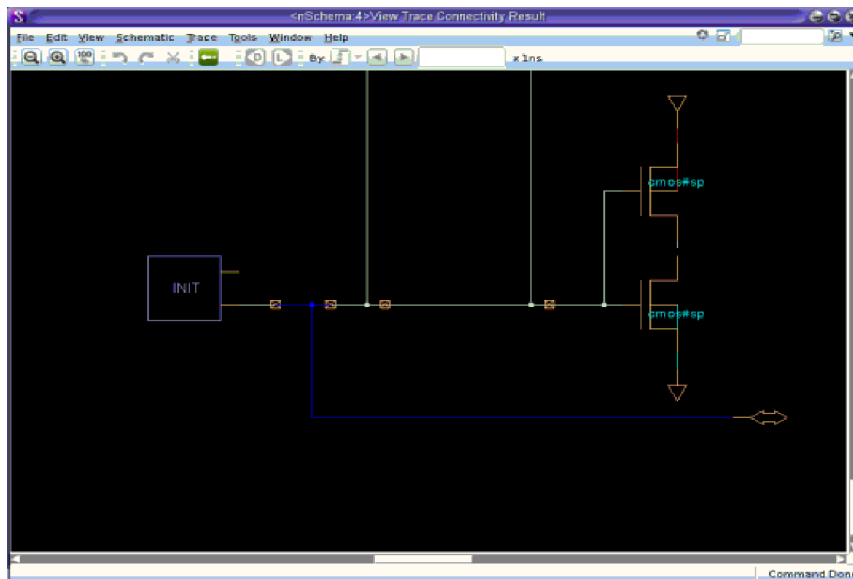
Note:

The digital and analog values are annotated as a special symbol (^?) to depict that it is a forced value.

Viewing Runtime Interface Elements in nSchema

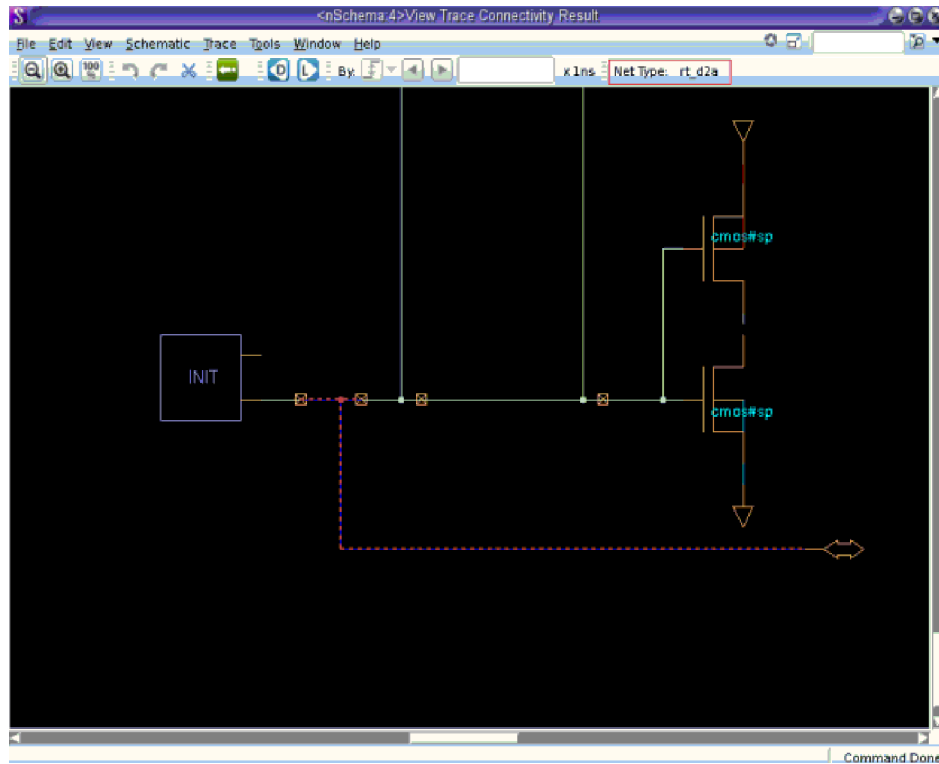
nSchema displays interface elements created at runtime in *nSchema*. These interface elements are either created by using the `force`, `hdl_xmr`, or `hdl_xmr_force` options. These interface elements are displayed in blue, as shown in the following figure:

Figure 89 Display of Runtime Interface Elements



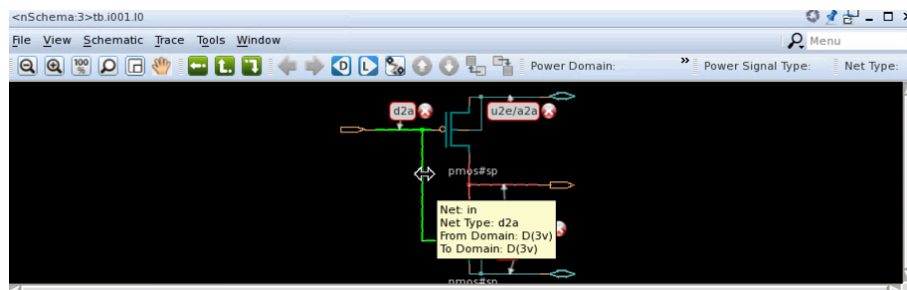
If the runtime interface element is selected or clicked, the complete element is highlighted and the type of this interface element is displayed in the toolbar, as shown in the following figure. The nettype of this element is `rt_d2a`.

Figure 90 Nettype of the Runtime Interface Element



A tooltip is shown when you hover cursor on the interface element as shown in the following figure:

Figure 91 Example for Tooltip of Interface Elements

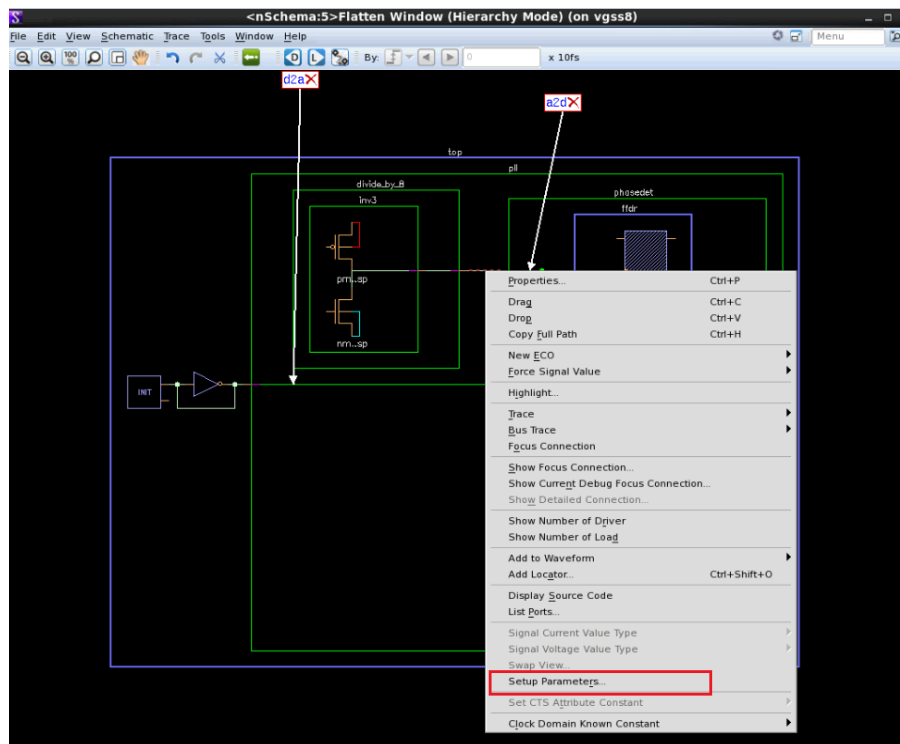


Editing the Interface Elements in Verilog-AMS Under Interactive Debug Mode in the nSchema Window

To edit the IE elements, perform the following steps:

1. Right-click or drag-and-drop the a2d, d2a, e2n, n2e, e2r, r2e node(s) or the segments in the *nSchema* window and select **Setup Parameters** option, as illustrated in the following figure:

Figure 92 Setup Parameters



You can drag-and-drop the node(s) or the segments in the *nSchema* window.

2. The *AMS Configuration* form appears containing the following three tabs, as illustrated in the following figure:

Figure 93 AMS Configuration

AMS Configuration (on vgss8)

A/D E/R E/N

Analog to Digital interface node:

	primary_name	node	loth	hith	hiz	minv	minv_logic	midv_tir
1	top_xpll.xpfd.fb	top_xpll.xpfd.fb	0.5v	0.5v				
2	+							

Digital to Analog interface node:

	primary_name	node	rf_time	rise_time	fall_time	delay	x2v	hiv
1	top_xpll.fin	top_xpll.fin						1v
2	+							

Apply OK Cancel

Note:

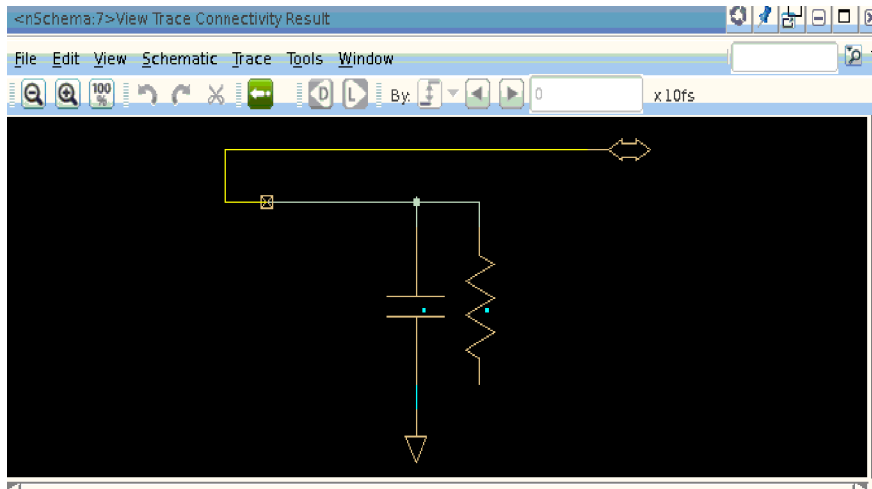
A warning message appears if the selected segments belong to same IE group.

For more details about the form, refer the [Editing the Interface Elements in Verilog-AMS Under Interactive Debug Mode in the nTrace Window](#) section.

Viewing Through-Nets in nSchema

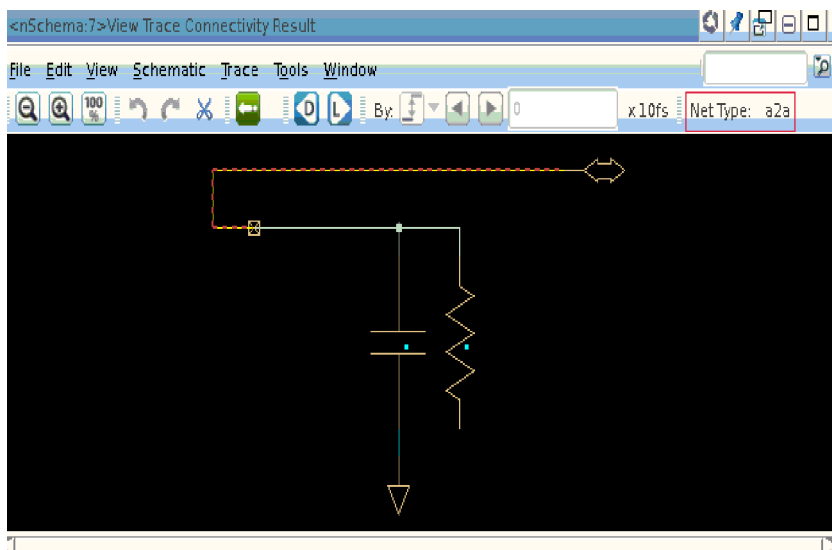
nSchema displays through-nets in the source viewer and schematic viewer, as shown in the following figure. The default color of the through-net is yellow.

Figure 94 Through-Net in nSchema



If the through-net is selected or clicked, the complete net is highlighted and the type of this through-net is displayed in the toolbar, as shown in the following figure.

Figure 95 Nettype of the Through-Net



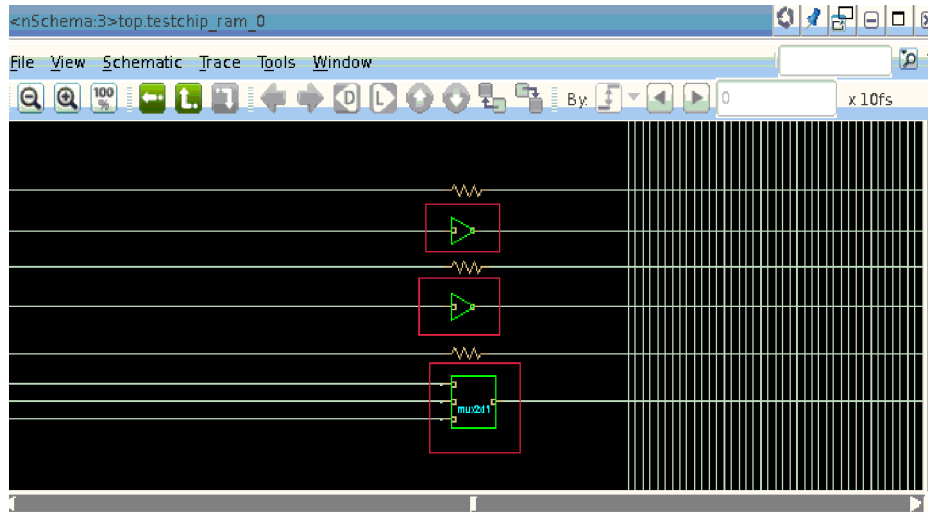
Color-Coding Signals and Blocks in nSchema

nSchema color-codes the following signals and blocks:

- Analog or digital blocks
- Analog, digital, or mixed signal signals

The analog blocks are displayed in green, as shown in the following figure:

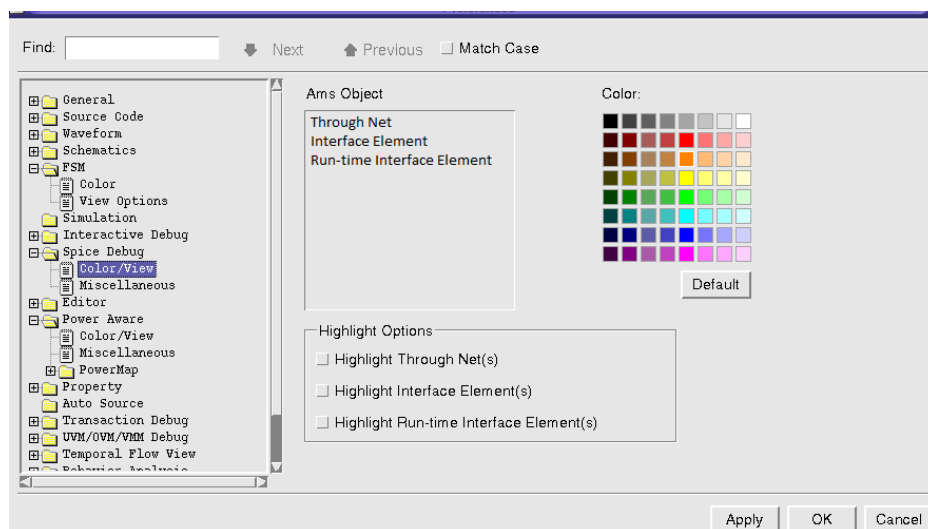
Figure 96 Display of Analog Blocks



When you open the *nSchema* window, all analog signals are displayed in the default color and you can change the default color of signals also.

You can also select non-default colors for analog, digital and mixed signals from the menu, as shown in the following figure. However, there might be performance issues if non-default colors are selected.

Figure 97 Choosing Custom Colors for Signal Representation



If you have set the color selection, then it is reflected in all *nSchema* windows.

Mapping Symbols and Models in nSchema

Verdi AMS allows you to define a symbol library (config file) for SPICE designs that maps symbols with models, in a `.csv` format. You can then import this config file along with your design into *nSchema* to map the model in your design with a specific symbol. You can also define a symbol for an empty model of your design.

The config file format must define one model per line, as follows:

```
[SYMBOL_TYPE], [MODEL_NAME], [PIN1;PIN2;PIN3...]
```

Where,

- **SYMBOL_TYPE**: Defines the symbol type. If the symbol type is not specified, *nSchema* uses macro symbols as the default symbol. The **SYMBOL_TYPE** can be any one of the types defined in table [Symbol Mapping Table](#).
- **MODEL_NAME**: Specifies the model names that you select as the special symbol in *nSchema*.
- **PIN**: Specifies the **PIN** names of the user model. The **PIN** names and the total number of **PIN**s must be equal to the model definition. The **PIN** number decides the kind of symbol that is shown in *nSchema*.

The following are examples of the mapping file entries:

```
NMOS,nch_mac_psp,D;G;S;B
NMOS,n_elvt_dnw_psp,D;G;S;B;ISO;SUBST
PMOS,pch_lvt_mac,D;G;S;B
RESISTOR,rmlw,PLUS;MINUS
PNP,pnp1d6_mis,C;B;E
.....
```

After you have created the config file, use the following command to import it into *nSchema*:

```
%spicom [filename.lvs] -lib TOP
%verdi -msv -spicetbl [filename.csv] -f [designfile]
```

nSchema loads this config file and creates a table for the design. If a model in the design source code has the same name as the one in the mapping table, the model symbol is replaced with the specific symbol from the table. The **SYMBOL_TYPE** and **PIN** number decides the kind of symbol that is shown.

Note:

Reloading the design reloads the mapping file. Use the **Save/Restore Session** option to keep the config table settings.

Save/Restore session keeps/imports the mapping file. The segment of session file in under [hb] and the key word as follows:

```
spiceTblFile = filename(absolute path)
```

The direction of the symbol pin uses the rules of the current SPICE model pin direction, if there are no other suggestions provided. The default pin direction is *inout*.

Table 3 Symbol Mapping Table

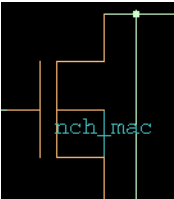
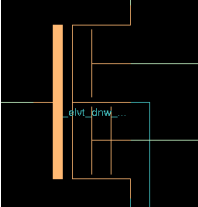
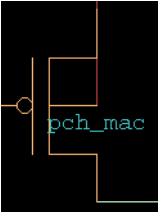
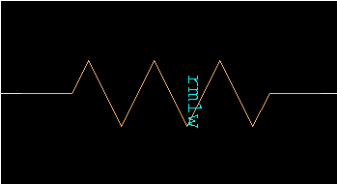
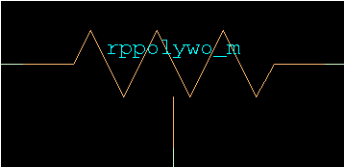
SYMBOL_TYPE	SYMBOL	PIN NUMBER
NMOS		4 Pins
NMOS		6 Pins
PMOS		4 Pins
RESISTOR		2 Pins
		3 Pins

Table 3 Symbol Mapping Table (Continued)

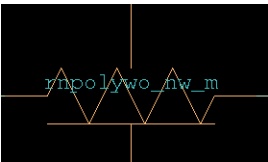
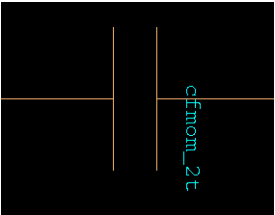
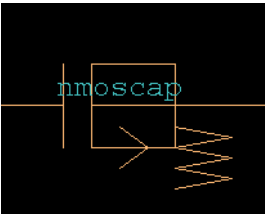
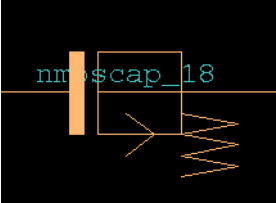
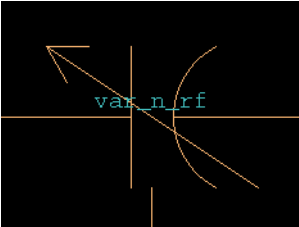
		4 Pins
MOMCAP		2 Pins
		3 Pins
NMOSCAP		2 Pins
NMOSCAP_18		2 Pins
VAR_N_RF		3 Pins

Table 3 Symbol Mapping Table (Continued)

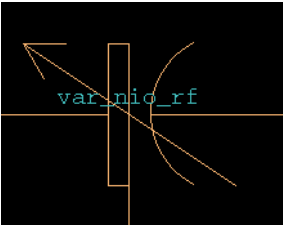
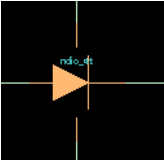
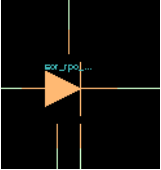
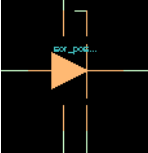
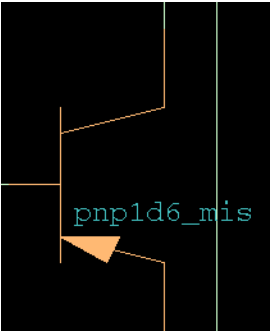
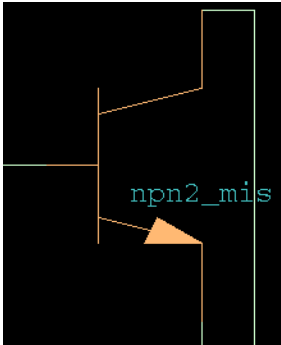
VAR_NIO_RF		2 Pins
DIODE		2 Pins
		4 Pins
		5 Pins
DIODE		6 Pins
PNP		3 Pins

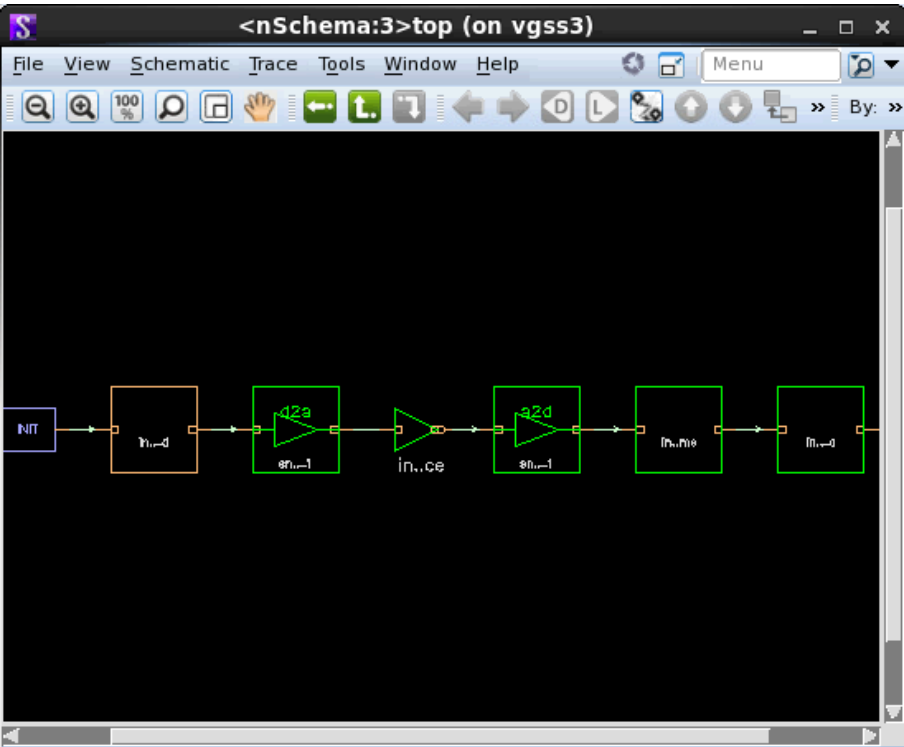
Table 3 Symbol Mapping Table (Continued)

NPN		3 Pins
-----	---	--------

Support for Verilog-AMS in the nSchema Window

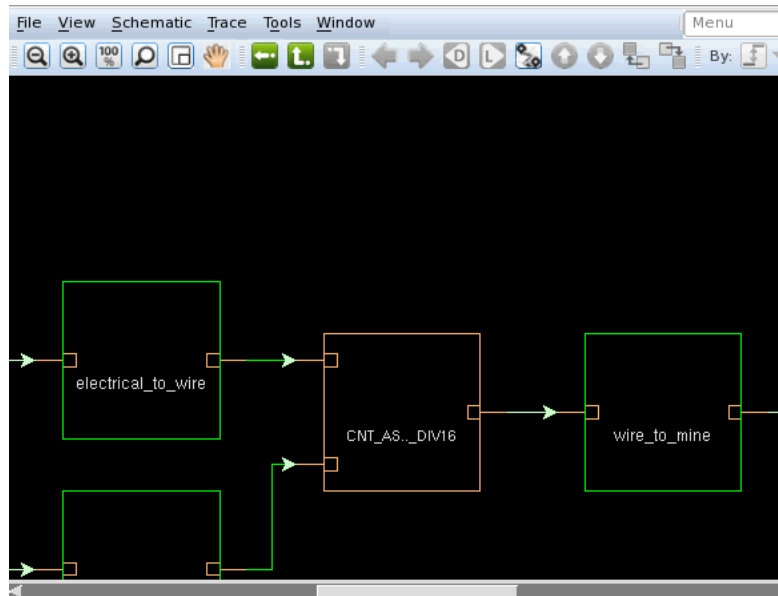
The Verilog-AMS instances in the *nSchema* window appear in green color, as illustrated in the following figure:

Figure 98 Verilog-AMS Instances in nSchema Window



Signals which are instrumented, that is, the signals that connect the connection module to another normal instance are also colored green (ID_GREEN5), as illustrated in the following figure:

Figure 99 Instrumented Signals in nSchema Window



Debugging in WaveView

You can debug in Verdi using **WaveView** as it offers state-of-the-art analog debugging capability, and it also helps in analysis of the measurement and calculator debugging functions.

Prerequisites

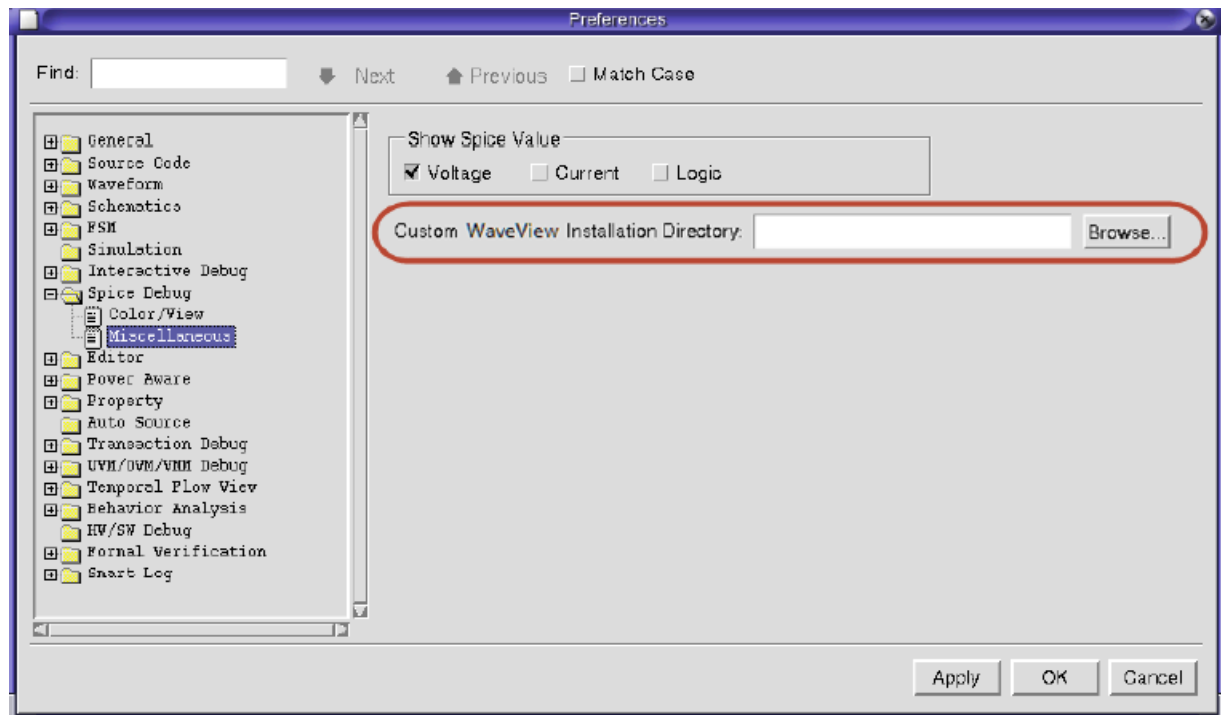
Before invoking **WaveView**, you must perform the following steps:

1. Set the **setenv NOVAS_WAVEVIEW_BETA** environment variable.
2. Import your design with the `-msv` option.
3. Set the **WaveView** install directory using either of the following options:
 - Set the **setenv NOVAS_WAVEVIEW_DIR \$WaveViewInstallDir** environment variable.

Or

- Specify the **Custom WaveView Installation Directory** in the *Preferences* form as shown in the figure [Preference Setting for WaveView](#).

Figure 100 Preference Setting for WaveView



Invoking WaveView

To invoke **WaveView**, choose **Tools -> Custom WaveView** in *nTrace* and *nWave* as shown in the figures [WaveView for nTrace](#) and [WaveView for nWave](#) respectively.

Note:

If you invoke **WaveView** without executing the prerequisites, a warning message is displayed.

Figure 101 WaveView for nTrace

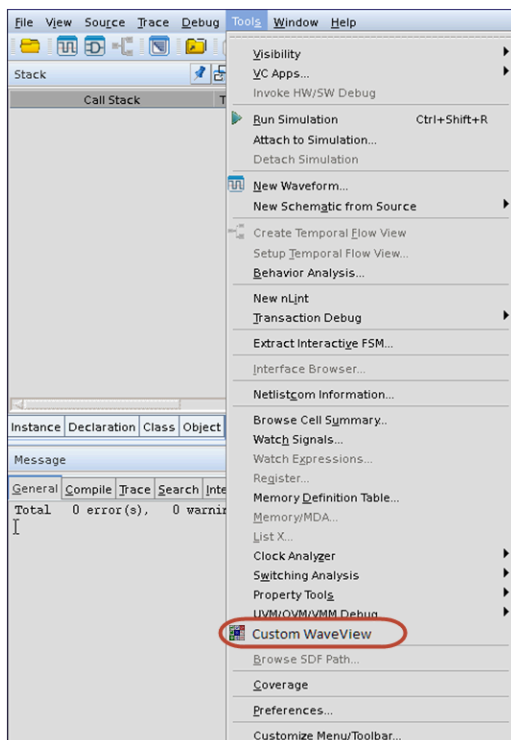
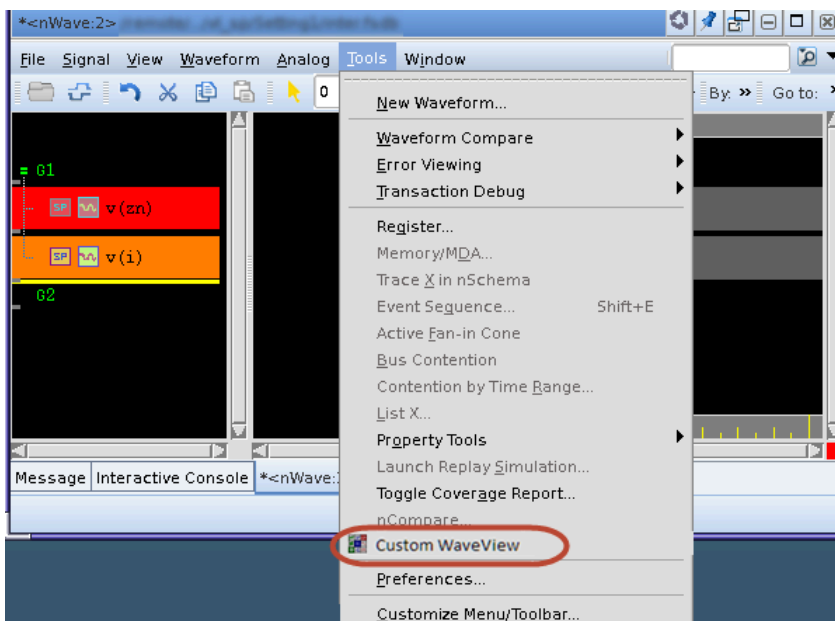


Figure 102 WaveView for nWave



You can also select **WaveView** from the toolbar item as shown in the figure [WaveView Toolbar Item](#):

Figure 103 WaveView Toolbar Item



After you select **Custom WaveView** in the menu, the **WaveView** tab is automatically added and sets the layout to *stack layout*.

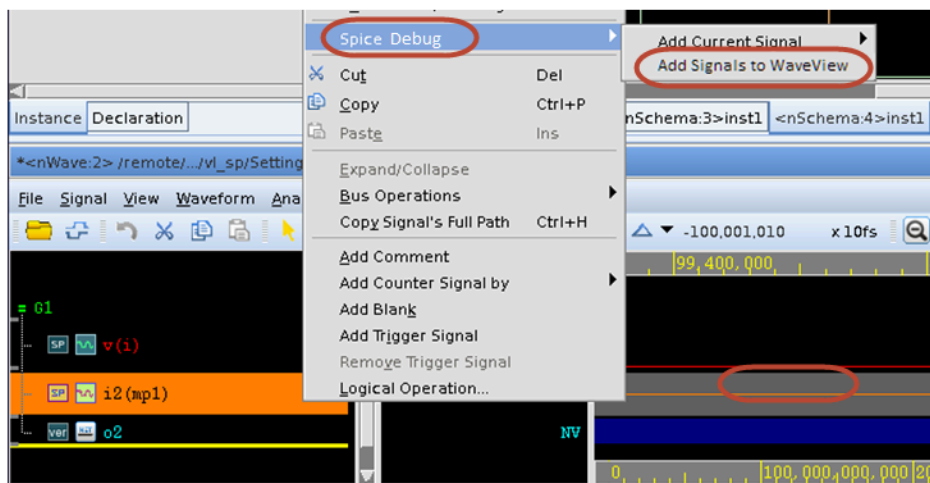
Note:

Stand-alone *nWave* does not support this feature.

Adding Signals to WaveView

To add selected signals to **WaveView**, right-click and select **Add Signals to WaveView** command in *nWave* as shown in the figure [Right-click Option in nWave](#):

Figure 104 Right-click Option in nWave



Note:

These menu options are only visible when the prerequisites are all set (as mentioned in section [Prerequisites](#)) and a FSDB file is loaded in Verdi.

Invoking WaveView and Loading FSDB File Automatically

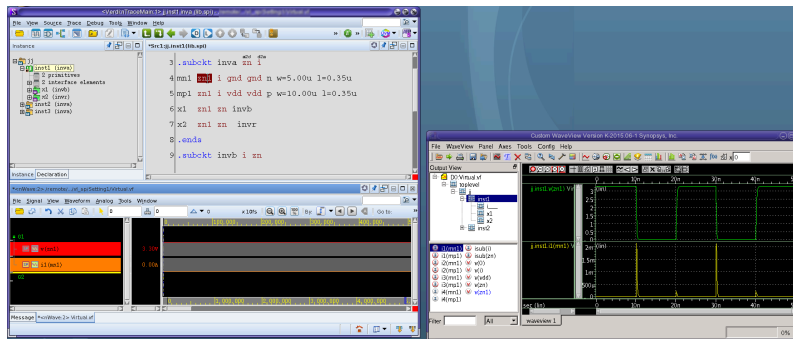
The **Add Signals to WaveView** command does not request you to invoke **WaveView** or load FSDB file in **WaveView** first. It invokes **WaveView** automatically if you have not invoked from Verdi. It also loads the corresponding FSDB file to **WaveView**.

After you have invoked **WaveView** and opened an FSDB file, you can add signals to it.

To add signals into **WaveView**, perform the following steps:

1. Check if **WaveView** has opened the target FSDB file; if not, open the target FSDB file.
2. Retrieve signals from the target FSDB file.
3. Add signals into the current active **WaveView** tab.
 - A new panel is opened for each signal added to **WaveView**. As shown in figure [Add Signals from Verdi to WaveView](#), two signals are added to **WaveView**; hence two panels are created for each signal.
 - The current active **WaveView** tab does not allow adding new signals; in this case, another **WaveView** tab is opened automatically.
 - For example, **WaveView** does not allow adding the X-Y axis panel to a **WaveView** tab as it is in stack layout with non X-Y axis panels.

Figure 105 Add Signals from Verdi to WaveView

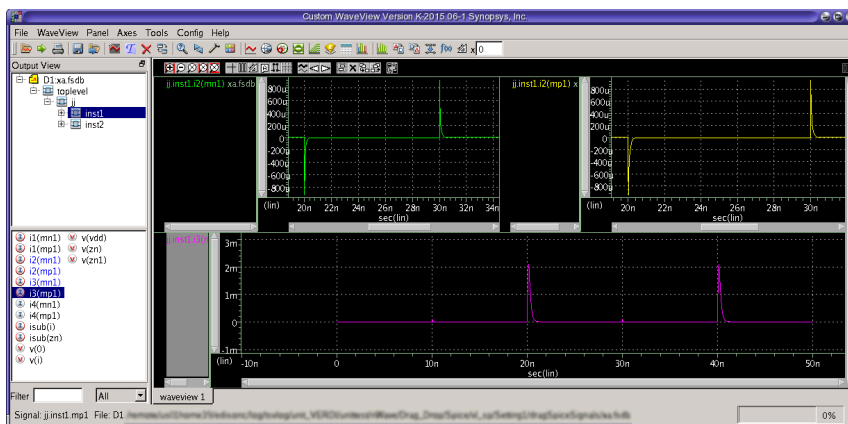


WaveView Tab Layout Limitation

WaveView supports various layouts to display its panels and it is not mandatory for you to change the layout to *stack layout*. Due to this, the following limitations are applicable:

- The signals to be added may be inserted to the upper right column of the current panel as shown in figure [WaveView Layout Limitation](#).
- The time range between each panel may be different. Hence, all the panels are synced to the first X-Y axis panel in the current **WaveView** tab. This is explained in more detail in section [Synchronizing Cursor and Viewing Range from nWave to WaveView](#).

Figure 106 WaveView Layout Limitation



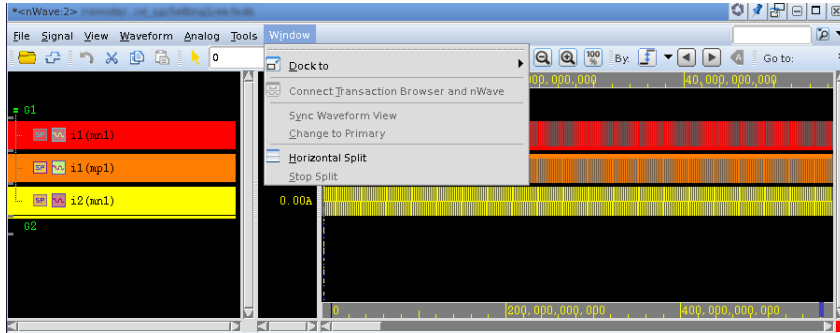
Synchronizing Cursor and Viewing Range from nWave to WaveView

nWave provides the **Window -> Sync Waveform View** option to sync view range and cursor to the other *nWave* windows. This option is also applicable to **WaveView** if it is already invoked by Verdi as shown in figure [Updating WaveView Waveform in Verdi Interactive Mode](#). This command is enabled only in the following scenarios:

- If more than one *nWave* or **WaveView** windows are invoked by Verdi.
- **Window -> Sync Waveform View** is disabled.

- Unidirectional connection from *nWave* to **WaveView** is available.

Figure 107 Menu Command to Sync View Range and Cursor to WaveView



Updating WaveView Waveform in Verdi Interactive Mode

This capability is only supported in the Verdi Interactive Mode. After you invoke this mode, Verdi checks if the file is updated and requests **WaveView** to update all **WaveView** tabs in the following scenarios:

- Simulation time is changed.
- New signals or scopes get dumped.

Limitations

The following are the limitations for this feature:

- **WaveView** updates the waveform only when the session is increased in the FSDB. Hence, it is still possible that the **WaveView** window does not get updated even if Verdi requests for the update.
- **WaveView** updates the signal list only when the file is regenerated. Hence, new dumped signals cannot be added to **WaveView** unless you manually select the file in **WaveView** and reload it using the right-click option **Reload WDF Signal List**.

Tracing Signals in the Temporal Flow View

To debug a design, you need to understand the structure and the behavior of the design. Visualizing the structure and behavior of the design is very useful and convenient to become familiar with the design and then debug it.

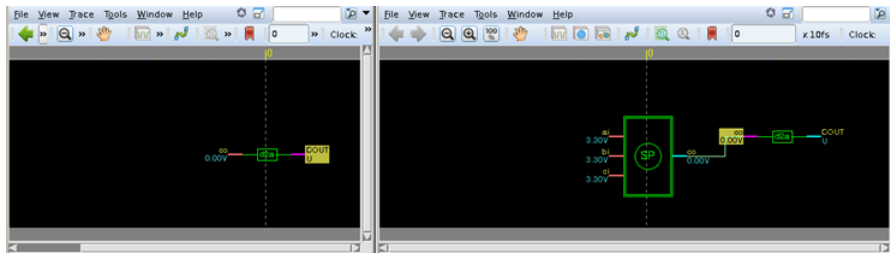
Temporal Flow View (TFV) provides flow views that are unique temporal views and analysis tools that allow the visualization and analysis of the design's behavior through time. TFV identifies and displays causal control and data paths - the registers and signals

that actually caused the erroneous value to occur - through multilevel combinational logic within one or more register-to-register transfer stages.

TFV provides the following capabilities for the analysis of analog and digital signals:

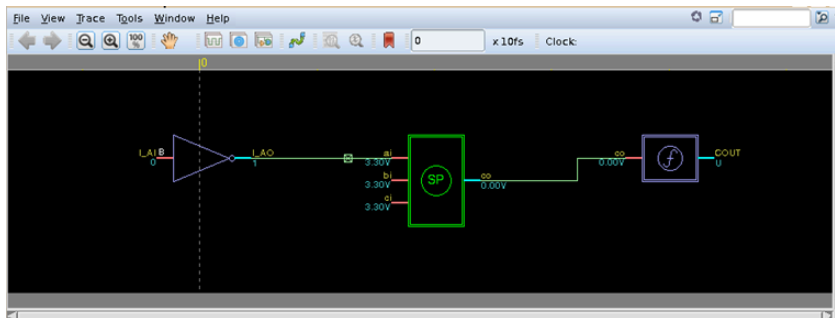
- Traces digital signals and analog signals on the tracing path using **Trace This Value** (cycle based) / **Trace Triggering Path** (transition based).
- Displays Interface Elements (IEs) and Through Nets (TNs), as shown in the following figure:

Figure 108 IE and TN Elements in TFV



- Uses function blocks with specific labels to represent analog blocks on TFV. TFV does not trace into the analog blocks; TFV only traces through analog blocks and uses a specific label as analog blocks, as shown in the following figure (the green symbol is an analog block symbol):

Figure 109 Tracing Through Analog Blocks in TFV



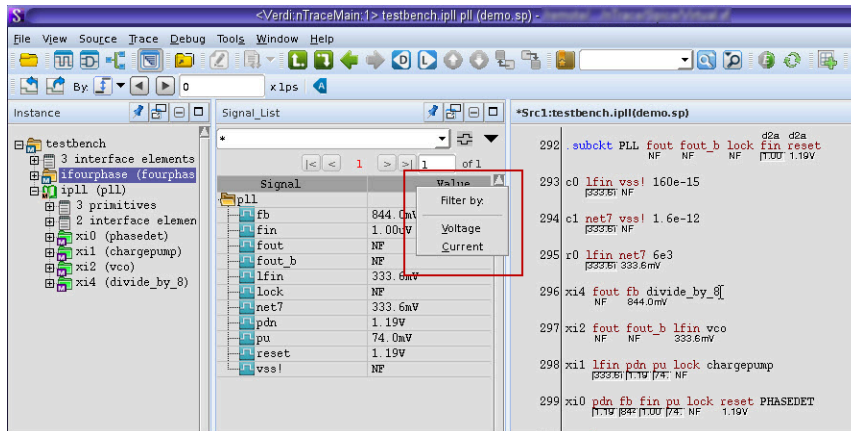
- Syncs with *nWave*, *nTrace*, and *nSchema* thereby showing the waveform, *Source Code* pane, and circuit detail of digital and analog signals.
- Allows you to drag and drop (D&D) signals to *nWave*, *nTrace*, and *nSchema*, showing the waveform, source code, and circuit, respectively.

For more information on TFV, see Chapter *Temporal Flow View Tutorial* in the *Verdi User Guide and Tutorial* and Chapter *Flow Views* in the *Verdi and Siloti Command Reference Manual*.

Filtering Signals Based on Signal Types in nTrace

You can filter signals in *nTrace*: Input, Output, Inout, Register, Wire, Other, and All. In addition, you can also filter signals captured in the FSDB file based on voltage and current as shown in the following figure:

Figure 110 Filtering Signals Based on Signal Type



In the *Signal_List* pane, right-click the *Value* column to open the *Filter* drop-down list and then select either *Voltage* or *Current* options.

These options are available for filtering only if:

- The *Mixed-Signal Verification* option is turned on.
- The *Active Annotation* option is turned on.

Mapping the Port Configuration

In AMS support for leading trace, the following options are supported and ensures that the pass-through across different design languages (Verilog/VHDL/Spice) is accurate.

In AMS designs, each block in the design can have definitions in more than one view. For example, a block can have both SPICE and Verilog definitions. A particular view of a multiview cell can also be selected explicitly by using `use_spice` or `use_verilog` mixed-signal options.

By default, mixed-signal simulation assumes that a multiview cell (for example, a cell with both Verilog and SPICE views) has the same number of ports and port names in both views. But, if the number of ports or the names of the ports differ between the two views then compilation errors are issued.

To resolve the port inconsistency problem between different views the `port_map` option of the `use_spice`, `use_verilog`, or `use_vhdl` options must be used.

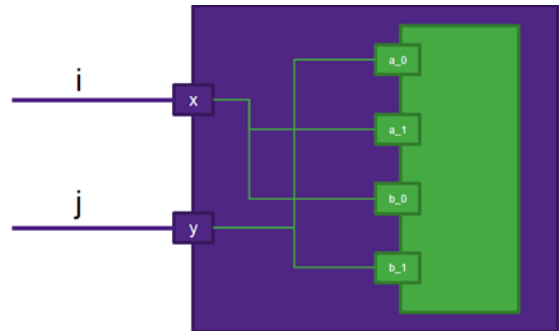
Also, to handle discrepancy in the number of ports (for example, if one view has extra ports), you can use the `port_connect` option.

The following table describes the mixed-signal commands and their descriptions:

Command	Description
<code>use_spice</code>	Used to explicitly instruct the tool to use the SPICEview of a multiview cell for a child cell under a Verilog or VHDL parent.
<code>use_verilog</code>	Used to explicitly instruct the tool to use the Verilog view of a multiview cell for a child cell under a SPICE parent.
<code>use_vhdl</code>	Used to select the VHDL view of a multiview cell instantiated in a SPICE block.

The following figure is an example of the working of the `port_map` option for Verilog parent modules with spice sub-module.

Figure 111 Port Mapping Example



The following table describes the category and the details used in the figure:

Category	Details
SPICE	<code>.subckt inv a_0 a_1 b_0 b_1</code>
HDL	<pre>module inv(x, y); input [1:0] x; input [1:0] y;</pre>
Instantiated	<code>inv il_inv(i, j);</code>

Port Map	<pre>use_spice -cell inv \ -inst top.i1_inv \ port_map (y -> {a_0, b_1}, x -> {b_0, a_1});</pre>
----------	--

Following are the different ways in which port mapping can be specified:

- `use_spice -cell inv1 -inst top.x1 port_map (a => y, b => x);`
- `use_spice -cell inv2 -inst top.x2 port_map (* =>snps_by_name);`
- `use_spice -cell inv3 -inst top.x3 port_map (* =>snps_by_position);`
- `use_spice -cell inv4 -inst top.x4 port_map (* =>snps_open);`
- `use_spice -cell inv5 -inst top.x5`

For more information about different ways of mapping the port, see the *Port Mapping* section in the *Mixed-Signal Simulation User Guide*.

Limitations

This feature has the following limitations:

- In the spice domain, the contributor analysis is limited. For example, the ports of primitive devices are in/out port. Therefore, all the inst-ports are listed as contributors.
- For analog signals with continuous waveform, active driver analysis cannot be performed as the correct value changes cannot be identified. In this situation, the status of drivers must be the possible drivers.

Modifying the Analog Configuration During Runtime (Using ace reread)

Generally, it is required to re-compile, re-elaborate, and re-simulate after you modify or adjust parameters or settings of a design. This change can happen frequently and the process of simulating the design takes longer time. To reduce the simulation time, CustomSim supports the interactive ucli command "**ace reread**". This enables you to change the parameter or simulation settings during runtime with a SPICE design and helps reduce the simulation time.

You can run the **ace reread** command in the Verdi **Interactive Console** to make modifications to the analog configurations, such as simulation accuracy, temperature, or limited aspects of a netlist during runtime (that is, during the simulation of a design).

For more information about the using the ace reread command in CustomSim, see the *Mixed-Signal Simulation User Guide*.

Prerequisite

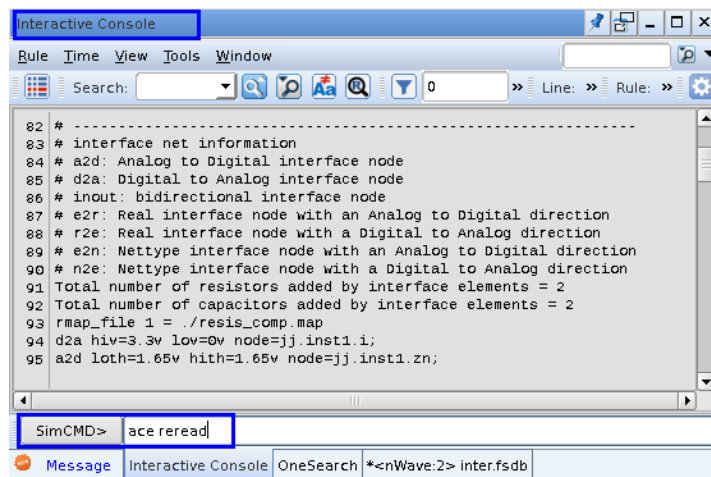
Requires CustomSim Version L-2016.06-SP2 or later.

Use Model

After compiling, elaborating, and simulating the design, load the design in the Verdi interactive mode. You can open *nWave* and dump signals using the drag and drop options and run the **ace reread** option as needed (refer to [Figure: Executing ace reread in the Interactive Console](#)).

For information on how to open FSDB and load the signals in *nWave*, see section [Debugging in Waveform View](#).

Figure 112 Executing ace reread in the Interactive Console



After executing the **ace reread** option in the **Interactive Console**, the hierarchy in FSDB file might change. The following situations might occur after the execution of **ace reread**:

- For additional signals dumped, the signals are reflected in FSDB after running **ace reread**.
- For signals that disappear in the design, the signals and their value before the time point still exists. However, no value is dumped after that time point when running **ace reread**.
- For the existing signals, changed value is later appended on the original signal.

Example

Consider the waveform shown in [Figure: New Signal v\(i\) Added and Signals v\(zn\) and v\(zn1\) are Removed and Added Back Later](#), which depicts the configuration change during runtime. To make this configuration change, in the **Interactive Console**, perform the following steps:

1. Run the simulation for time 1000 using the following command:

```
SimCMD> run 1000
```

2. Remove v(zn) and v(zn1) from the design and add a new net i that has a voltage value v(i).

3. Run the **ace reread** option:

```
SimCMD>ace reread
```

4. Again, run the simulation for time 1000 to progress the simulation for time 2000.

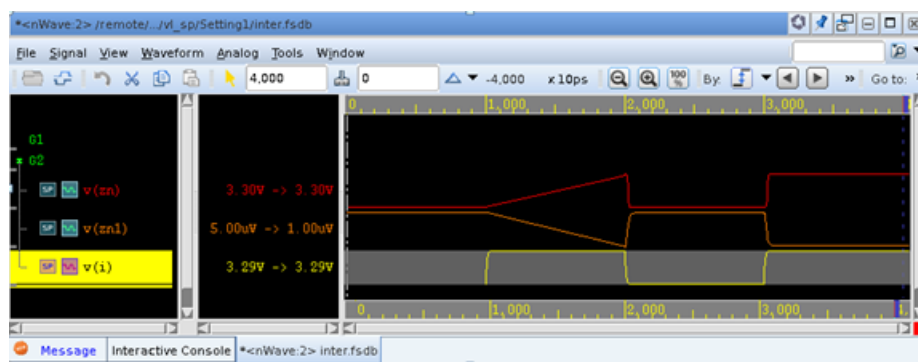
```
SimCMD> run 1000
```

5. Add v(zn) and v(zn1) back at time 2000. As the signals are removed from design at time 1,000, there are no value changes dumped for these signals after time 1000. Later, add the signals back as the signals that existed once are appended with the value change on them starting from 2,000.

6. Run the **ace reread** option:

```
SimCMD>ace reread
```

Figure 113 New Signal v(i) Added and Signals v(zn) and v(zn1) are Removed and Added Back Later



Limitations

This feature has the following limitations:

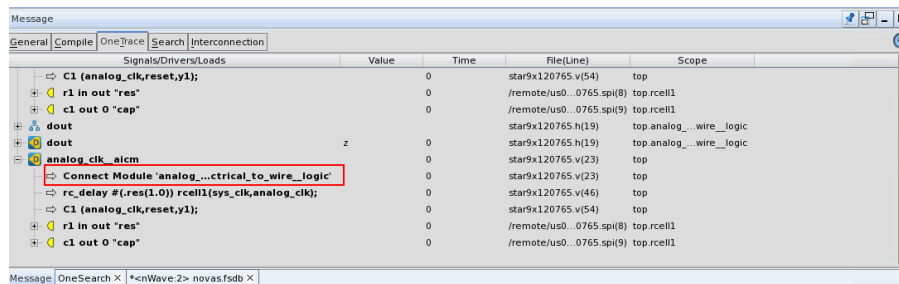
- Adding or removing a hierarchy with this feature is not supported as the same is not supported with CustomSim.
- Verdi does not reload the design to show the latest source code if you change the source code when you use **ace** **ace reread**. The changes in simulation are only reflected in the waveform.

Tracing Results of Connect Modules in the AMS Designs

In AMS designs, the connect modules are automatically inserted to connect the continuous and discrete disciplines (mixed nets) of the design hierarchy together.

In leading trace, the results are passed through the connect module instance and lists the result as shown in the following figure:

Figure 114 Traced Results for Connect Module



The behavior of tracing driver for the instrumented signal `top.analog_clk_aicm` is the same as the original signal `top.analog_clk`, and the connect module information is listed as shown in the figure.

Note:

The tracing feature for the signal under connect module is not supported.

IE_All_Only Option for MSV Common Dump

The `+IE_All_Only` option dumps both the analog and digital signal of all types of IEs, that is `d2a`, `a2d`, `n2e`, `e2n`, `r2e`, and `e2r`.

Use Model

```
ucli% dump -add <scope> [-fsdb_opt +IE_All_Only]
```

The scope for the above command works in accordance to the behavior that is allowed for the new IE placement/ reporting algorithm, that is if you use an alias to control IE options then the same mechanism would also be followed here or else it would be ignored.

Example

If the Interface Elements (IEs) reported to the user are:

```
d2a -> top.ia2.ib2.in  
n2e -> top.ia4.in
```

The UCLI usage and system task and behavior are as follows:

UCLI Usage & System task	Behavior
dump -add / -fsdb_opt +IE_All_Only	Dumps all IEs of the design. Both the analog view and the digital view will be in the same scope.
\$fsdbDumpvars("+IE_All_Only");	
dump -add top -fsdb_opt +IE_All_Only	All the IEs that lies below the top gets dumped.
\$fsdbDumpvars(top , "+IE_All_Only");	
dump -add top 2 -fsdb_opt +IE_ONLY	All the IEs that lies two depth below the top gets dumped.
\$fsdbDumpvars(top , 2, "+IE_All_Only");	

Limitations

- Does not support typenet.
- Extra view dumping
 - For digital extra view in analog scope: only supports atom type.
 - For analog extra view in digital scope: only supports atom/packed/unpacked type. For the case of nettype, it only supports atom and one dimension unpacked type.

Event-Based Real-Time View Swapping (RTVS)

The mixed-signal simulation supports multiple views for a cell. A cell may have both digital and analog views, and at compile time, you can select one view for the cell (cell-based partitioning) or a few instances of the cell (instance-based partitioning). The selected view determines the master of the respective instances when the design is elaborated, and this cannot be changed during the simulation which is the limitation of the existing mixed-signal solution.

The Real-Time View Swapping (RTVS) feature addresses this limitation and enables you to dynamically choose between the digital and the analog views for specified time windows at simulation time. By selecting the analog view for certain time windows, high simulation accuracy can be achieved. On the other hand, you get overall good simulation performance as the digital view is simulated for the remaining time periods.

Using event-based RTVS configurations, you can use a control signal in the design to choose various configurations at different time windows at runtime.

Licensing Requirements

To use an event-based RTVS configuration, you must have the Verdi Apex (`verdi -apex`) license.

Use Model of Event-Based RTVS Configurations

In an even-based RTVS, switching views is selected by a logic signal or an integer variable defined in the Verilog code. The signal or integer variable is identified by the following command:

```
rtvs_option -sel_sig <signal_name>;
```

Where `signal_name` is the hierarchical path to the signal or integer variable. Only one signal or integer variable can be used to control window switching.

The syntax of the `rtvs_config` command supports the `sel_val` option that determines the signal value or integer value that selects each RTVS configuration.

The syntax of the `rtvs_config` command is as follows:

```
rtvs_config -sel_val <value> [-icfile <icfile_name>] [-sim_opt  
<file_name>] -control <control_file_name> -label "<config_label>";
```

Arguments

```
-sel_val <value>
```


Defines the signal value or integer value associated with the mixed-signal configuration defined in the command. The signal can be from the Verilog netlist, or SPICE netlist that is accessed through XMR. The signal can only be of logic type with value 1b'0, 1b'1, 1b'x, or 1b'z. Any integer value equal or bigger than 0 is acceptable.

```
-icfile <icfile_name>
```

(Optional) You should use this option with care because it affects all windows created by the events. You can specify the initial values of the window using this option.

```
-sim_opt <file_name>
```

(Optional) You should use this option with care because it affects all windows created by the events. You can change analog simulation settings using this option.

```
-control <control_file_name>
```

This local control file defines the new configuration for specific blocks using `use_spice`, `use_verilog`, and related commands. This configuration also inherits the blocks that are defined in the default configuration by the main control file. In case of conflicts, commands in the local control file override the ones in the main control file.

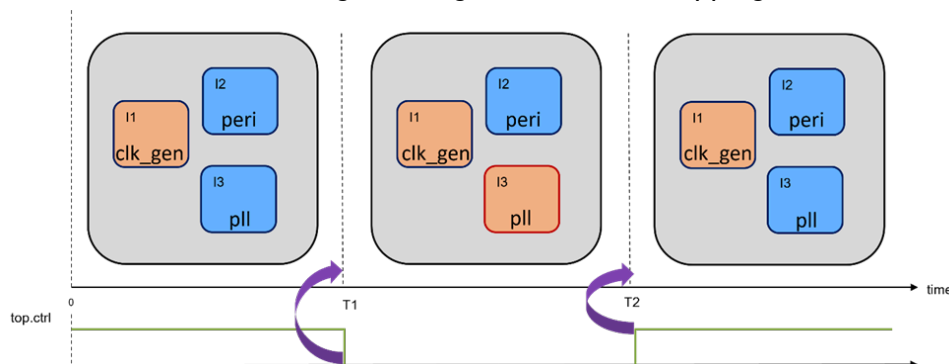
```
-label "<config_label>"
```

Specifies a configuration label for the new configuration. This label could be inquired and used in the Verilog testbench to handle the differences in circuit structures and cross module references between different configurations. The use of this option for each `rtvs_config` command is mandatory, and each label must be unique (not used in another `rtvs_config` command).

Example

The following figure shows an example of the event-based RTVS configuration, where the simulation switches between two configurations.

Figure 115 Event-Based, Single-Configuration View Swapping



In this example:

- A mixed-signal, default configuration is defined by `use_spice -cell clk_gen; in vcsAD.init.`
- A mixed-signal configuration is defined by `use_spice -cell clk_gen; in vcsAD.init` and `use_spice -cell pll; in config2.init.`

The `rtvs_option -sel_sig` option identifies the logic signal `top.ctrl` in the Verilog `top` module, which controls view swapping. When `top.ctrl` is assigned a value of 0, VCS switches to the RTVS view defined in the second configuration. When the signal is assigned a value of 1, VCS takes the RTVS view defined in the default configuration.

File: vcsAD.init

```
choose xa spice.sp -o out/xa;
use_spice -cell clk_gen;
rtvs_option -sel_sig top.ctrl;
rtvs_config -sel_val 0 -control config2.init -label "CFG_2";
```

File: config2.init

```
use_spice -cell pll;
```

User Interface Changes in Verdi

The event-based RTVS feature has the following changes in Verdi:

nTrace

- **Instance Tree:** Displays both digital RTVS instance and SPICE RTVS instance. The digital instance name is appended with a postfix “_dig” to avoid a name conflict.
- **Signal List:** Displays the dumped value at cursor time, even if the instance is in the inactive period.
- **Source Code:** Annotation value displays the dumped value at cursor time, even if the instance is in the inactive period.

Figure 116 SPICE RTVS Instance

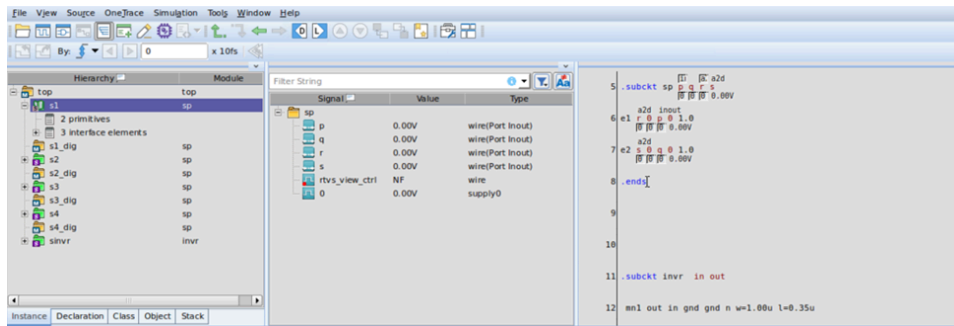
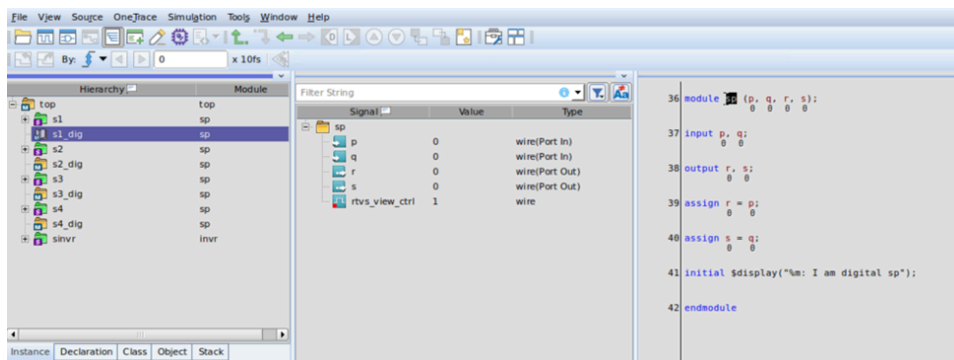


Figure 117 Digital RTVS Instance

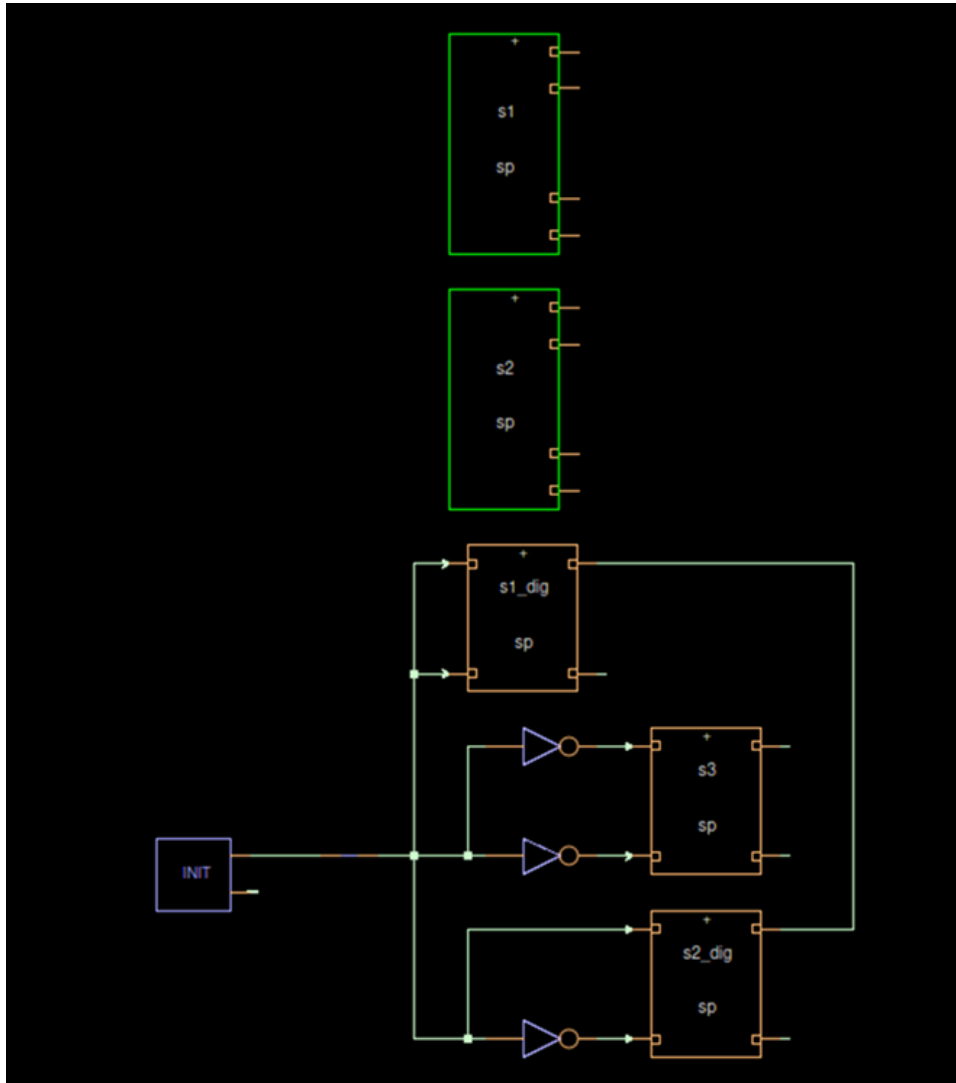


nSchema

RTVS Instances

- Displays both SPICE RTVS instance and Digital RTVS instance in different colors.

Figure 118 Different Colors for SPICE and Digital RTVS Instances



- If `rtvs_view_ctrl` is `Analog_is_ON`, displays the shadow border for the digital RTVS instance.

If `rtvs_view_ctrl` is `Analog_is_OFF`, displays the shadow border for the SPICE RTVS instance.

Figure 119 `s1_dig.rtv_view_ctrl` is `Analog_is_ON` ; `s2_dig.rtv_view_ctrl` is `Analog_is_ON`

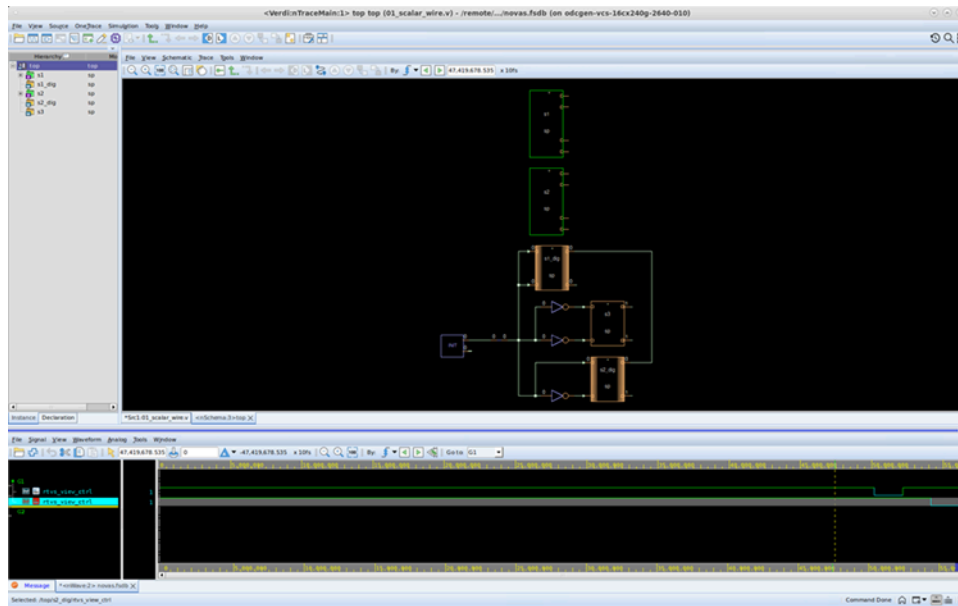


Figure 120 *s1_dig.rtv_view_ctrl is Analog_is_OFF ; s2_dig.rtv_view_ctrl is Analog_is_ON*

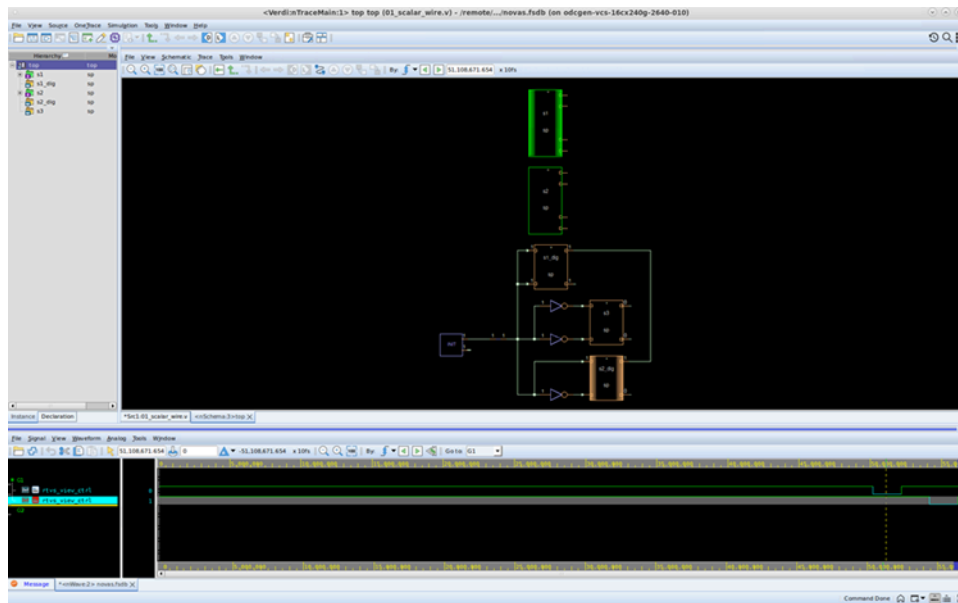
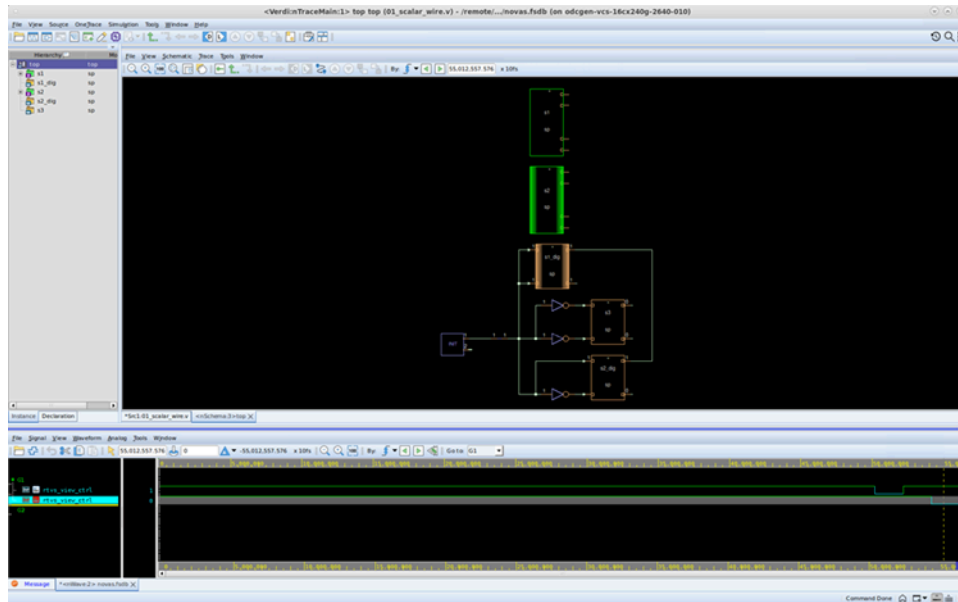


Figure 121 *s1_dig.rtv_view_ctrl is Analog_is_ON ; s2_dig.rtv_view_ctrl is Analog_is_OFF*



nWave

- Displays the mask on the inactive part of RTVS signals.
 - **Signal From SPICE RTVS Instance:** Hides the waveform in an inactive window (value of `rtvs_view_ctrl` is 1).
 - **Signal From Digital RTVS Instance:** Displays mask (dense dots) in an inactive window (value of `rtvs_view_ctrl` is 0).
- Hides the waveform or puts the mask only on original signals, that is, the signal dumped in FSDB. The mask is not applied on an overlap node.
- Displays the mask by default.

Figure 122 Displays the Mask on the Inactive Part of RTVS Signals



Limitations

The RTVS feature has the following limitation:

- An RTVS instance under an RTVS instance is not supported. It means that an instance cannot switch its view independently from its parent instance, which can also switch view.

4

Interactive Simulation Debugging of Verilog-A Designs

This chapter describes how to debug Verilog-A designs in the Interactive Debug mode. Use the following capabilities for interactive simulation debug of Verilog-A mixed-signal designs:

- [Compiling Mixed-Signal Design](#)
- [Performing Interactive Simulation Debug of Verilog-A Mixed-Signal Design](#)
- [Debugging Verilog-A Design in the Source View](#)
- [Debugging Verilog-A Designs in the Waveform View](#)
- [Debugging Verilog-A Designs in the Schematic View](#)
- [Tracing Verilog-A Designs in the Temporal Flow View](#)

Compiling Mixed-Signal Design

Verdi only provides the **Import Design from Library** feature to load your mixed-signal design into the Verdi platform. Therefore, you must compile your design using the Verdi KDB with both digital and analog hierarchy/signals. The *spicom* utility allows you to compile the analog modules.

The following section describes how to compile your Verilog-A design with the *spicom* utility and how to load KDB into Verdi:

- [Compiling Verilog-A Blocks](#)

Compiling Verilog-A Blocks

To compile the Verilog-A files with VCS, instantiate the Verilog-A module in a SPICE file and use the SPICE file for compiling it with VCS and CustomSim simulator. For compiling the SPICE designs, see [Compiling and Importing a SPICE Design](#) and [Compiling With the SPICE Unified Front-End Flow](#).

In a mixed-signal simulation, Verilog-A can be used as the analog view of a cell. Use the `.hdl` command in the SPICE netlist to read Verilog-A files. In this flow, passing all Verilog files, including Verilog-A to VCS at compile time is still possible although not recommended. For more information about Verilog-A model instantiation, see the *Mixed-Signal Simulation User Guide*.

Limitation

With Verilog-A model instantiation, as the KDB design hierarchy is generated at compile time, Verdi does not support analog simulator's commands, such as `set_va_view` and `.param HSIMUSEVA=<module name>` to switch between SPICE and Verilog-A view during simulation time.

Performing Interactive Simulation Debug of Verilog-A Mixed-Signal Design

After you have compiled your Verilog-A mixed-signal design, you can interactively debug the design in Verdi. The following features are supported with the Verilog-A Interactive Debug mode with a Verilog-A design:

- Provides various methods to dump the following Verilog-A signals:
 - Variables
 - Voltage and current of nodes (you can dump current if you have used current in your design).
 - Voltage and current of unnamed/named branches (you can dump current if you have used current in your design).

For more information about Verilog-A signal dumping, see [Dumping Verilog-A Signals](#).

- Shows the following values in the annotation of Verilog-A signals in the interactive debug mode:
 - Variable's value
 - Voltage of node and unnamed/named branch
- Shows Verilog-A signals in the interactive *Watch* tab.
- Saves/restores the simulation session/state.
- Waveform to save/restore a signal .rc file with Verilog-A signals in the form of `V(...)` being dumped and restored.

The following sections describe the supported interactive simulation debug features:

- [Getting Started](#)
- [Setting the Simulator Type in the Preference Page](#)
- [Using the Interactive Watch Tab](#)
- [Annotating Values](#)
- [Dumping Verilog-A Signals](#)
- [Saving or Restoring Simulation Session or State](#)

Getting Started

Verdi supports the Verilog-A interactive debug mode with this enhancement. The user flow is as follows:

1. Prepare the AMS design with Verilog-A.
2. Set the `XA_HOME`, `VERDI_HOME`, and `VCS_HOME` environment variables of Compile the Verilog-A AMS design with VCS and get the simulation executable that has the debug capability. For more information, see the document of VCS/CustomSim/PLI.
3. Use Verdi to load the Verilog-A design with the `cosim` option, which you can specify using the following methods:
 - Specify `-msv` when importing the design from the command line.
 - In *nTrace*, select **File -> Import Design From library -> Options** toggle on the **Cosim Debug** command and load the Verilog-A design.
4. Select **Simulation -> Invoke Simulator**.

Verdi creates and opens a single FSDB to include the digital and analog signals dumped by the FSDB dumper.

You can run AMS interactive debug with the Verilog-A design. However, the simulation is controlled by VCS and there are some limitations in VCS/CustomSim when running Verilog-A AMS interactive debug:

- Does not support step or next into analog design.
- Breakpoints cannot be added into analog design.
- Rewind is blocked in the AMS interactive debug mode.

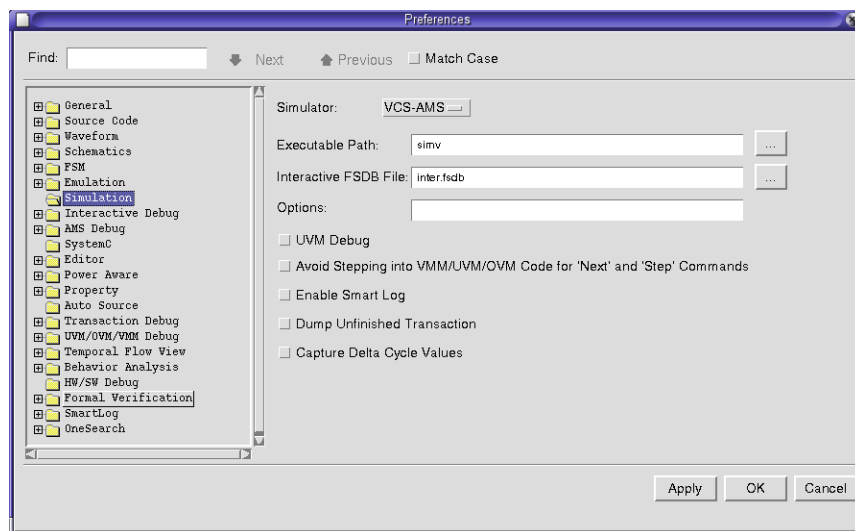
These limitations disable some menu commands and right-click menu options.

For more information on the Simulation and Interactive Debug commands, see the *Verdi Command Reference Guide*.

Setting the Simulator Type in the Preference Page

You can set the **VCS-AMS** option from **Tools -> Preference -> Simulation -> Simulator** menu. If you specify `-msv` in the command line, the simulator type in preference is changed to **VCS-AMS** as illustrated in the following figure:

Figure 123 Simulator Type in Preference Form



When an error occurs, the following methods are available to resolve errors:

- Verdi supports `-simType` to indicate the simulator type when invoking Verdi. If you specify both `-msv` and `-simType`, then Verdi displays a warning message and honors `-msv`.
- Verdi provides a warning message when you invoke the simulation by different simulator types when running Verilog-A AMS interactive debug. For example:
 - You can use the simulator type as **VCS** and run interactive mode with Verilog-A AMS `simv`.
 - You can use the simulator type as **VCS-AMS** and run interactive mode with digital only `simv`.

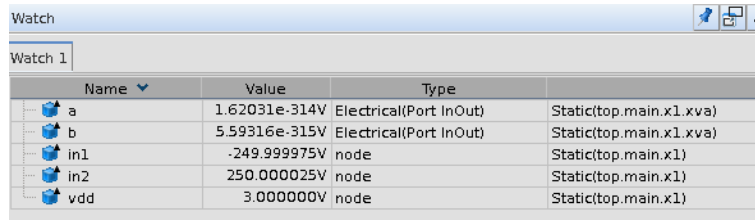
Using the Interactive Watch Tab

The interactive *Watch* tab supports to show Verilog-A signals that include variable, electrical nodes (electrical, voltage, and current), and branch at the current simulation

time. The *Interactive Watch* tab displays the variable's values and the voltage of electrical nodes and branches at the current simulation time.

You can add Verilog-A signals to the *Watch* tab using the drag and drop method or the **Add to Watch** right-click menu option. Then, the selected signal is added to the *Watch* tab, as shown in the following figure:

Figure 124 Signals Added to the Interactive Watch Tab



Name	Value	Type	Scope
a	1.62031e-314V	Electrical(Port InOut)	Static(top.main.x1.xva)
b	5.59316e-315V	Electrical(Port InOut)	Static(top.main.x1.xva)
in1	-249.999975V	node	Static(top.main.x1)
in2	250.000025V	node	Static(top.main.x1)
vdd	3.000000V	node	Static(top.main.x1)

The *Watch* tab provides the following information about analog variables:

- **Name:** Variable, node, or named branch (this is displayed as specified in the design). Unnamed branch, displayed in the (a,b) format.
- **Value:** Variable (Shown as variable's value) Node, unnamed/named branch, that is, shows the voltage value at the current simulation time. For example, 3.30V.
- **Type:** Variable, node, unnamed branch, or named branch.
- **Scope:** Full hierarchical scope.
- **Icon:** Synchronize *nTrace* icon for Verilog-A signals.

You can summarize the effectiveness of each right-click menu option in the interactive *Watch* tab in the analog design.

The following features are disabled with Verilog-A signals:

- **Set Breakpoint**
- **Set Radix**
- **Add Object to Watch Tab**
- **Show Declaration, Show in Class Browser, View Object Reference**
- **Dump Object to FSDB file, Dump, Add Object to Waveform**

Limitations

The following are the limitations with the *Watch* tab:

- Mathematic types such as 'real', 'integer', 'parameter' and 'genvar' cannot be added to the *Watch* tab (it is due to the limitation of CustomSim as mathematic types cannot get value from the simulator).
- For internal nodes in a leaf Verilog-A scope, the value is not displayed (this is due to the limitation of CustomSim).

Annotating Values

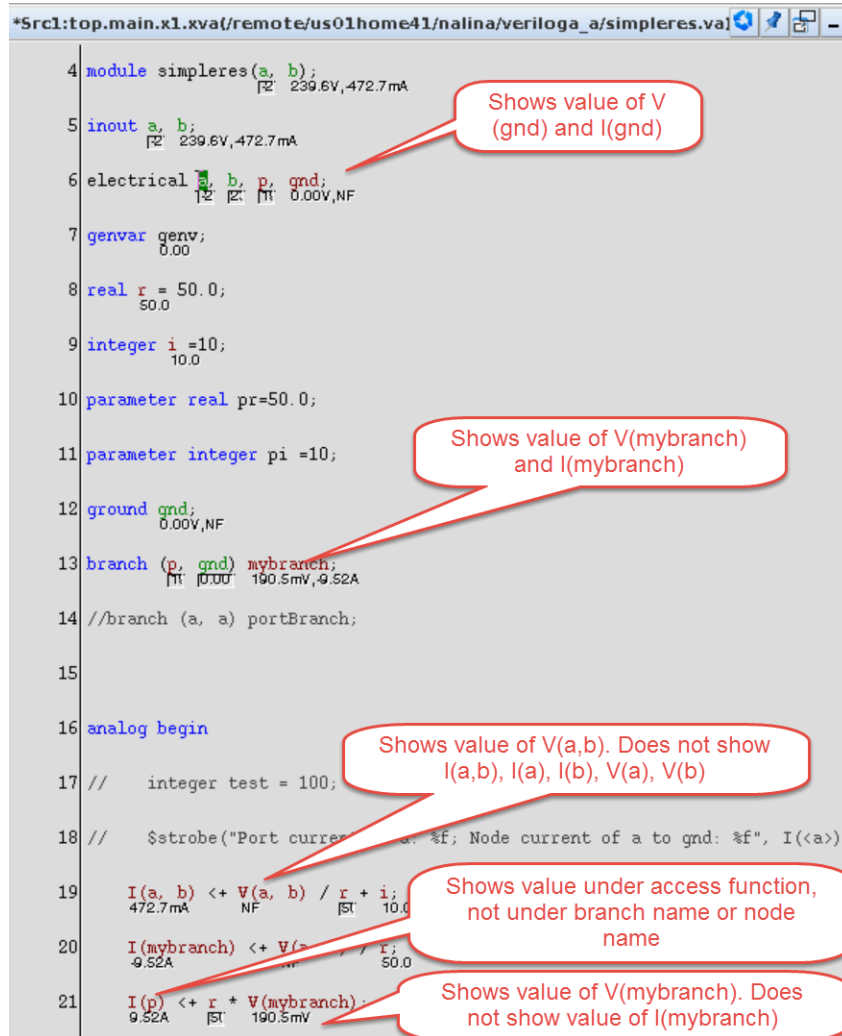
You can turn on the source view annotation using the **Source -> Active Annotation** toggle menu (the hotkey is 'x').

If the cursor time is not equal to simulation time, you can get the annotation value from FSDB. For more information, see section Verilog-A annotation in *nTrace* in the post-simulation mode.

If the cursor time equals simulation time, you can get the annotation value from IPI/simulator (this is same as annotation of Verilog signals). Due to the known limitation of CustomSim, only the voltage of electrical node and branch is displayed. For example, the voltage value is shown as 3.30V.

The variable, node, and branch values are displayed as shown in the following figure:

Figure 125 Active Annotation



Limitations

The following are the limitations with annotating Verilog-A signals:

- Changing the analog format and precision in annotation is not supported.
- No annotation for $I(a, b)$ or $I(mybranch)$ as it requests current value and it is not currently supported in simulator.

- Mathematic types such as 'real', 'integer', 'parameter' and 'genvar' cannot be added to the *Watch* tab (it is due to limitation of CustomSim, mathematic types cannot get value from simulator).
- For internal nodes in leaf Verilog-A scope, the values are not displayed (it is due to limitation of CustomSim).

Dumping Verilog-A Signals

Verdi allows you to dump variable, node, and branch values of Verilog-A signals. Dumping a variable is similar to dumping a digital signal. The following sections discuss how to dump nodes and branches.

Dumping Nodes

Similar to SPICE, the current of a node is annotated with different value according to their position in the code. For example, if you drag and drop node *a* from the position of module definition, then its port current *I(<a>)* is dumped.

However, naming a node might not be the same as naming a design. For example, simulator does not dump signal, such as *I(<a>)* in FSDB, it dumps *isub(a)* or *in(va_inst)*. The following table summarizes the signals that Verdi triggers to dump when you right-click or drag and drop to dump *a*.

Syntax	Signals Dumped	Dumping Command
module test (a, b)	top.va.v(a), top.in(va), top.va.in(va) top.va.isub(a)	fsdbDumpvars top.va.a +msv+vaV fsdbDumpvars top.va +msv+val fsdbDumpvars top.va.a +msv+val
electrical node a	top.va.v(a) top.va.i(a)	fsdbDumpvars top.va.a +msv+vaV fsdbDumpvars top.va.a +msv+val
V(a)	top.va.v(a)	fsdbDumpvars top.va.a +msv+vaV
I(a)	top.va.i(a)	fsdbDumpvars top.va.a +msv+val
I(<a>)	top.in(va), top.va.in(va) top.va.isub(a)	fsdbDumpvars top.va +msv+val fsdbDumpvars top.va.a +msv+val
xsp sp (a, b);	top.va.v(a), top.va.sp.isub(a)	fsdbDumpvars top.va.a +msv+vaV fsdbDumpvars top.va.sp.a +msv+isub
xva va2 (a, b);	top.va.v(a) top.va.va2.in(va2) top.va.va2.isub(a)	fsdbDumpvars top.va.a +msv+vaV fsdbDumpvars top.va.va2 +msv+val fsdbDumpvars top.va.va2.a +msv+val

For more information about *fsdbDumpvars*, see the *Linking Novas Files with Simulators and Enabling FSDB Dumping Manual*.

Dumping Branches

You can dump two branch types in Verilog-A: unnamed branch and named branch. The following table summarizes the signals dumped when you select "unnamed/named branch".

Syntax	Signals to be Dumped
V(a,b)	v(a,b)
I(a,b)	i(a,b)
V(branch)	v(branch)
I(branch)	i(branch)
branch	v(branch), i(branch)

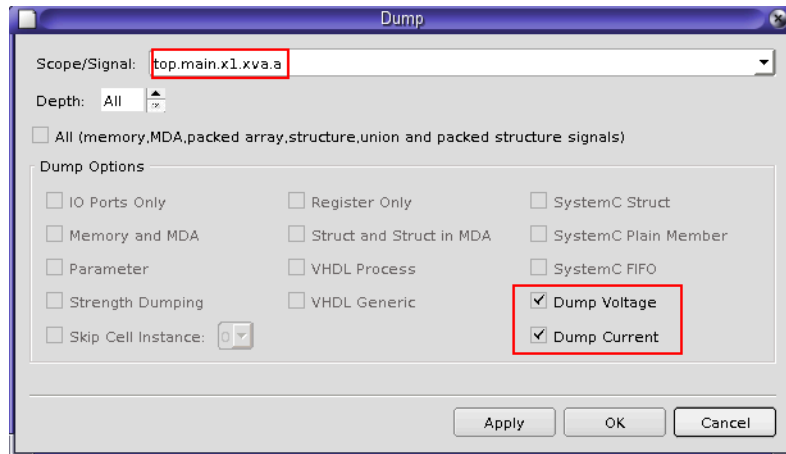
Methods for Dumping Signals

Similar to interactive debug mode with digital design, you can dump Verilog-A signals in the interactive debug mode using one of the following methods:

- In *nTrace*, right-click the signal/instance and select the **Drag** option. In *nWave*, right-click the signal pane and select the **Drag** option.
- In *nTrace*, right-click the node/branch and select the **Dump to FSDB File** option.
- You can use the *Dump* dialog box, which is used to dump other HDL signals to dump Verilog-A signal/scope as well. Select the **Dump Voltage** and **Dump Current** checkboxes to dump the signals. The selected signal/scope in source view is automatically

displayed in the **Scope/Signal** field (see [Figure: Dumping Verilog-A Signals \(Dump Dialog Box\)](#)).

Figure 126 Dumping Verilog-A Signals (Dump Dialog Box)



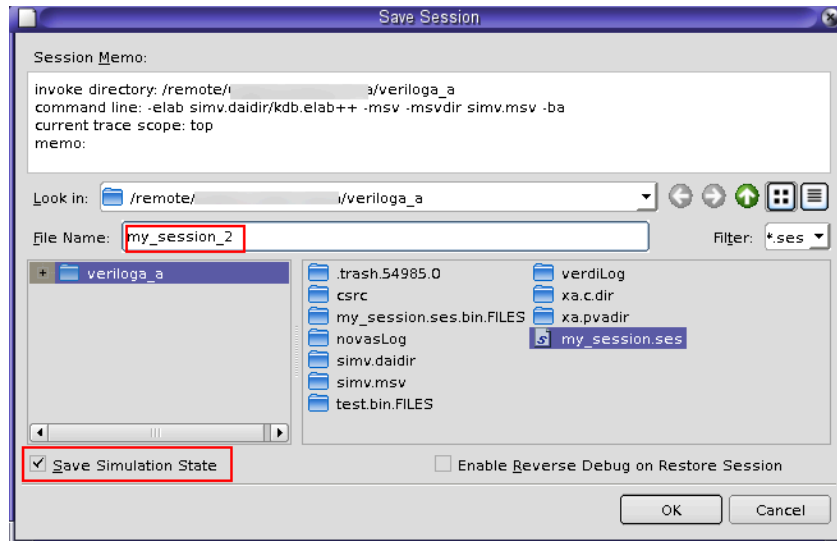
Saving or Restoring Simulation Session or State

Verdi already supports to save/restore the simulation session/state in the interactive mode with the digital design. This feature provides the capability to save the currently debug process or restore the point to continue the unfinished debug process. Verdi also supports this feature in the Verilog-A interactive debug mode. Both the simulation states in digital and analog are saved/restored.

The usage is as same as that of the digital design. You can use this feature with the following commands:

- **nTrace -> Simulation -> Save State**
- **nTrace -> File -> Save Session.** Then, select the **Save Simulation State** option, as shown in the following figure:

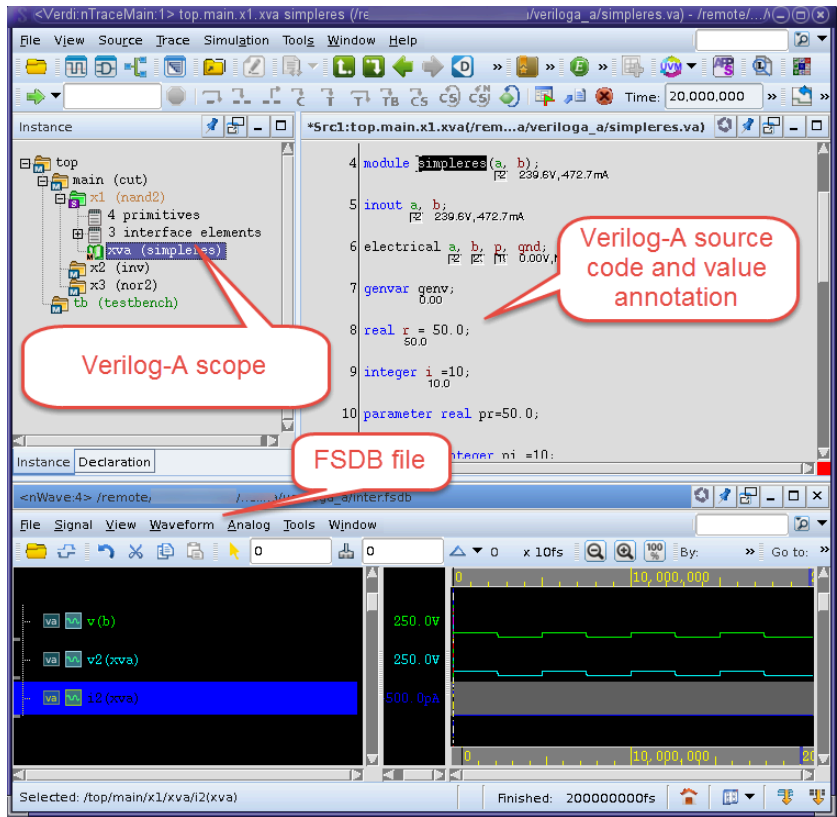
Figure 127 Save/Restore Session/State



Debugging Verilog-A Design in the Source View

After loading a mixed-signal design into Verdi, you can see the design hierarchy with both analog and digital scopes in the *Instance* frame. You can also check the Verilog-A source code in the *source code* window (see [Figure: Showing Verilog-A Information](#)).

Figure 128 Showing Verilog-A Information



Note:

If your design includes SPICE source code, the SPICE nodes are also shown in the *Instance* frame.

The following table shows the nodes shown in the *Instance* frame:

Node Figure	Node Meaning
	A Verilog-A mixed-signal module
	A SPICE mixed-signal sub-circuit

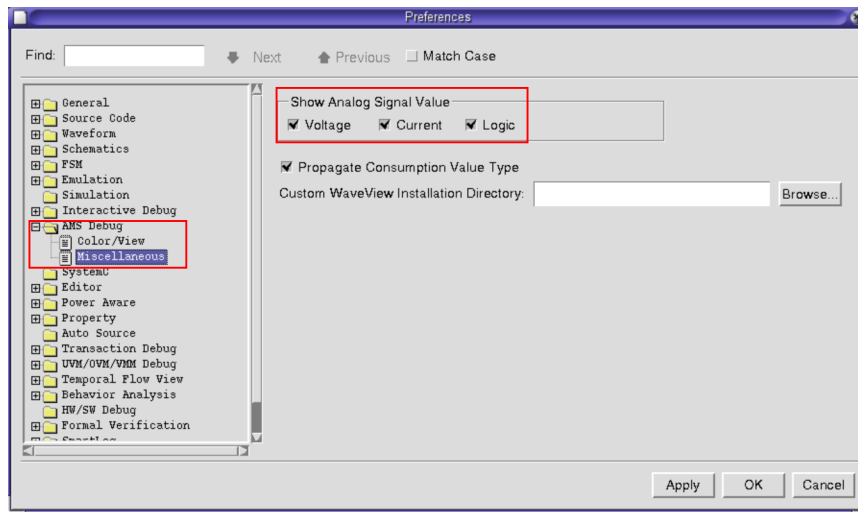
Double-click a node in the *Instance* frame to jump to the corresponding definition source code in the source code frame. The right-click menu commands are available for Verilog-A signals.

Setting Preferences

The type of the attached value is the voltage value by default. You can use the following steps to add or remove other types for the attached value:

1. Select the **Tools -> Preferences** menu to invoke the *Preferences* form.
2. Select the **AMS Debug** page and enable the types. You can select multiple signal types.

Figure 129 Selecting Verilog-A Signal Value



3. Click the **OK** button to apply the setting.

Note:

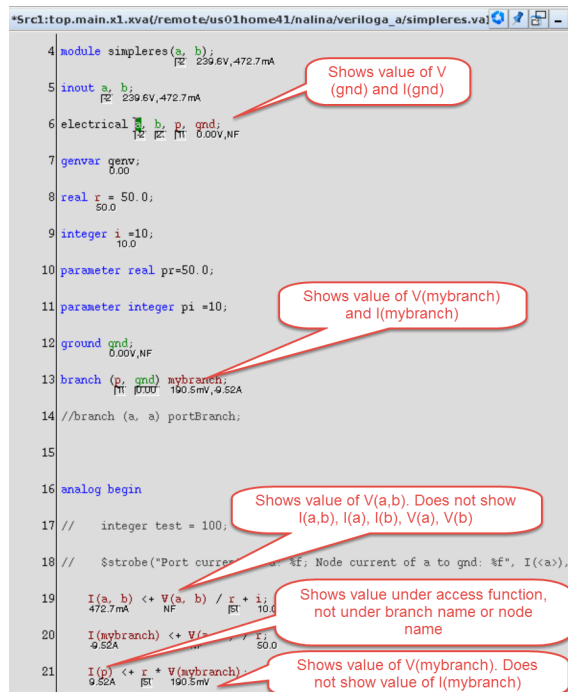
Only V(signal) and I(signal) values are supported for the annotation of the voltage and current types.

Active Annotation

After the virtual file containing digital and analog data files is loaded into the Verdi platform, you can select the **Source -> Active Annotation** menu to see the node, branch, variable, V(node), V(branch), I(node), and I(branch) values annotated in the source code (see [Figure: Active Annotation in Source Code](#)). Verdi shows V(node) and V(branch) when the **Voltage** option is toggled. Similarly, it shows I(node) and I(branch) when the **Current** option is toggled.

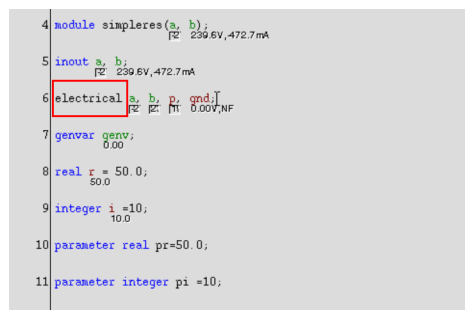
Chapter 4: Interactive Simulation Debugging of Verilog-A Designs Debugging Verilog-A Design in the Source View

Figure 130 Active Annotation in Source Code

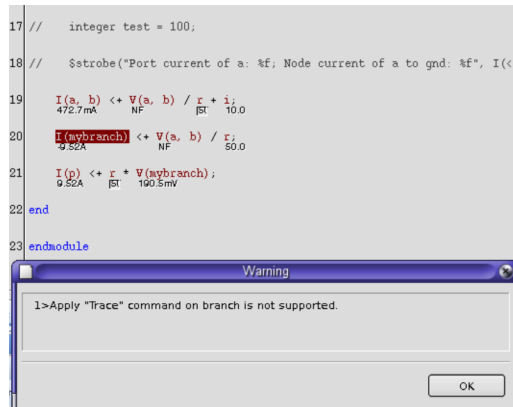


The disciplines, such as electrical are not color-coded, as shown in the following figure:

Figure 131 Disciplines - electrical



The access function and its node/branch are shown as one word. Note that you cannot select `mybranch` inside `I(mybranch)`, as shown in the following figure:



The color-coding of `I (node)`, `I (branch)`, `V (node)`, `V (branch)` are same as that of the signal.

Dragging and Dropping Verilog-A Signals

You can drag the node, variable, branch, `V(node)`, `I(node)`, `V(branch)`, `I(branch)` to the other Verdi panes.

You can drop the node, variable, branch, `V(node)`, `I(node)`, `V(named branch)`, `I(named branch)` to the source window. The the source window shows the definition of node, variable and named branch as default. However, you can change the drop action from the **Preferences** setting.

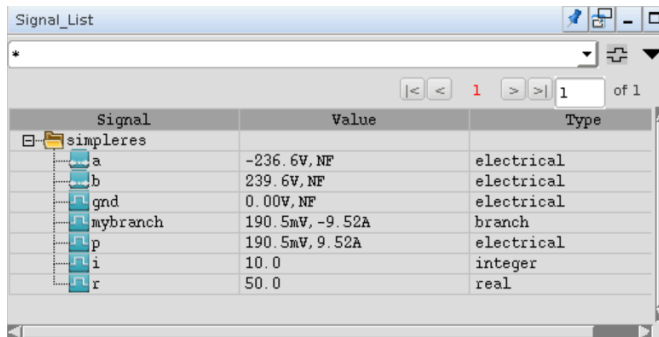
Note:

Verdi does not support drag and drop of unnamed branch.

Viewing the Signal List

Use the **View -> Signal List** menu to see the *Signal_List* frame (see [Figure: Verilog-A Signals in Signal List](#)).

Figure 132 Verilog-A Signals in Signal List



Signal	Value	Type
a	-236.6V, NF	electrical
b	239.6V, NF	electrical
gnd	0.00V, NF	electrical
mybranch	190.5mV, -9.52A	branch
p	190.5mV, 9.52A	electrical
i	10.0	integer
r	50.0	real

Verdi also allows you to drag-and-drop signals from the *Signal_List* pane to the *nWave* window.

Limitations

The following are the limitations:

- Tracing a signal is not supported inside an analog block.
- Verdi does not support swapping of Verilog-A view with AMS setup.

Debugging Verilog-A Designs in the Waveform View

When you load a file containing digital and analog data files into Verdi, you can add both digital signals and analog signals into the *nWave* window that opens the file. You can also use the drag-and-drop and synchronization features to add these signals.

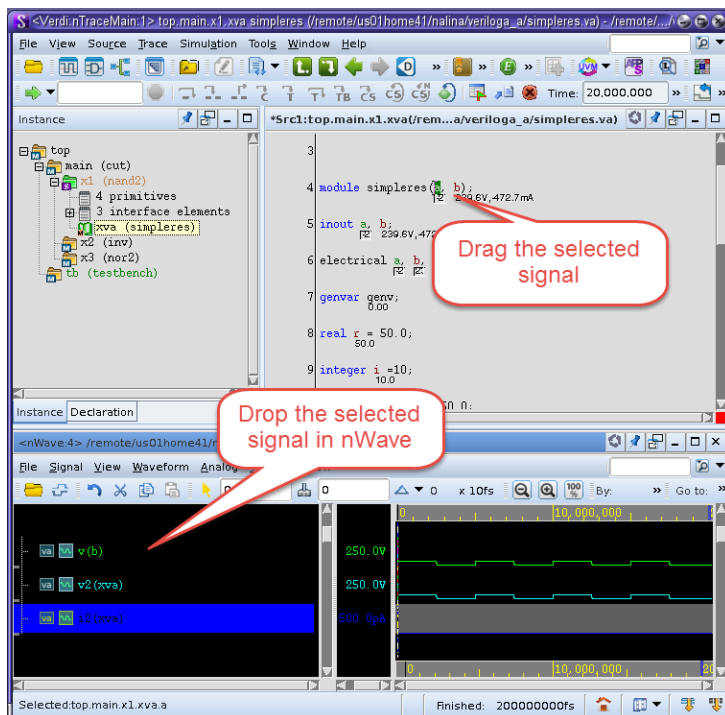
Drag and Drop to nWave

You can drag Verilog-A signal (or multiple signals) in the source code window and drop it to the *nWave* window to add the selected signal, as shown in the following figure. Verdi allows you to drag and drop variable, node, and branch.

Chapter 4: Interactive Simulation Debugging of Verilog-A Designs

Debugging Verilog-A Designs in the Waveform View

Figure 133 Add Signals by Dragging and Dropping



Note:

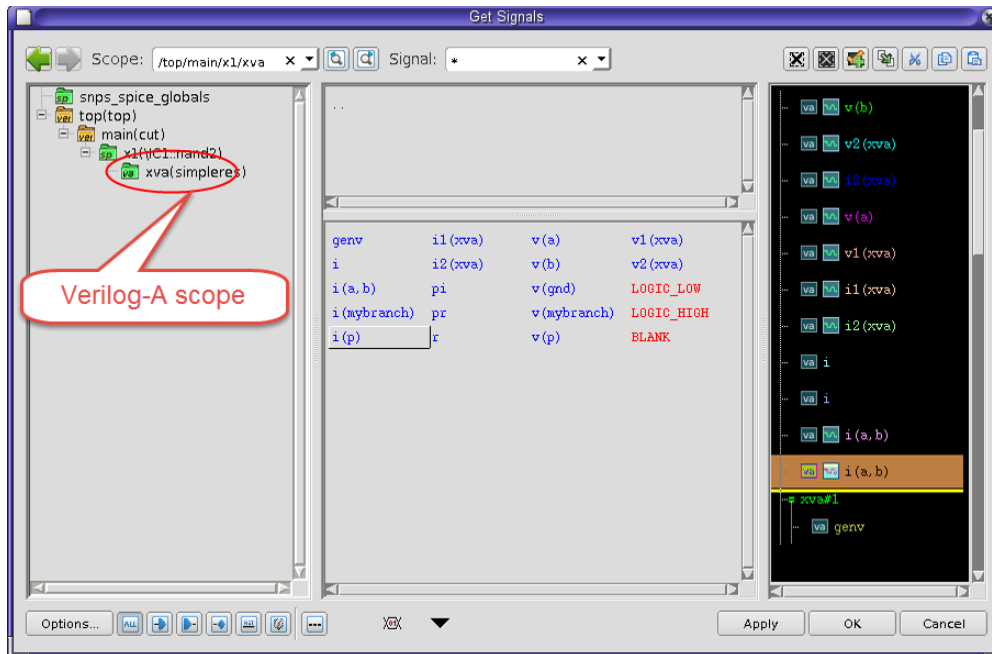
If the selected signal has not been dumped into the FSDB file, the **List of Unrecognized Signal(s)** dialog shows and lists the selected signal as an unrecognized signal.

When you drag and drop a scope to *nWave*, it adds port signals to waveform. To add all signals within the scope, set the preference using **Tools -> Preferences -> Waveform -> View Options -> Waveform Pane Miscellaneous**.

In the *nWave* window, you can also choose the **Signals -> Get Signals** command to open the *Get Signals* form. The *Get Signals* form enables you to select the dumped signals.

The Verilog-A signal is shown as  in the *Get Signals* form. In the *Signal* pane, Verilog-A signal is shown as  (see [Figure: Get Signals Form](#)).

Figure 134 Get Signals Form



Limitations

The following are the limitations with Verilog-A support in the waveform view:

Node:

- Adding node current of a primitive device in Verilog-A instance is not supported. For example, consider the following statement in Verilog-A:

```
resistor #(.r(10k)) R2 (b, gnd);
```

Verdi does not show the node current annotation passing through `b` or `gnd` and it is unable to add node current when you drag-and-drop `b` or `gnd` to waveform.

Branch:

- Dragging and dropping of unnamed branch from *nWave* to design-based AP is not supported.

Drag and Drop from nWave

You can drag Verilog-A variable, node, or branch in the *nWave* window and drop it to *nTrace* or *nSchema* to add the selected signal.

Limitation

Branch:

Dragging and dropping unnamed branch from *nWave* to design-based AP is not supported.

Dumping

If you drag and drop or right-click to add Verilog-A signals to *nWave*, *nWave* triggers the corresponding signals to be dumped. However, dumper does not support to dump specific port current if a Verilog-A instance is a leaf instance. That is, all nodes, branches are dumped if you dump port current of a leaf Verilog-A instance.

Limitations

The following are the limitations with the drag and drop feature:

- Does not dump *single* port current if a Verilog-A instance is a leaf instance due to simulator and dumper.
- Dumping voltage of an unnamed branch is not supported as simulator does not support it. As CustomSim cannot dump voltage for unnamed branch, it is shown as an unrecognized signal if you drag and drop the signal to *nWave*.

Batch Dumping

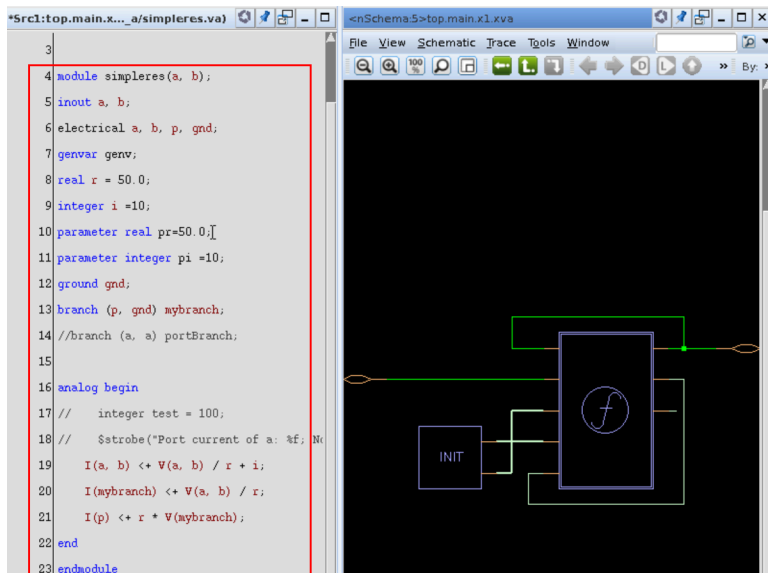
The Save/Restore signal *rc* file enables batch dumping for Verilog-A signals if restore signals are not existing in FSDB.

Debugging Verilog-A Designs in the Schematic View

You can load and debug a Verilog-A scope in *nSchema* similar to *nWave*.

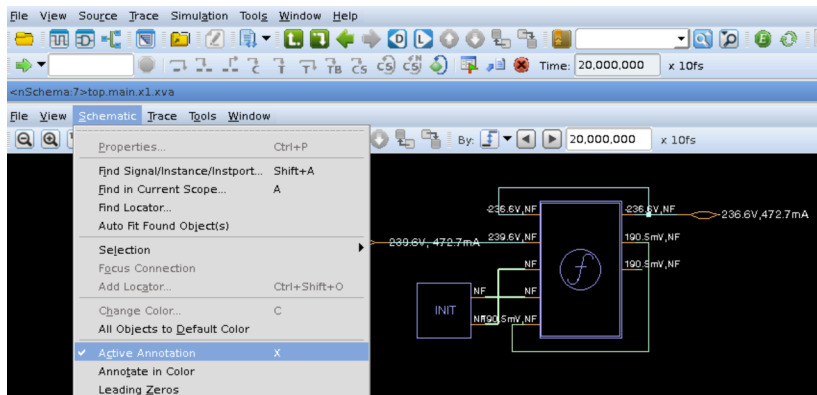
After the file containing digital and analog FSDB files is loaded into the Verdi platform, you can also see the analog values in the *nSchema* window. To debug in the schematic view, click  to invoke the *nSchema* window for the selected scope.

Figure 135 Verilog-A Scope



In the *nSchema* window, select the **Schematic -> Active Annotation** command to annotate the current value, as shown in the following figure:

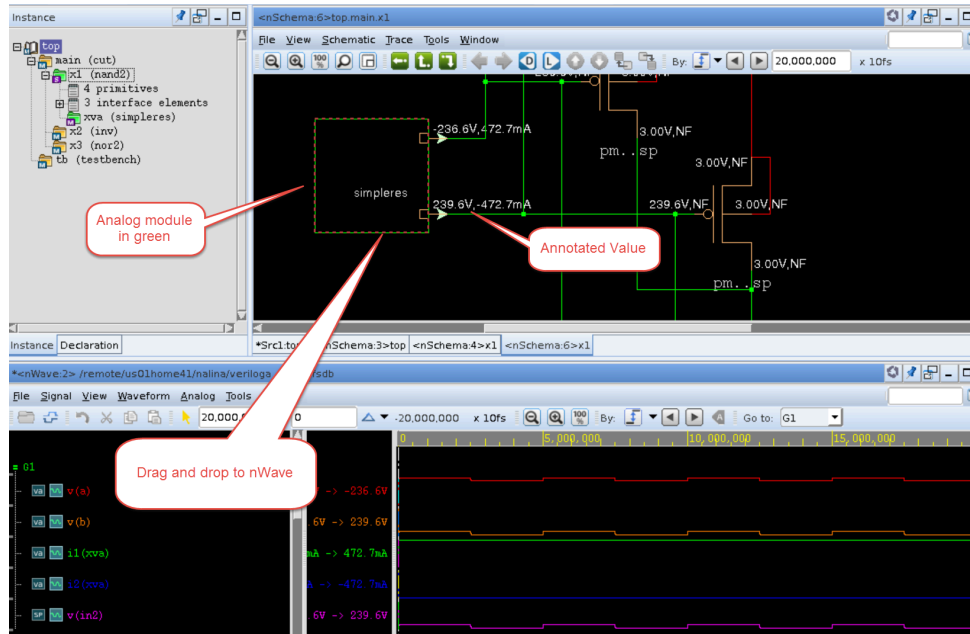
Figure 136 Annotating Values



The drag-and-drop features are available for Verilog-A signals. For example, you can drag and drop a Verilog-A signal between the *nSchema* and *nWave* windows to highlight the corresponding Verilog-A signals (see [Figure: Drag-and-Drop to nWave](#)). To add the voltage and current into *nWave*, drag and drop the signal under analog module to *nWave*.

The settings in the *Preferences* form (see [Active Annotation](#)) are also applied in the *nSchema* window:

Figure 137 Drag-and-Drop to nWave



Note:

nSchema does not have a model branch. Dragging and dropping a branch to *nSchema* is not supported and a warning message is issued.

Color-Coding Signals in nSchema

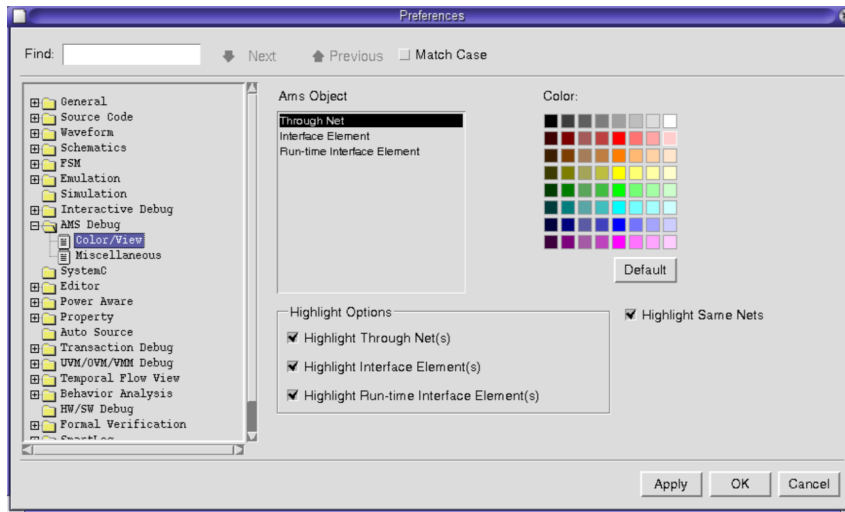
nSchema color-codes the following signals and blocks:

- Analog or digital blocks
- Analog, digital, or mixed signal signals

The analog blocks are displayed in green. When you open the *nSchema* window, all the analog signals are displayed in the default color and you can change the default color of the signals as well.

You can also select non-default colors for analog, digital, and mixed-signals from the menu, as shown in the following figure. However, there might be performance issues if non-default colors are selected.

Figure 138 Choosing Custom Colors for Signal Representation



If you have set the color selection, then it is reflected in all *nSchema* windows.

Tracing Verilog-A Designs in the Temporal Flow View

To debug a design, you must understand the structure and the behavior of the design. Visualizing the structure and behavior of the design would be very useful and convenient to become familiar with the design and then debug it.

Temporal Flow View (TFV) provides flow views that are unique temporal views and analysis tools that allow the visualization and analysis of the design's behavior through time. TFV identifies and displays causal control and data paths - the registers and signals that caused the erroneous value to occur - through multi-level combinational logic within one or more register-to-register transfer stages.

TFV provides the following capabilities for the analysis of analog and digital signals and are described in the following sections:

- Traces digital signals and analog signals on the tracing path using **Trace This Value** (cycle based) / **Trace Triggering Path** (transition based).
- Displays Interface Elements (IEs) and Through Nets (TNs).
- Uses function blocks with specific labels to represent analog blocks on TFV. TFV does not trace into the analog blocks; TFV only traces through analog blocks and uses a specific label as analog blocks (the green symbol is an analog block symbol).

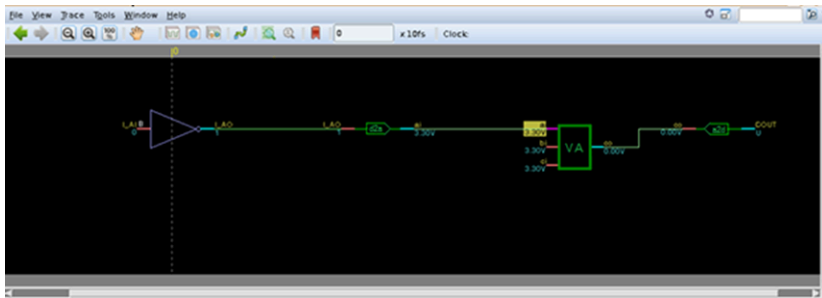
- Synchronize with *nWave*, *nTrace*, and *nSchema*, thereby showing the waveform, source code, and circuit detail of digital and analog signals.
- Allows you to drag and drop signals to *nWave*, *nTrace*, and *nSchema*, showing the waveform, source code, and circuit, respectively.

For more information about creating TFW, see Chapter *Temporal Flow View Tutorial* in the *Verdi User Guide and Tutorial* and Chapter *Flow Views* in the *Verdi and Siloti Command Reference Manual*.

Tracing Verilog-A Signals

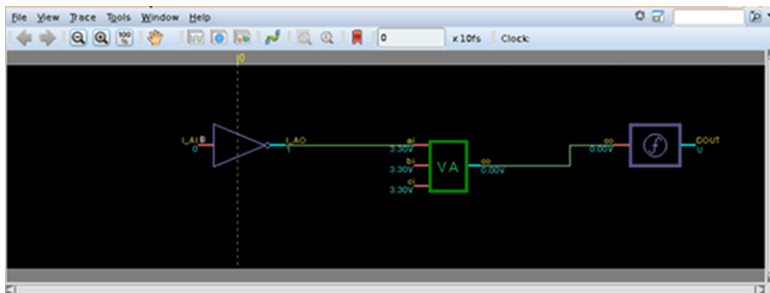
The tracing rules of Verilog-A signals are the same as digital tracing rules. Both cycle-based and transition-based tracing are available for tracing analog blocks. The following figure shows the process of tracing a digital signal, while a Verilog-A signal exists in the tracing path.

Figure 139 Tracing a digital signal and reaching a Verilog-A signal with IE reports



The following figure shows TFW tracing without IE ports.

Figure 140 Tracing a digital signal and reaching a Verilog-A signal without IE reports



The snap time of Verilog-A blocks are the same as their digital outputs. The detailed Verilog-A circuits are not displayed in TFW. Alternatively, you can trace to analog circuits using the schematic view in *nSchema*. You can later use the synchronization button in the TFW toolbar (see [Synchronization with nWave, nTrace, and nSchema](#)).

Note:

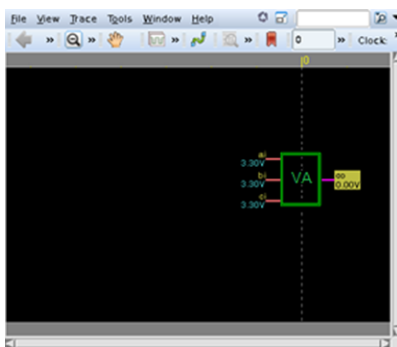
If SPICE signals are in Verilog-A's modules, the symbols for the function blocks is **VA** instead of SPICE.

Creating Verilog-A Signals in TFB

Creating TFB for Verilog-A signals is similar to creating TFB for a digital black box. TFB uses function blocks with the specific word "VA" to represent Verilog-A signals. The analog blocks are shown in green in TFB.

The following figure shows the initial window for generating TFB for a Verilog-A signal. If an internal analog signal is selected for creating TFB, an output analog signal is mapped. For example, if a signal in the analog block and the signal is neither an input nor an output, an output pin under the same scope is mapped, so that the final viewing of the internal signal is displayed (see [Figure: Verilog-A Block in TFB](#)).

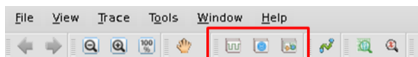
Figure 141 Verilog-A Block in TFB



Synchronization with nWave, nTrace, and nSchema

You can use the synchronization buttons in the TFB toolbar to synchronize analog signals to *nWave*, *nTrace*, and *nSchema*. Figure [Synchronization from TFB](#) shows the synchronization buttons in TFB.

Figure 142 Synchronization from TFB



The order from left to right is the sync button to *nWave*, *nTrace*, and *nSchema*.

Dragging and Dropping Signals

You can drag and drop Verilog-A signals to *nWave*, *nTrace*, and *nSchema*. When you drag Verilog-A signals from *nWave*, *nTrace*, and *nSchema*, TFV adds the dragged signals as reference signals and shows the respective analog blocks.

Force debug for Verilog-A signals

TFV does not display any information about internal signals of analog modules, so the force debug information of analog internal signals is not displayed in TFV.

Limitation

TFV cannot trace into analog instances. Also, tracing the internal signals of analog blocks is not supported. If you create TFV for the internal signals, TFV is created with an out-most signal instead of the internal signals. Detail structures can be found by toggling up the sync button of *nSchema*. That is, you can trace through analog blocks and see the structural details if required.

5

Known Problems and Limitations

- For **spicom**
 - The `spicom` utility does not support inline `subckt` for Spectre format.
 - The `spicom` utility does not fully support global net. Trace connective feature is not supported for global nets.
 - The `spicom` utility does not fully support global parameter, such as showing the definition feature.
 - The `port_map` option for the `use_verilog`, `use_spice`, and `use_vhdl` commands is not supported.
- For AMS virtual FSDB file
 - The digital FSDB file should be set as the *master file* which is the first FSDB file in the virtual file.
 - Verdi only supports the virtual file that contains one digital FSDB file and one analog FSDB file for AMS debugging.
 - If there are some scopes with the same name and different letter cases in the digital FSDB file, the referenced scope in digital FSDB for merging the analog FSDB file is randomly chosen.
- For nWave
 - Case Issue

For example, Signal A and v(a) are dumped in Verilog scope.

Drag-and-Drop and synchronize the selection may not work correctly for v(a) since the signal name in the design is A.
- For nScehma
 - TFV related features are not supported for value types signals.
- For AMS debug
 - The Rebuild and Restart flow is not supported in Mixed Signal Verification flow.

- Support for editing the Interface Elements (IE) in Verilog-AMS
 - The runtime interface elements and `inout` interface elements are not supported.
- Support for Verilog-AMS in Verdi
 - You cannot add node current of the primitive device in Verilog-AMS instance. For example, if the following statement is available in Verilog-AMS `resistor` `#(.r(10k)) R2 (b, gnd);` then the annotation for `b` or `gnd` signals are not displayed and are unable to add node current when drag-and-drop `b` or `gnd` signals to waveform.
 - Since XA cannot dump voltage for unnamed branch, it is not recognized in *nWave*.
 - Unnamed branch does not support DND from *nWave* to design-based AP.
 - You cannot dump "single" port current if a Verilog-AMS instance is leaf instance.
 - You cannot dump voltage of unnamed branch.